

Personal Radio: Migration Package

One file with everything: what the project is, where it stands, how to move it to your PC, what is left to do, and (in the technical parts at the end) the full design history and research. Written 8 October 2026.

Who this is for. The first half (sections 1–9) is written for a non-technical reader. The second half (Parts A–I) is the complete technical record, kept as it was produced, for anyone (including a future AI assistant) who needs the evidence behind a decision. You can stop reading after section 9 and still run the project.

Contents

Section	What it covers
1. What this project is	Plain-language purpose and the idea in one paragraph
2. Where things stand	What works, what does not, real numbers
3. Moving to your PC	A checklist, including what to copy from the Mac
4. What you still need to do	Decisions and chores only you can do
5. What comes next	The remaining plan, in order
6. Things that surprised us	Known quirks and risks
7. Command and folder map	Old names versus new names (the technical parts use the old ones)
8. Words used in this project	Glossary
9. The ideas, illustrated	Fourteen diagrams that explain the core concepts
Parts A–I	The technical record: original handoff, session logs, research reports, research notes, research prompts, build log

1. What this project is

Personal Radio is a private program that runs on your own computer. It quietly watches what you play on Spotify (from any of your devices), works out which songs you liked and which you skipped, and every week builds you a private playlist of about 30 songs you have not heard. It finds those songs using free, public music databases rather than Spotify's own recommendations, because Spotify closed those off to apps like this one.

The main goal is simply a better weekly playlist for one person. A second goal is to measure honestly whether it is actually better than what Spotify already gives you, which is why a share of each playlist is deliberately random: it is how the tool tests itself.

The documents in `docs/` explain it in more detail: `GETTING_STARTED.md` (setup), `HOW_IT_WORKS.md` (the logic), `PRIVACY.md` (what is stored and Spotify's rules).

2. Where things stand

Working

- **The watcher.** It records plays from Spotify, including from your phone. It checks less often when idle, catches up on anything missed once an hour (and right after the computer has been asleep), and writes a "still alive" signal you can check with `python -m radio health`.
- **Likes and dislikes.** Finished songs count as likes, early skips as dislikes. A save only counts as proof for plays *before* the save. Autoplay counts as much as hand-picked plays.
- **Song identification.** About 80% of your songs are matched to the public music databases (the research found no published match rate to compare this with).
- **The weekly playlist.** It was written to your Spotify on 8 October as the private playlist "Personal Radio - Weekly Auto" (30 songs: 15 close matches, 6 deep cuts from artists you like, 6 explore picks, 3 random wildcards). Every pick's reason and selection chance is recorded.
- **Privacy tools.** Old raw data is deleted automatically; one command erases everything Spotify-derived.
- **Self-checks.** 58 automatic tests pass.

Not working yet, or not proven

- **Your full history is not loaded.** The tool has only about two days of data: 110 plays, of which only 19 say whether you finished or skipped (91 came from Spotify's "recently played" list, which does not say). That is far too little to measure anything. Your Extended Streaming History would fix this (section 4).
- **Importing that history is untested** on a real file, because you do not have the file yet.
- **The playlist does not refresh itself.** You run `python -m radio playlist --write` by hand each week.
- **The Windows PC is not set up.** The watcher has been running on the Mac, which sleeps; that is why recordings have gaps.
- **Only about 4 in 10 of your songs have "listeners also played" data** in the public database, so the pool of recommendations is small (81 candidate songs for the last playlist).
- **The accuracy numbers do not exist yet.** The scoring tool is built but needs weeks of data.

3. Moving to your PC

Checklist

1. **On the Mac:** stop the watcher if it is running (`Ctrl+C` in its window). Never run two at once on the same Spotify account.
2. **On the Mac:** copy the whole `personal_radio` folder to a USB drive or cloud drive. It contains `data/` (your history and login) and `config/`. Do **not** email it or put it somewhere public: `data/` is private.
 - If you are using the GitHub copy instead, the code comes from GitHub, but you must still copy the `data/` and `config/` folders by hand from the Mac, because Git deliberately does not contain them.
3. **On the PC:** install Python 3.11 or newer, ticking "Add Python to PATH".
4. **On the PC:** put the project folder somewhere permanent (for example `C:\PersonalRadio`).
5. **On the PC:** follow `docs/GETTING_STARTED.md`, Parts 1–2. Have your Spotify Client ID and Client secret ready (the Spotify developer dashboard shows them). They are not stored in the project.
6. **On the PC:** run the one-time Windows task installer so the watcher starts at login.
7. **On the PC:** turn off Sleep while plugged in.
8. After a day, run `python -m radio health`. It should report `ok`.

What does and does not travel

Item	Travels in the folder copy?
The program code	Yes (also on GitHub)

Item	Travels in the folder copy?
Your listening history and recommendation log	Yes, in <code>data/</code>
Your Spotify login token	Yes, in <code>data/.spotify_cache</code> (it should keep working; if not, approve in the browser again)
Client ID and Client secret	No. Re-enter them on the PC.
Which playlists count as your taste	Yes, in <code>config/</code>
The playlist already in Spotify	It is in your Spotify account, not on a computer

4. What you still need to do

These are the chores and decisions that only you can do:

1. **Request your Extended Streaming History.** spotify.com/account/privacy, "Download your data", tick *Extended streaming history*, confirm via the email link. Can take up to 30 days. This is the most valuable single thing you can do for the project. (Step by step: `docs/GETTING_STARTED.md`, Part 4.)
2. **Read Spotify's developer policy yourself** (developer.spotify.com/policy), especially rule III.13: "Do not analyze the Spotify Content or the Spotify Service for any purpose". The research flagged this as the biggest risk to the project. We only read a summary, not the original wording. The practical danger is Spotify switching off your developer key, not a lawsuit. Decide whether you are comfortable. (`docs/PRIVACY.md`.)
3. **Replace your Spotify Client secret.** It was pasted into a chat during development. In the Spotify developer dashboard, open your app's Settings and rotate the secret. Then use the new one on the PC. A chat is not a safe place for a secret.
4. **Get a free ListenBrainz token** (listenbrainz.org, Settings). It makes the popularity lookups reliable. The public service sometimes answers "unauthorized" without one, and the program copes, but a token is better.
5. **Tell us whether you deleted the playlist.** The tool's playlist has vanished from your account twice (once within hours of being written and confirmed). The cause is unknown. If you deleted it yourself, no action is needed. If not, it is probably a Spotify quirk; the program notices and recreates it, but please report it.
6. **Decide how long to keep your history.** By default, plays and saves are kept forever and only the raw snapshots expire after 30 days. If you would rather expire everything, say so.

5. What comes next

In order of value:

1. **Import the Extended Streaming History** when it arrives, then use it to give every artist and genre you have heard a sensible starting score. This replaces the thin two-day dataset.
2. **Run the playlist on a schedule** (weekly, automatically) once the PC is the home of the watcher.
3. **Add the missing popularity correction:** make each playlist's mix of famous, middling and obscure songs match your own listening.
4. **Run an honest comparison against Spotify's Discover Weekly:** copy that week's playlist by hand into a playlist you own, shuffle both together, and score only songs you had not heard.
5. **Later, only if the data justifies it:** smarter exploration (a method called Thompson sampling), a learned ranker (needs roughly 2,000–5,000 labelled plays), and the "inject songs into the queue to learn about a genre" idea. The second research round advised against that last one for now: it would add about 2 percentage points to how many playlist songs you finish at best, and would take years to even detect.

6. Things that surprised us (and risks)

- **Spotify removed most of its recommendation features** for new apps in 2024–2026 (audio features, related artists, "top tracks", popularity scores). Everything here is built around that.
- **Batch song lookups return an error** ("403") for apps in Development Mode, so songs are looked up one at a time.
- **A successful "write" to a playlist is not proof it stays.** Playlists vanished after writes that Spotify had accepted; the cause is unknown. The program reads the playlist back after writing and recreates it if needed.
- **Laptops sleep.** That is why the watcher belongs on an always-on PC.
- **Private Session** plays in Spotify are invisible to the program.
- **A few dozen plays cannot teach a model anything.** Early results will look bland or odd. That is expected.
- **Rule risk.** See section 4, item 2.
- **MusicBrainz sometimes answers "busy" (503).** The program waits and retries; run `python -m radio identify` again later to finish.

7. Command and folder map

The project was reorganised on 8 October 2026. The technical parts below were written before that and use the old names. Translate them with this table.

Old command (in the old text)	New command
<code>python -m radio.run</code>	<code>python -m radio track</code>
<code>python -m radio.poller</code>	<code>python -m radio track</code>
<code>python -m radio.health</code>	<code>python -m radio health</code>
<code>python -m radio.ids</code>	<code>python -m radio identify</code>
<code>python -m radio.playlist</code>	<code>python -m radio playlist</code>
<code>python -m radio.history_import FILE</code>	<code>python -m radio import-history FILE</code>
<code>python -m radio.evalkit.run</code>	<code>python -m radio evaluate</code>
<code>python -m radio.retention</code>	<code>python -m radio privacy</code>
<code>python -m radio.smoke</code>	<code>python -m radio login-check</code>

Old file	New file
<code>radio/tracker.py</code> , <code>poller.py</code> , <code>sync.py</code> , <code>history_import.py</code> , <code>health.py</code> , <code>run.py</code>	<code>radio/collect/...</code>
<code>radio/labels.py</code> , <code>pipeline.py</code> , <code>basis.py</code> , <code>ids.py</code>	<code>radio/taste/...</code>
<code>radio/playlist.py</code> , <code>radio/candidates/</code>	<code>radio/recommend/...</code>
<code>radio.duckdb</code> , <code>radio_ids.duckdb</code> , <code>radio_recs.duckdb</code> , <code>.spotify_cache</code> , <code>.api_cache</code> , <code>heartbeat.json</code> , <code>playlist_state.json</code> , <code>radio.log</code> (project root)	the same names inside <code>data/</code>
<code>taste_basis.json</code> (project root)	<code>config/taste_basis.json</code>
<code>install_windows_task.ps1</code>	<code>scripts/install_windows_task.ps1</code>

New file locations are all decided in one place, `radio/paths.py`. Setting the environment variable `RADIO_HOME` moves the whole `data/` folder elsewhere.

Other documents were folded into this one: the original handoff, the session logs, both research reports, the six research notes, both research prompts, and the first README. A plain-text copy of the first report (word-for-word identical to Part C) was dropped as a duplicate.

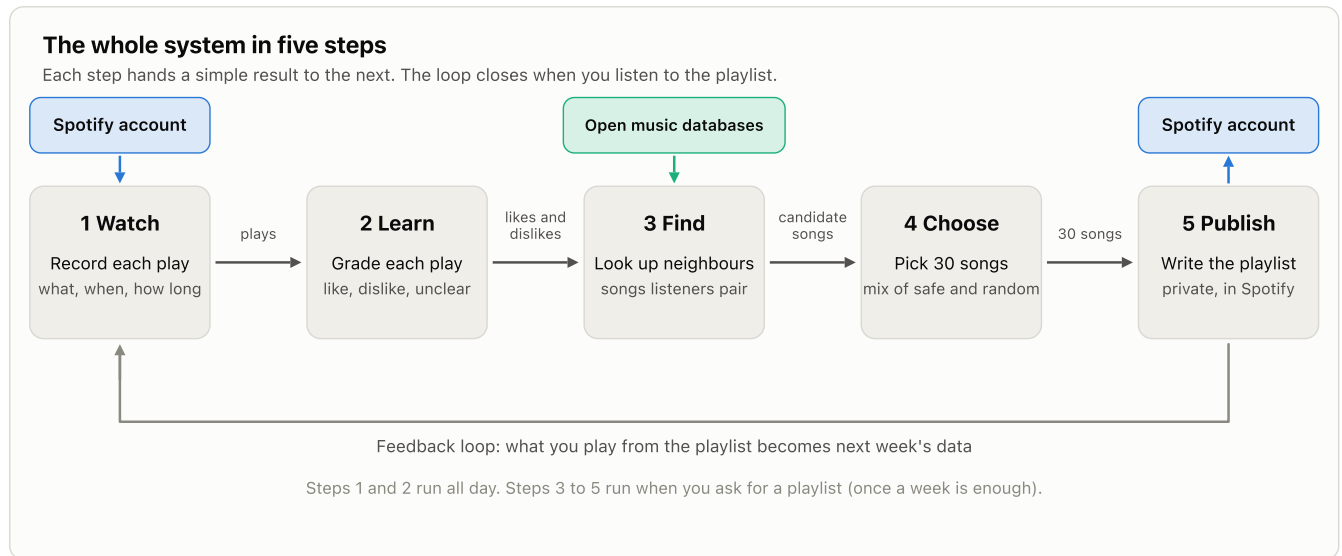
8. Words used in this project

Word	Meaning
API	The official way for a program to talk to a service such as Spotify
Developer app	The registration that gives this program permission to use Spotify's API
Poller, watcher, tracker	The background program that checks what is playing
ISRC	A code identifying one specific recording of a song, like a barcode
MBID	A MusicBrainz ID: a permanent code for a song or artist in the public database
MusicBrainz, ListenBrainz	Free, non-profit music databases (MetaBrainz Foundation)
Label	The program's judgement of one play: like, dislike, or unclear
Seed	One of your songs or artists that the search for new music starts from
Propensity	The recorded chance that a song would be picked for the playlist
Baseline	A deliberately simple method (such as "replay my recent favourites") that a fancy method must beat
Explore / wildcard	Playlist slots filled partly or wholly at random, so the tool can test itself
Rolling-origin split	Testing on the future only: train on the past, test on what came next
Extended Streaming History	Your full listening record, which Spotify sends you on request
Cache	A saved copy of an answer, so the program does not ask twice
Heartbeat	A small file the watcher keeps updating so you can tell it is alive
DuckDB	The simple database format used for the files in <code>data/</code>
Thompson sampling, LightGBM, DPP, MMR, IPS	Advanced techniques named in the research. Only matter if you read Parts C and D

9. The ideas, illustrated

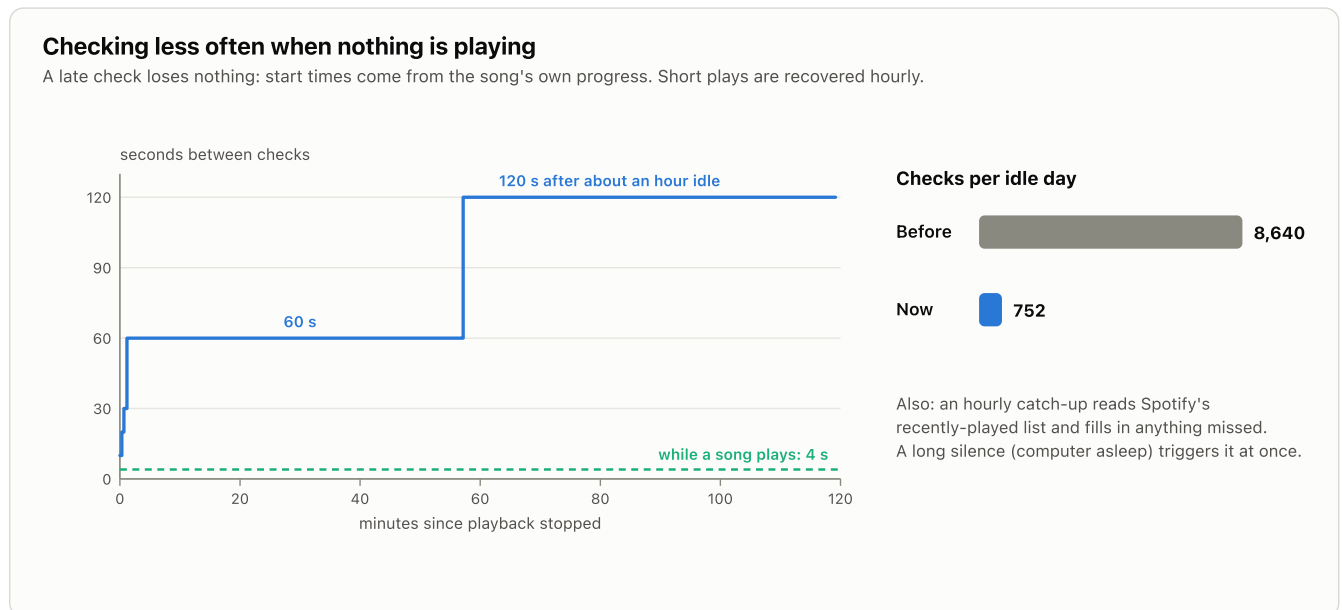
Each figure explains one idea. They are generated from the real program logic by `scripts/make_diagrams.py`, so the numbers match what the code does.

9.1 The whole system



Five steps run in a loop. The program watches your Spotify plays, grades them, looks up similar songs in open databases, picks 30, and writes a private playlist. What you play from that playlist becomes next week's data.

9.2 Watching without wasting effort

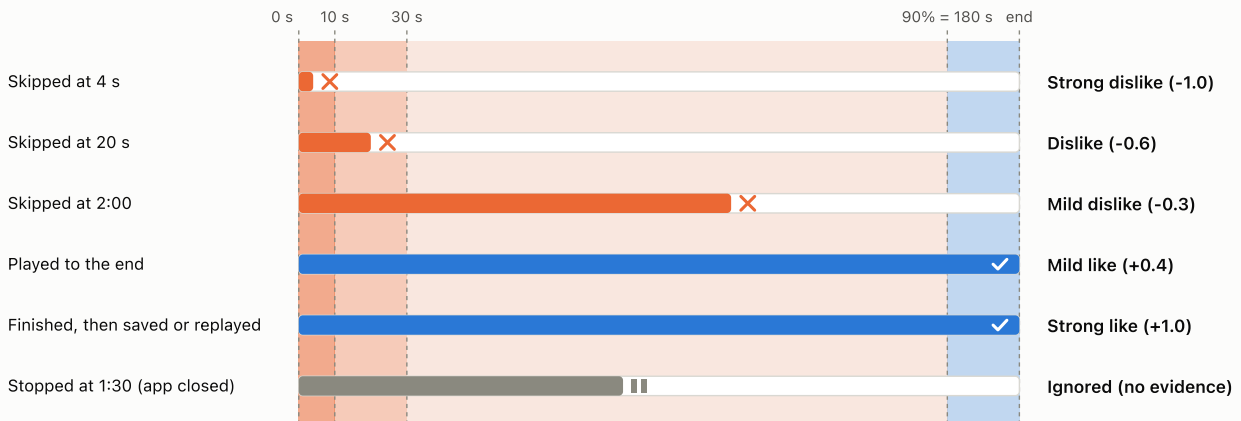


The program asks Spotify what is playing every 4 seconds while a song plays. When nothing plays it slows down to once a minute, then once every 2 minutes. This cuts checks from about 8,600 to about 750 on an idle day. Nothing is lost, because a song's start time is worked out from how far into it you were when the program noticed. Once an hour, and right after a long silence such as the computer sleeping, it reads Spotify's "recently played" list to fill any gaps.

9.3 From a play to a like or a dislike

How one play becomes a like or a dislike

Example song: 200 seconds long. Bars show how far each play got.



Thresholds: under 10 s is a strong dislike, 10 to 30 s a dislike, a later skip a mild one. Finished means 90% or more of the song.

The program never asks you to rate anything. It reads behaviour. An early skip is a strong dislike, a late skip is a mild one, and finishing is a like. Stopping because the app closed says nothing, so it is ignored. Saving or replaying a song after you finish it makes the like strong.

A save only counts for plays that came before it

Why: saving a song you have loved for years says nothing about one particular play today.

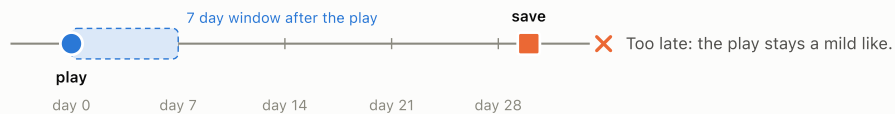
A. Play, then you save it 2 days later



B. You saved it long before the play



C. Play, then you save it 30 days later



● a play ■ a save (added to Liked Songs)

Timing matters. A save counts as proof only if it happens within 7 days after the play. A song saved a year ago and played today tells the program you already knew the song, not that this play was good. Counting it would inflate the scores of old favourites and make the program look better than it is.

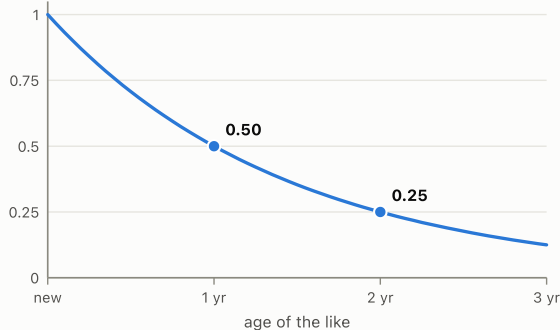
9.4 How much each like counts

Two ways the taste profile avoids being swamped

Both are simple curves, shown here exactly as the program computes them.

1. Older likes count a little less

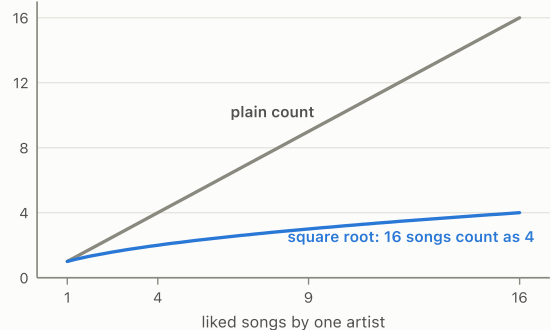
Weight of a liked song by its age (half-life: 1 year)



Result: an artist with 16 liked songs has 4 times the pull of an artist with 1, not 16 times.

2. One artist cannot take over

Artist weight by number of liked songs



Two simple curves shape your taste profile. Older likes fade: a like loses half its weight each year. And an artist's total pull grows with the square root of the number of liked songs, so 16 liked songs give 4 times the pull of one, not 16 times. Without this, one heavily saved artist would decide the whole playlist.

9.5 Matching songs across services

Matching one song across services

Spotify, MusicBrainz and ListenBrainz label songs differently. A recording code is the bridge.



How many songs matched (172 songs in the library):



Unmatched songs are not used as starting points for finding new music. This is why the candidate pool is smaller than the library.

The codes shown are examples. Coverage depends on how well MusicBrainz knows each recording.

Spotify, MusicBrainz and ListenBrainz each name songs differently. A recording code (ISRC) is the shared key. The program converts each Spotify song to an ISRC, then to MusicBrainz IDs. About 79% of the library matched. Unmatched songs cannot be used as starting points.

9.6 Finding similar songs without Spotify

Where 'similar songs' come from

No audio analysis is involved. Neighbours are songs that other listeners play in the same sitting.

Step 1: other people's listening sessions

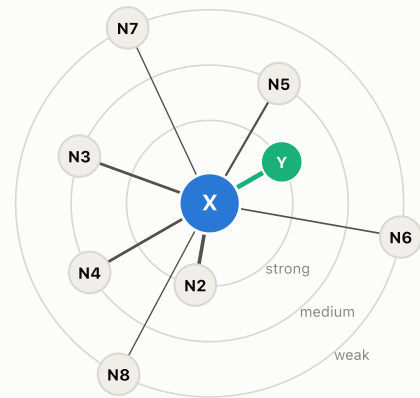


X = your song Y = a candidate

X and Y appear together in 2 of the 3 sessions.

Repeated over millions of sessions, this gives each song a ranked list of neighbours with a score.

Step 2: the neighbour map for X



Line thickness = how often the pair is played together

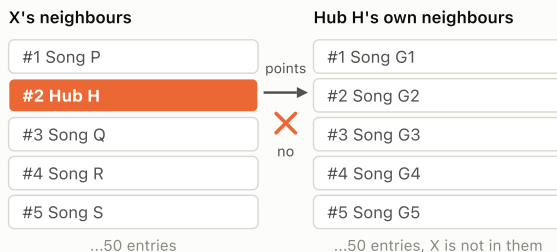
There is no audio analysis. ListenBrainz publishes which songs people play in the same listening session. If song X and song Y keep showing up together, Y is a neighbour of X. The closer the ring in the map, the more often the pair is played together.

9.7 Why famous songs do not dominate

The two-way check removes hub songs

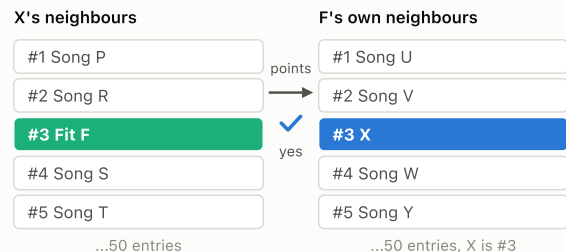
A candidate only counts as similar if it also points back at your song.

A hub song (listed by everyone)



Forward score: 0.98 (rank 2 of 50)
Backward score: 0.10 (X not in the hub's list)
Combined = $\sqrt{0.98 \times 0.10} = 0.31$ (low)

A good fit



Forward score: 0.96 (rank 3 of 50)
Backward score: 0.96 (X is #3 in F's list)
Combined = $\sqrt{0.96 \times 0.96} = 0.96$ (high)

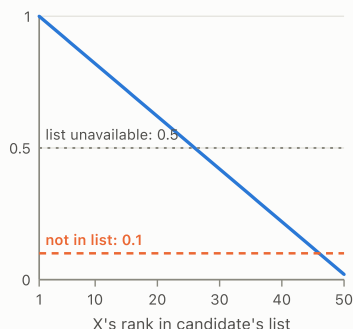
Some songs are neighbours of almost everything. These are hubs. A candidate is trusted only if it also points back: your song X must appear in the candidate's own neighbour list. Hubs fail this check. Good fits pass.

Three corrections applied to every candidate

Each one lowers the score of a song that looks similar only because it is everywhere.

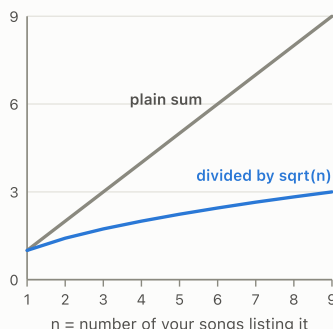
1. Two-way check

Backward score by X's rank in the candidate's list



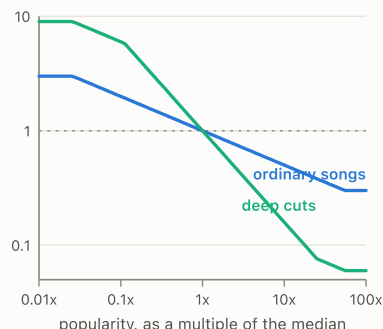
2. Breadth discount

Score of a song listed by n of your songs



3. Popularity discount

Multiplier by popularity vs the typical candidate



Example: a song 10 times as popular as the median keeps 50% of its score. A deep cut that popular keeps about 16%.

Three corrections adjust every score. The two-way check scales the score by how high your song ranks in the candidate's list. The breadth discount stops a song from winning only because many seeds list it. The popularity discount lowers the score of very well-known songs, and lowers it more for deep cuts so they are actually less famous.

9.8 Choosing the 30 songs

How the 30 playlist songs are chosen

Four slots with different jobs. The final order is shuffled.

The playlist, grouped by slot



The same 30, shuffled into play order

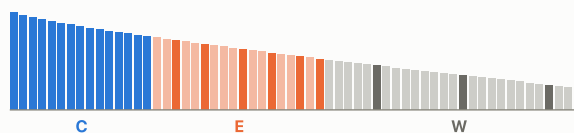


- C** Confident: 15 songs (50%)
- D** Deep cut: 6 songs (20%)
- E** Explore: 6 songs (20%)
- W** Wildcard: 3 songs (10%)

Shuffling means position says nothing about how a song was chosen.

Where each slot draws from

New-artist candidates, best score on the left



C Confident

The top 15 by score, always included.

D Deep cut

6 drawn at random from the best 18 songs by artists you already like.

E Explore

6 drawn from the next 18 new songs. Higher scores are more likely.

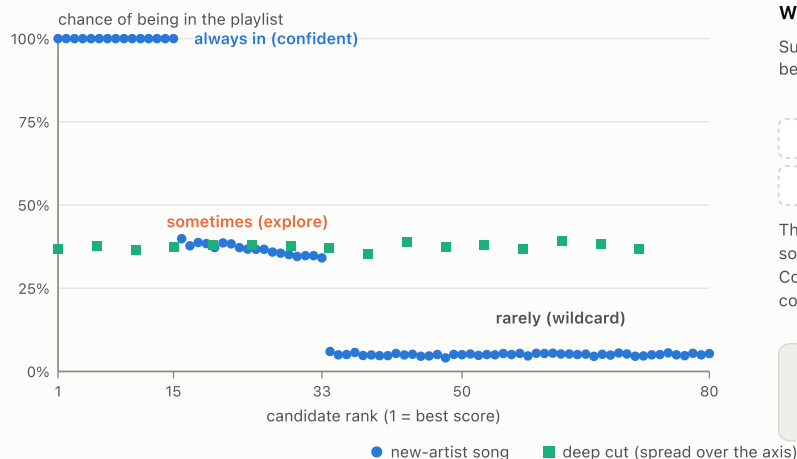
W Wildcard

3 drawn evenly from everything left. Tests what the scores miss.

The playlist has four kinds of slots. Confident picks (15) are the best scores. Deep cuts (6) are songs you have not heard by artists you already like. Explore picks (6) are drawn at random with better scores more likely. Wildcards (3) are drawn evenly from the rest. The order is shuffled so position never reveals how a song was chosen.

Every pick has a recorded chance of being chosen

Measured by running the real playlist builder 3000 times on 80 example songs plus 16 deep cuts.



Why the chance is recorded

Suppose a song had a 1 in 20 chance of being picked, and you loved it.



The one shown stands in for roughly 20 similar songs that could have been shown but were not. Counting it 20 times (1 divided by its chance) corrects for the picks you never saw.

Without the recorded chance, the confident picks would always look better than they are.

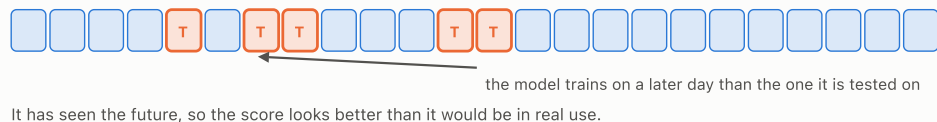
The program records how likely each song was to be chosen. This chart comes from running the real playlist builder 3,000 times. Confident songs are always in. Explore songs appear about a third of the time. Wildcards appear rarely. Recording the chance allows a fair measurement later: a rare pick that you loved is counted as standing in for about 20 similar songs that were never shown. Without it, safe picks would always look better than they are.

9.9 Checking honestly whether it works

Testing on the future, not on a shuffle

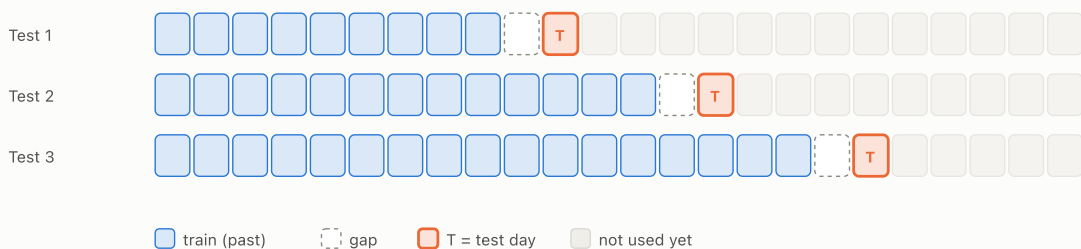
Each square is one day of listening. Time runs left to right.

Random split (misleading)



It has seen the future, so the score looks better than it would be in real use.

Rolling split (what this project uses)



A random split trains on later days and tests on earlier ones. That leaks the future and flatters the result. The program tests on the future only: train on the past, leave a short gap, test on the next day, then slide forward.

Scoring a ranked list: why position matters

NDCG@10 gives more credit to a hit near the top. Both lists below contain the same two liked songs.



Scores (best possible = 1.00)

List A hits at positions 1 and 4



List B hits at positions 7 and 9



Best case for two liked songs: positions 1 and 2.
Credit at position i is $1 / \log_2(i + 1)$, then divided by that best case.

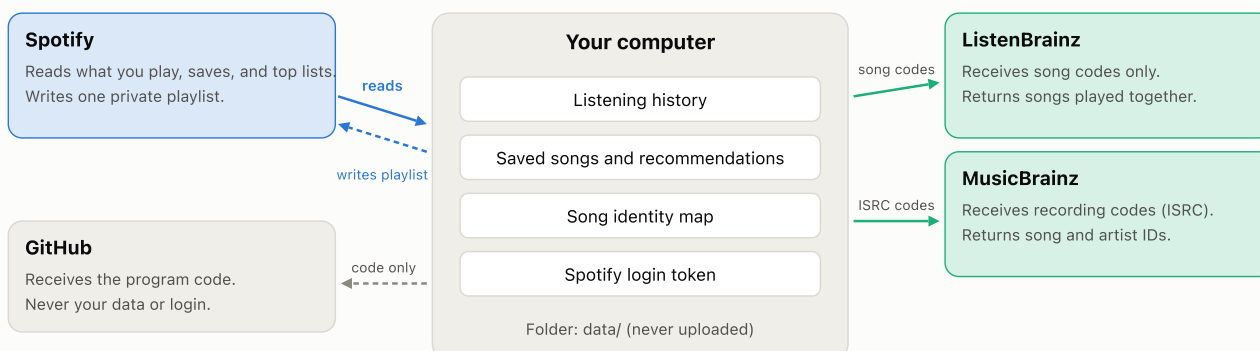
Why it matters: a playlist is read from the top, so late hits are worth less.

Scores use a measure called NDCG. It gives more credit to a liked song near the top of a list. Two lists with the same two liked songs score 0.88 and 0.39 depending on where the hits sit. The recommender must beat simple baselines, such as replaying recent favourites, before it is trusted.

9.10 What leaves your computer

What stays on your computer and what is sent out

Arrows show requests leaving your computer and the kind of data they carry.



The open databases receive song and artist codes, not your listening history or account details.
If you set RADIO_CONTACT, that contact is included in requests to MusicBrainz, as they ask.

Your history, saves and Spotify login stay in the `data/` folder. The program sends requests to Spotify, to ListenBrainz (song codes) and to MusicBrainz (recording codes). GitHub receives only the code. See [PRIVACY.md](#).

THE TECHNICAL RECORD (Parts A–I)

Everything below is the original material, kept as produced. It is aimed at engineers and AI assistants. Command and file names in it are the pre-reorganisation ones: use the map in section 7. "Part 0" in the text refers to the earlier front section, which is now Part I, the build log.

Part	What it is	Read when
A	Original handoff: goal, constraints, design, code inventory, roadmap	Need project background
B	Session log: what was verified, bugs fixed, code added	Need to know why something is the way it is
C	Research report 1: final synthesis, with links	Making design decisions
D	Six research notes: evidence, grades, gaps	Need citations or nuance
E	Research prompt 1	Re-running the research
F	The old package README	Historical only; superseded by README.md and docs/
G	Research prompt 2	Running the next research round
H	Research report 2: labels and candidates before probing; the verdict on queue injection	Planning the next build stage
I	Build log: status snapshots, research-integration matrix, and every change made 7–8 October 2026	Need the detailed history

PART A — Original handoff (as written at the start of the project)

Handoff: Personal Radio — personal Spotify recommender

For Claude Code. Date of original work: 2026-10-07. Code lives in `personal_radio/` (this folder's subdirectory). Re-verify anything marked **[VERIFY]**: Spotify's API/policies change fast.

1. Goal

A personal-use tool that tracks the user's Spotify listening in real time, learns from their behavior, and produces better personal song recommendations than Spotify's own, ending in new tailored songs written into a Spotify playlist. User framed it as "imitation learning" of their listening. Wants practical, buildable output.

2. User context (answered)

- Listens on **Windows desktop (most)** and **iPhone**. Server-side polling sees both; iOS has no device-side hook.
- Wants tracking active whenever listening, **auto-start at logon, ingest from startup**.
- Wants **artists they search for and deliberately play** to count as a signal, plus any other usable Spotify signals.
- Assumed (not confirmed, ask if it matters): discovery-first priority; runs on the Windows desktop (not Pi/VPS); **no Extended Streaming History or ListenBrainz/Last.fm account confirmed yet**.
- Project saved at `~/personal_spotify_rec`. Dev machine is a Mac; the tracker is meant to run on the Windows PC.

3. Verified constraints (Oct 2026)

- Nov 2024: `/recommendations`, `/audio-features`, `/audio-analysis` removed for new apps. No replacement.
- Feb 2026: Development Mode needs **Premium on the app owner**; 5 allowlisted users; search `limit` max reportedly cut to 10 (third-party report).
- Jul 23 2026: up to 25 Client IDs per developer account, sharing one quota; 429s now carry `QUOTA_EXCEEDED` reason. No numeric rate limit published. Refresh tokens expire after 6 months. Sources: developer.spotify.com/blog/2026-07-23-web-api-quota-updates, [.../references/changes/july-2026](https://vorplabs.com/agent-tools/spotify-api-changes), vorplabs.com/agent-tools/spotify-api-changes.
- `preview_url` is null for new apps; don't depend on it.
- **ML-training clause [VERIFY against primary text]**: Developer Policy reportedly forbids using Spotify Content to train/ingest into ML models and limits storing Spotify data beyond need. Only secondary sources checked. Mitigation: store minimal Spotify metadata (URI, ISRC, names); take features/candidates from open data; treat user behavior labels as the training signal (gray area, unresolved).
- **No search-history API**. Searches typed in the app are not exposed.
- No playback webhooks (polling only). Queue: can add, cannot remove/reorder. Playback control needs Premium + active device.
- Unknown whether Feb 2026 endpoint removals (per Vorp Labs) affect top-items/follows/saved: sync steps are isolated so failures don't stop tracking. Check on first real run.

4. Design principles

1. Spotify = player/catalog, not the feature/recommendation source.
2. Open data (ListenBrainz, Last.fm, MusicBrainz) does candidate generation.
3. User behavior (skips, completes, repeats, saves, hand-picked plays) = ground truth labels.
4. Cheap online adaptation, slow offline learning (nightly retrain).
5. Modular; keep own copy of all data.
6. Exploration is mandatory (pure imitation reproduces current taste via exposure bias).

5. What is built (`personal_radio/`)

Python, DuckDB, spotipy. **Tests: 10 stdlib unittest tests pass** (`python3 -m unittest discover -s tests -t . from personal_radio/`). **Not yet run against real Spotify, DuckDB, or Windows** (the authoring sandbox had no network, so `duckdb/spotipy` were never installed; `.venv` was not copied). First job: install deps, run for real, fix what breaks.

File	Role
<code>radio/tracker.py</code>	Pure state machine: playback snapshots → finished plays. Handles completed/skipped, repeats (progress rewind), pauses, seek counting, podcasts ignored, stop-near-end = completed. Tags <code>start_kind</code> (user/auto/nav: auto if same context continues within 30s of a completed track, nav after skip, else user) and <code>context_type</code> (artist/album/playlist/collection/None).
<code>radio/poller.py</code>	Spotify polling loop (<code>current_playback</code>): 4s playing / 8s paused / 10s idle / 1.5s near track end; 429 <code>Retry-After</code> handling; exponential backoff on errors; writes <code>raw_events</code> + <code>plays</code> . OAuth cache at <code>.spotify_cache</code> , scopes: playback-state, currently-playing, recently-played, library-read, follow-read, top-read.
<code>radio/sync.py</code>	Startup/periodic ingest: <code>backfill_recent</code> (last-50 recently played newer than latest stored play, <code>end_reason='other'</code>), <code>sync_saves</code> , <code>sync_follows</code> , <code>sync_top</code> (short/medium/long). Each step isolated by try/except.
<code>radio/run.py</code>	Supervisor: startup sync, 6-hourly resync thread, poller loop restarted on crash; logs to <code>radio.log</code> . Uses one DuckDB connection with a <code>cursor()</code> per thread.
<code>radio/history_import.py</code>	Imports Extended Streaming History (zip or dir of <code>Streaming_History_Audio*.json</code>). Maps <code>reason_end</code> → completed/skipped/other and <code>reason_start</code> → start_kind. Podcast rows dropped.
<code>radio/labels.py</code>	Graded labels: <code>strong_pos</code> +1.0 (completed + saved/repeated), <code>weak_pos</code> +0.4 (completed), <code>strong_neg</code> -1.0 (skip <10s), <code>neg</code> -0.6, <code>weak_neg</code> -0.3 (skip ≥30s), None for incognito/other. <code>skip_streaks()</code> .
<code>radio/db.py</code>	Schema: <code>raw_events</code> , <code>plays</code> (PK source+started_at+track_uri), <code>saves</code> , <code>follows</code> , <code>top_items</code> , view <code>intent_plays</code> (user-started plays from artist/album context or lone tracks = proxy for "searched and chose").
<code>install_windows_task.ps1</code>	Registers hidden logon scheduled task "PersonalRadio" (30s delay, restart x99, pythonw). Untested.
<code>README.md</code>	Setup and Windows instructions.

Known gaps / things to check first

- `plays.source='recent'` rows have no `ms_played`; `label_play` returns None for `other`, fine, but dedupe vs poller rows is only by "newer than latest play".
- `saved/ repeated` args to `label_play` are not yet wired to the `saves` table or repeat detection; no pipeline yet joins plays→labels.
- `incognito`: poller always writes False (Private Session plays are invisible to the API).

- History imports can't distinguish search-click from playlist-click (`clickrow` covers both); `context_type` is NULL for history rows.
- Track identity: nothing resolves to ISRC/MBID yet (poller stores ISRC when present; history rows have none).
- DuckDB single-writer: if the user opens the DB from another process while the tracker runs it will lock; plan a read-only copy or small query CLI.
- Timestamps stored as naive UTC.
- Token refresh/6-month expiry and sleep/wake behavior untested.

6. Roadmap (my intended plan, in order)

Phase 2 — candidates + baseline scoring + first playlist (next):

1. `radio/ids.py`: identity resolver (Spotify URI ↔ ISRC ↔ MusicBrainz MBID), cached in DuckDB tables; back-fill ISRC for history rows via batched `tracks` lookups (mind quotas).
2. `radio/pipeline.py`: join plays→labels (wire `saves`, repeat detection, `start_kind` weights; weight `intent_plays`, follows, top items higher as taste signals).
3. `radio/taste.py`: taste profile — 5–15 clusters over liked tracks/artists (tag/metadata embeddings from MusicBrainz/Last.fm tags), artist familiarity/novelty scores.
4. `radio/candidates/`: generators — ListenBrainz recommendations/similar, Last.fm `track.getSimilar`/`artist.getSimilar`, MusicBrainz relationships; dedupe; drop heard-and-disliked and already-known. Cache API responses; respect rate limits and API ToS.
5. `radio/resolve_spotify.py`: map candidates to Spotify URIs (ISRC search preferred; small search limits → cache).
6. Heuristic scorer: cluster similarity + familiarity/novelty + intent boost; no ML yet.
7. `radio/playlist.py`: nightly job writing "Weekly Auto" playlist (replace items): ~70% confident / 20% exploration / 10% wildcard; every pick logged with its reason for the dashboard.
8. Loop closes automatically: the poller logs skips/completions on those tracks → labels.

Phase 3: LightGBM ranker on own labels (features: cluster similarity, artist familiarity, tag overlap, popularity, time of day/day of week, device, similarity to last few tracks, `start_kind`); time-split evaluation (skip-prediction AUC, hit rate on later-saved tracks). **Phase 4:** exploration policy (Thompson-sampling bandit), feedback loop, blind A/B vs Discover Weekly (shuffle both sources into one playlist, rate without knowing source). **Phase 5:** session engine (exponentially weighted recent positives/negatives; 3 consecutive skips ⇒ pivot) + opt-in "Smart Radio" one-track-lookahead queue controller (Premium + active device; can't edit queued items). **Phase 6:** local FastAPI dashboard (why each track, taste clusters, skip rate by source, thumbs up/down overrides, diversity/novelty metrics). **Phase 7:** audio embeddings (only from legitimately sourced audio), SASRec-style sequence model pretrained on public data (LFM-2b, Yambda) then fine-tuned, optional LLM natural-language steering ("darker this week"). Optional: Windows UI-Automation watcher on Spotify's search box (fragile; deliberately not built), Windows SMTG gap-fill, redundant scrobbling to ListenBrainz/Last.fm.

7. Assessment given to the user (keep expectations honest)

- Capture/labels: highly feasible. Open-data candidate generation: weakest link (small data vs Spotify's CF; ID mapping is tedious). Ranker on own labels: needs a few thousand labelled plays — Extended History gives that on day one; otherwise 2–4 weeks of tracking.
- Will NOT beat Spotify's collaborative filtering generally. Plausibly beats Discover Weekly on hit rate for this user by exploiting: hand-picked intent signals, discovery focus (fixed exploration budget), live session adaptation, steerability/explainability. Must be proven by the blind A/B, not assumed.
- Main failure modes: exposure bias/taste narrowing, bad labels (skip ≠ dislike), ToS gray area, API lockdowns, always-on host requirement (sleeping PC = no data), Private Session/offline plays invisible.

8. Immediate to-dos for Claude Code

1. `cd personal_radio && python -m venv .venv && pip install -r requirements.txt`; run the unit tests; smoke-test `db.connect()` and `history_import` against a synthetic export (needs DuckDB, never run).
2. User creates a Spotify dev app (owner needs Premium), sets `SPOTIPY_CLIENT_ID/SECRET/REDIRECT_URI` (`http://127.0.0.1:8888/callback`), runs `python -m radio.run` once on the Windows PC for login, then `install_windows_task.ps1`.
3. Ask the user to request the Extended Streaming History (Spotify privacy page; can take days) and optionally create ListenBrainz/Last.fm accounts + API keys.
4. After 1–2 days of data, inspect `radio.log` and `plays` to validate skip/complete/repeat detection and `start_kind` against what the user actually did; fix tracker edge cases first (everything downstream depends on label quality).
5. Re-verify the [VERIFY] items against primary Spotify docs (Developer Policy text, quotas, whether top/follows/saved endpoints still work).
6. Then start Phase 2 above.

9. Style notes

Match existing code density and idiom (small modules, type hints, short docstrings, pure logic separated from I/O so it's unit-testable). Keep Spotify-specific code isolated so it can be swapped.

PART B — Session log (2026-10-07)

Update — session log, 2026-10-07 (Claude Code, second session)

Everything below was done after the original handoff above. Credentials are deliberately not recorded here; the Spotify Client ID/Secret live in the user's shell env (SPOTIFY_CLIENT_ID/SECRET/REDIRECT_URI). The Client Secret was pasted into chat during the session, so rotate it in the Spotify dashboard when convenient.

A. Environment and setup (done)

- personal_radio/.venv created on the Mac (Python 3.13); duckdb 1.5.6, spotipy 2.26.0 installed. PyPI initially reset connections (transient network issue); a retry in the user's own terminal worked.
- Spotify dev app personalized-spotify-rec created in Development Mode (owner has Premium). Refresh token lifetime shown in dashboard: **180 days** (so re-login around early April 2027). Redirect URI is http://127.0.0.1:8888/callback (a stray leading "h" typo was found in the dashboard and had to be fixed; verify it is clean).
- First real login succeeded. Token cached in personal_radio/.spotify_cache. Do not Ctrl-C while the login listener on port 8888 waits, or the browser redirect gets ERR_CONNECTION_REFUSED.
- The tracker (python -m radio.run) is running on the **Mac** for testing. It is still meant to run on the Windows PC long-term (not yet set up there; install_windows_task.ps1 untested).
- ! commands inside Claude Code run in a separate shell, not the user's terminal, so env vars exported there don't carry over. Run interactive things (login) in the user's own terminal.

B. Verified against the real API

Check	Result
current_user_recently_played backfill	works (50 rows)
Saved tracks sync	works (102 rows)
Follows sync	works (4 rows)
Top items sync	works (278 rows); first run hit a 10s read timeout, fixed with a 30s client for sync
Batch GET /tracks?ids=	403 Forbidden for this app → use single GET /tracks/{id}
GET /tracks/{id} (single)	works, returns external_ids.isrc
isrc: search, limit=10	17/20 known ISRCs returned the exact URI; 2 empty; 1 hit a 429
Poller 429 handling	observed once in ~10 min of polling (Retry-After ~31s honored, recovered)
ISRC format	Spotify sometimes returns hyphenated ISRCs (QZ-L38-24-68720); must be normalized before lookups
MusicBrainz ISRC lookup	works; occasional transient 503 (retry succeeds). 30 of 46 tracks matched to an MBID (~65%)

Check	Result
ListenBrainz Labs similar-artists	works with no token
Playlist writes (verified 2026-10-07 via radio.smoke)	all OK: POST /me/playlists, POST /playlists/{id}/items, PUT /playlists/{id}/items (replace), GET /playlists/{id}/items, DELETE /playlists/{id}/followers (cleanup/unfollow). Response entries carry the track under item, not track. Old POST /users/{id}/playlists was not tried because the new path worked. Scope needed: playlist-modify-private.

C. Bugs found and fixed

- Tracker completion window (important).** COMPLETE_TAIL_MS was 4000 ms, but the last sample before a track change usually lands 4–5.5 s short of the end, so tracks played to 97–98% were stored as skipped, and the next track was wrongly tagged nav instead of auto. Fixed: window is now 10 s (radio/tracker.py), regression test added. Nine existing rows were repaired (backup: radio.duckdb.bak-1653). Note labels.label_play already treated $\geq 90\%$ plays as completed, so labels were right; only stored end_reason/start_kind were wrong.
- Credentials:** poller.build_client() now prompts for missing SPOTIFY_* values (secret via getpass); build_client(timeout=...) added.
- Sync timeout:** run.py builds a second client with timeout=30 for bulk sync calls.
- Hyphenated ISRCs:** ids.norm_isrc added. **Still to do:** 4 rows already stored in track_ids have hyphens; fix with a one-time UPDATE track_ids SET isrc = replace(isrc, '-', '') when the tracker is stopped (DuckDB single writer).

D. New code added this session (all in personal_radio/)

File	Role
radio/pipeline.py	plays → labelled rows. Repeat = same track again within 24 h; saved from saves; weights by start_kind (user 1.0, nav 0.8, auto 0.6, other 0.5) × 1.5 for intent plays. build_rows pure, load_rows(con) DB loader
radio/ids.py	identity cache table track_ids (URI, ISRC, recording MBID, artist MBID, checked timestamps). seed_tracks(ids_con, ev_con) (plays + saves + top tracks), backfill_isrc (single-track calls, 0.3 s pause, stops on 429), resolve_mbids (MusicBrainz, ≥ 1.1 s between calls, RADIO_CONTACT env for User-Agent contact). CLI: python -m radio.ids
radio/db.py	connect_snapshot() (read-only copy of DB + WAL while the tracker holds the lock); connect_ids() + IDS_SCHEMA: the track_ids identity cache now lives in its own file radio_ids.duckdb (moved out of the events DB later in the session)
radio/candidates/	cache.py (file JSON cache in .api_cache/), listenbrainz.py (similar artists), artists.py (seed weighting, aggregation, CLI: python -m radio.candidates.artists)
radio/smoke.py	live playlist-write smoke test (creates one private playlist, add/replace/read items, tries cleanup); passed
tests/	22 tests pass: test_core, test_pipeline, test_ids, test_candidates

First candidate run (13 labelled plays, 18 known artists) produced plausible but very mainstream suggestions (The Weeknd, Ariana Grande, Frank Ocean, Bieber, Kendrick, Ye). Known weaknesses: popularity/hub bias, seeds only from this session's plays, artists not tracks.

E. Real data captured so far

About 12–15 tracked plays (Mac session) plus the 50-row recent-plays backfill, 102 saves, 4 follows, 278 top items. Untested tracker cases: pause mid-track (must stay one play), replay/repeat detection, seek counting, quick skip <10 s on a later track, phone (iOS) plays.

F. Next steps, in order

1. ~~Run `python -m radio.smoke`~~ **Done: playlist writes work** (see §B). The `POST /me/playlists` ambiguity in the report's contradictions table is resolved: the new path works and uses `/items`.
2. Stop tracker once → fix the 4 hyphenated ISRCs, rerun `python -m radio.ids`, restart tracker. Keep the tracker restart commands handy (activate venv, export 3 env vars, `python -m radio.run`).
3. Resolve saved tracks + top tracks to ISRC/MBID (single calls, ~100–300; mind 429s because the smoke/ISRC tests share quota with the tracker) to get real seeds.
4. Candidate fixes: hubness/popularity penalty; recordings per candidate artist (ListenBrainz top-recordings or Last.fm); optional Last.fm key and ListenBrainz token.
5. Resolve candidate tracks → Spotify URIs (`isrc:` search, verify `external_ids.isrc`, fall back to `track: + artist:`), small limits, cache.
6. Per research report §H, build baselines **before** any ranker: recency-decayed replay and item-kNN; rolling-origin time-split harness.
7. Heuristic scorer + nightly "Weekly Auto" playlist with every pick's reason and **sampling probability logged** (propensities from day one); Thompson sampling over clusters/artists (~10% exploration).
8. Blind A/B vs Discover Weekly scoring only unfamiliar tracks.
9. Windows PC setup + `install_windows_task.ps1`; request Extended Streaming History (spotify.com/account/privacy; can take up to 30 days) and, optionally, ListenBrainz/Last.fm accounts.
10. Read the Spotify Developer Terms/Policy primary text (ML-training clause) — still unverified firsthand.

G. Operational notes

- DuckDB is single-writer: while the tracker runs, other processes must use `connect_snapshot()` or file-based caches. DB-writing tools (`radio.ids`, repairs) need the tracker stopped.
- Keep Spotify-derived tables and open-data tables logically separate; store only URI/ISRC/names from Spotify.
- Search `limit` max is 10 (default 5). Batch endpoints may be 403 in Development Mode; test before relying on any.
- The shared API quota means ad-hoc scripts can trigger 429s for the live tracker. Keep experiments short.

Session addenda (later the same day)

- Identity cache moved to `radio_ids.duckdb`; `python -m radio.ids` seeds from a snapshot of the events DB, so it runs while the tracker runs. Last run: 162 tracks, 162 ISRCs, 71 MBIDs (88 checked; many MusicBrainz 503s, retry later).
 - `radio.smoke` passed against the live API (see §B). `GET /playlists/{id}/items` entries use the key `item`.
 - Tests: 22 passing. Tracker restarted with the fixed completion window and has been logging `auto/completed` correctly.
 - Research folder fully read; gaps recorded in section 0.2 above.
-

PART C — Research report (final synthesis)

Source: `reports/Personal music recommender methods.md`. Includes primary-source links.

Rebuild Personal Radio around open data, baselines, and logged exploration

Personal Radio's architecture (open-data candidates, a LightGBM ranker, a bandit, a nightly Spotify playlist) matches the pattern that survives in the wild, but the research says to change the order of work. Build ListenBrainz/Last.fm candidates plus an ISRC-to-MBID identity layer first. Then beat strong, simple baselines (recency-decayed replay, item-kNN) before trusting any ranker. Log propensities from the first exploratory pick, and judge the whole system with a blind, novelty-adjusted A/B rather than offline metrics. The evidence for sequence transformers, HSTU, Mamba, LLM recommenders, imitation learning and offline RL on one person's history is thin to nonexistent, so those belong in a late stretch phase or out of the plan. The legal picture is workable for a private tool but not clean. Spotify's Developer Terms (v10, 15 May 2025) reportedly bar training ML models on Spotify Content, and Spotify metadata should therefore stay minimal and live. Open sources (MusicBrainz core CC0, Last.fm non-commercial, ListenBrainz) fit a private tool. Several famous datasets (LFM-2b, Million Playlist Dataset, Sequential Skip) are reported as withdrawn. The most urgent uncertainty is operational: the notes disagree on whether `POST /me/playlists` was removed or is the replacement for the old user-scoped endpoint, so **the HANDOFF assumptions about playlist writes must be re-tested live against the current API before anything else is built**. Most of this evidence comes from summarized fetches and a limited search budget, so "unverified" flags are preserved below. This is not legal advice.

Executive summary: eight changes ranked by gain per unit effort

1. **Run a one-hour live API smoke test first.** Create a playlist, write items via `/items`, run an `isrc:` search with `limit=10`, and check that top/saved/follows endpoints respond. The sources conflict on playlist creation (see the contradictions section), and no 2026-dated confirmation of `isrc:` search exists. (Evidence: official changelogs, but contradictory summaries.)
2. **Make "replay what I recently played" and item-kNN the bar to beat.** On Yambda, DecayPop was strongest on the Like task and ItemKNN matched or beat SASRec at 50M scale ([arXiv 2505.22238](#)). Personal listening is repetitive, so these baselines are cheap and hard to beat.
3. **Log propensities and randomize a small exploration slice from day one.** Inverse-propensity and doubly robust methods need probabilities that Spotify will never give you; only a system you control can log them. This also supplies unbiased evaluation data.
4. **Mirror plays to ListenBrainz and make MBID/ISRC the primary track key.** This gives you a redundant record, a personalized CF/similarity feed, and insulation from Spotify ID churn.
5. **Evaluate with rolling-origin time splits and a pre-registered, novelty-adjusted blind A/B.** Leave-one-out distorts model rankings ([arXiv 2507.16289](#)).
6. **Use Beta-Bernoulli Thompson sampling over clusters or artists, not per-track contextual bandits.** At 20 plays a day with 10% exploration you get roughly two exploratory plays a day.
7. **Add DPP or MMR re-ranking with a calibration floor and monitor narrowing metrics.** It costs about 30 lines of code and addresses a core success criterion.
8. **Keep the LLM off the hot path.** Use it to parse steering requests into a validated schema, and to explain picks from features the ranker used. Defer SASRec pretraining, audio embeddings and RL.

Prior art confirms the architecture and exposes the missing pieces

No verified end-to-end single-user project combines a learned ranker, a bandit and Spotify write-back, though "not found" is weak evidence given partial coverage. What exists is a layer of open-data tools that consume rather than compute recommendations. Explo (about 2,018 stars, MIT, pushed 2026-09-09) pulls ListenBrainz Weekly Exploration and Jams playlists and resolves them to files via YouTube, Soulseek or Lidarr, with no ranking of its own (README). AudioMuse-AI (about 2,693 stars, AGPL-3.0, pushed 2026-09-27) does local sonic analysis for Jellyfin/Navidrome with a reported ~2.2 GB of models and heavy CPU use (GitHub, DEV post). Spotify-centric LLM playlist builders are tiny or stale; llmusic dates from 2023 (repo). The one transferable idea is a three-tier fallback chain (a replacement audio-feature service, then local scoring, then Spotify-only search) in spotify-playlist-curator (repo); the surviving projects all treat ListenBrainz/MusicBrainz as the substrate.

The scrobbler ecosystem is healthy and reusable. Per GitHub API stats in the notes, troi-recommendation-playground (GPL-2.0, pushed 2026-09-28) powers ListenBrainz Radio and Weekly Jams (GitHub). multi-scrobbler (MIT, about 1,255 stars, pushed 2026-10-07) forwards Spotify listens to ListenBrainz, Last.fm and Maloja (GitHub). Koito (MIT) and Maloja (GPL-3.0) are self-hosted scrobble stores, and pylast (Apache-2.0) and liblistenbrainz cover the API layer. Because troi and listenbrainz-server are GPL, call their APIs rather than copying code. ListenBrainz's CF endpoint returns recording MBIDs with scores but is documented as experimental and "probably will change" (docs), so treat it as one noisy candidate source, never the backbone. The repo listenbrainz-labs was archived in 2020, which is a reminder that open-data tooling also rots (GitHub API).

Single-user academic evidence is sparse. The best-matching result is Deezer's semi-personalized cold-start work, which shows population-level priors help when per-user data is scarce (arXiv 2106.03819). LLM taste-profile work is preliminary: in a 64-person Deezer-employee study, LLM profiles built from one's own data were rated above random ones, but profile ratings linked weakly to downstream recall and profiles hallucinated tracks (arXiv 2507.16708). The consistent implication is to prefer item-transferable features (artist similarity, tags, popularity, recency, related-artist skip history) over per-item IDs, because a single user supplies thousands of labeled events at best.

Resilience lessons are concrete. Spotify's Nov 2024 and Feb 2026 changes broke anything that relied on recommendations, audio features, popularity, or other users' data (Spotify blog, TechCrunch). The design response is to store the raw event log yourself, key tracks by MBID/ISRC, isolate each source behind an adapter with a fallback, cache every API response, tolerate missing features in the ranker, and watch the changelog. Because Development Mode needs an active Premium subscription on the app owner, a lapse stops the app, so the nightly job needs a health check and alert.

Open data covers candidates, but identity mapping is the real bottleneck

The table below compares candidate and metadata sources. Facts come from the notes' fetches; items marked unverified were not confirmed on a primary page.

Source	License / terms (private use)	Rate limit	Role	Key risk
ListenBrainz API (CF recs, LB Radio, metadata/lookup)	Dumps described as open; license text not verified on docs (dumps docs)	About 1 call/s; token may raise it; descriptive User-Agent required (API docs)	MBID-only recs, similarity, popularity, feedback write-back	CF endpoint experimental; returns 204 until generated; outputs need resolving to Spotify

Source	License / terms (private use)	Rate limit	Role	Key risk
MusicBrainz web service	Core data CC0; supplementary data CC BY-NC-SA 3.0 (license)	About 1 req/s per IP, User-Agent with contact, else throttled or declined (docs); the page is dated 2012	Identity spine, tags, relationships	Coverage of remixes, live and regional variants
Last.fm API	"Solely for non-commercial purposes"; 100 MB stored-data cap; follow cache headers (ToS)	Discretionary per ToS; about 5 req/s from secondary sources only	Similar artists/tracks, tags	Cap forbids bulk mirroring; rate numbers unconfirmed
Discogs	CC0 dumps and split API terms, not verified (official page returned 403)	Unverified (60/min authenticated from memory)	Optional genre/credits enrichment	Terms and image rights unread
AcousticBrainz	Discontinued; no MetaBrainz successor found (blog mirror)	n/a	Do not plan on it	Audio descriptors must be computed locally
ReccoBeats	Vendor claims a free audio-feature substitute; one review reports inconsistent coverage (summary)	Unknown	Fallback tier only	Reliability and terms unverified; does not solve the Spotify ML clause for audio you fetch from Spotify

Wikidata, Bandcamp tags and Cover Art Archive were not researched. At about one request per second per service, a single-user pipeline is feasible only if candidates and lookups are cached aggressively, which the notes estimate at a few thousand lookups per hour per service.

Identity resolution is where most pipelines will quietly lose data. MusicBrainz's own documentation says remasters, remixes and edits receive distinct ISRCs ([MB ISRC terms](#)), and a UK government metadata report found **4 to 11% of recordings carried more than one ISRC** in a roughly 5M-recording sample ([GOV.UK](#)). No authoritative Spotify-to-MBID match rate was found, so measure your own on your library, cache every mapping, log the unresolved tracks, and layer ISRC lookup, then ListenBrainz's name-based `metadata/lookup` (token required), then fuzzy artist, title and duration matching. In the reverse direction (MBID to Spotify), batch `GET /tracks` is gone, so each mapping costs one call, and search `limit` is capped at 10 (default 5) per the [migration guide](#). `external_ids` (carrying ISRC) was removed in February and then reverted in March 2026 ([March changelog](#)), so `isrc:` search is plausible but unconfirmed in 2026; older community threads report empty results for valid ISRCs and formatting quirks ([thread](#)). Always verify `external_ids.isrc` in the response and fall back to `track:` plus `artist:` search.

Datasets: little is both available and useful

Dataset	Size	License / status	Mappable to MBID/ISRC?	Fits single-user pretraining?
Yambda (Yandex)	50M/500M/5B variants; up to 4.79B events, 9.39M items	Apache 2.0 on the page, "published exclusively for scientific and research purposes" (HF); paper reportedly CC BY 4.0	No documented MusicBrainz/ISRC/Spotify mapping; Yandex catalog skews Russian/CIS	Architecture ablation and benchmark, not item-level transfer; <code>is_organic</code> flag is useful
ListenBrainz dumps	More than 800M listens (ISMIR 2024) (poster)	License not verified on primary page	Native MBIDs when mapped; unmapped listens have names only	Best candidate for MBID-space item embeddings (my inference, untested)
Music4All-Onion	109,269 tracks, about 253M listening records	Zenodo "Open"; license unconfirmed (Zenodo); Music4All A+A is CC BY-NC-SA 4.0	Names/IDs need mapping	Possible, check license first

Dataset	Size	License / status	Mappable to MBID/ISRC?	Fits single-user pretraining?
LFM-2b / LFM-1b	2.0B / over 1B events	Host pages say "not available for download anymore due to license issues" (LFM-2b)	Spotify ID mappings reported	Do not plan on them
Spotify Million Playlist Dataset	1M playlists	Not hosted on Alcrowd; request from Spotify Research (Alcrowd)	Spotify URIs only	Conflicts with the "no Spotify content in ML" stance
Spotify Sequential Skip	About 130M sessions	Not downloadable; contact Spotify Research (Alcrowd)	Spotify track IDs	Same conflict
MSD / Taste Profile	About 48M triplets (from memory)	Unverified	Mappable via metadata (unverified)	Dated (2011)

The notes found **no study showing that pretraining on a public music dataset and then fine-tuning for one user improves recommendations**. The closest evidence is general: Deezer projects new users into an existing latent space instead of retraining ([Deezer](#)), and a survey describes pretrain-then-fine-tune for cold start ([arXiv 2009.09226](#)). The realistic transfer design is to pretrain item embeddings on ListenBrainz co-listening in MBID space and fit only a light user vector. That is an inference, not a demonstrated result.

Model the simple things well, and treat skips and offline numbers as noisy

The strongest finding across the modeling notes is that fancy sequence models earn their keep only at scale. Dacrema et al. reproduced only 7 of 18 neural top-N algorithms, and 6 of those were often beaten by nearest-neighbor or graph heuristics ([summary](#)). BERT4Rec results varied across implementations and needed long training ([arXiv 2207.07483](#)), and SASRec does not consistently beat GRU4Rec ([arXiv 2408.03873](#)). On music, Yambda's baseline table (NDCG@10, Listen+ task, numbers from an LLM-summarized fetch that should be checked against the paper) shows:

Model	50M	500M	5B
MostPop	0.0186	0.0173	0.0175
DecayPop	0.0260	0.0267	0.0271
ItemKNN	0.0781	0.0708	not run
iALS	0.0407	0.0384	0.0388
SASRec	0.0748	0.0754	0.0647

At 10,000-user scale SASRec only matched ItemKNN, so one user's data cannot teach a transformer item embeddings. HSTU (Apache-2.0, actively pushed) makes trillion-parameter, industrial-scale claims, and a 2026 result shows retention on random subsets of data, not on intrinsically small datasets ([arXiv 2604.07739](#)). Mamba4Rec (MIT, last push 2025-04) has no independent replication found. For the ranker, the notes' evidence for "LightGBM beats neural at this scale" is an inference with no music-specific replication, so keep LightGBM but benchmark it against the baselines above on rolling time splits.

Skip labels are noisy. The WSDM Cup 2019 winners reached mean average accuracy of about 0.64 using RNNs over session-position and previous-skip features ([arXiv 1902.04743](#), [arXiv 1903.08408](#)), which is largely "skip begets skip" rather than taste. A later analysis of that data found a temporal leakage problem ([arXiv 2301.03881](#)). Spotify's Home bandit (BaRT) defines success as a stream of at least 30 seconds, per slide and blog summaries only ([Lalmas slides](#)). For the HANDOFF labels, condition skip features on `start_kind`, position,

and context, never use future in-session information, and treat a lone early skip as weak evidence. This supports the graded labels already in `labels.py`.

Imitation learning deserves skepticism. No credible evidence was found that imitation, inverse RL or offline RL beats supervised ranking on implicit feedback. Deffayet et al. argue that next-item-prediction evaluation of RL recommenders cannot show the benefit RL is meant to bring ([arXiv 2301.00993](#)), and a reported offline-RL win compares against a baseline that does not optimize cumulative reward ([arXiv 2310.00678](#)). Behavior cloning on your own history is next-item prediction, which inherits exposure bias: you clone the policy of the feeds you were shown, filtered by your acceptance. Reinterpret "imitation" as a labeled-feedback learner plus a deliberate exploration budget.

Exposure correction is feasible only for a system you control. The Open Bandit Dataset exists because true propensities are rare ([arXiv 2008.07146](#)); you cannot observe Spotify's. So your own picks must be sampled with logged probabilities, with clipped or self-normalized estimators and wide confidence intervals expected (background knowledge, unverified). Tag every play as recommender-sourced or organic, as Yambda does with `is_organic` (about 48.7% of listens are recommendation-driven there), and as Anderson et al. found algorithm-driven listening linked to reduced diversity at Spotify ([paper](#)).

Library choices

Library	License	Last push (per notes)	Verdict for one user
implicit (ALS/BPR/kNN)	MIT	2026-05-08	Adopt for co-occurrence and kNN baselines
LightGBM	Not confirmed in notes (MIT from memory)	Not checked	Adopt for the ranker; verify maintenance
LensKit	NOASSERTION	2026-09-30	Optional evaluation harness
RecBole	MIT	2025-02-24	Skip unless running pretraining experiments; slow-moving
Transformers4Rec, Merlin	Apache-2.0	2026-08, 2026-07	Overkill
TorchRec	BSD-3	2026-10-07	Built for embedding sharding; skip
gSASRec-pytorch	Apache-2.0	2024-02	Only if you pursue SASRec
HSTU (generative-recommenders)	Apache-2.0	2026-10-07	Defer; no small-data evidence
Mamba4Rec	MIT	2025-04	Defer

Bandit tool	License	Last push	Verdict
Custom Beta-Bernoulli Thompson sampling	n/a	n/a	Adopt: a few dozen lines
River (includes a bandit module)	BSD-3	2026-10-07	Optional; its own docs warn that batch learning is usually sufficient (PyPI)
Vowpal Wabbit	NOASSERTION per API (BSD-3 from memory)	2026-09-28	Active; heavier than needed
Open Bandit Pipeline	Apache-2.0	2024-06-03	Use for OPE estimators only; aging against newer numpy/sklearn
MABWiser	Apache-2.0	2024-09-05	Stale

The practical exploration budget (5 to 15% of slots) is a researcher suggestion, not a sourced figure, and the arithmetic favors coarse arms: at 20 plays a day and 10% exploration there are about two exploratory plays a day, so Thompson sampling over clusters or artists, exploring the unplayed side of the catalog near taste clusters, is the proportionate design.

Audio is optional, and the no-audio path is competitive

All strong open music encoders carry non-commercial weights, which is acceptable for a private, non-redistributed tool. MERT-v1-330M is cc-by-nc-4.0 ([HF](#)); MuQ weights are CC-BY-NC 4.0 with MIT code ([HF](#)); LAION-CLAP's music checkpoint is Apache-2.0 ([HF](#)); MusicFM code is MIT with weight license unstated ([GitHub](#)). Essentia's library is AGPLv3, and its pretrained models are CC BY-NC-ND on one page and CC BY-NC-SA on another ([licensing](#), [models](#)), so check before relying on either reading. If the tool might ever be shared, prefer CLAP.

Option	License	Needs audio?	Data need	Fits single user?	Key risk
Tag/metadata embeddings (MusicBrainz, Last.fm tags)	CC0 core / Last.fm non-commercial	No	Low	Yes, the default	Tag noise, coverage
ListenBrainz co-listening embeddings	Dump license unverified	No	Needs dump processing	Yes, via MBID space	Unmapped listens
Essentia MusiCNN / Discogs-EffNet	AGPL lib; NC models	Yes	Low	Yes, cheap first step	License pages disagree
MERT / MuQ / MusicFM	NC weights / MIT code	Yes	Moderate compute	Plausible	Small MIR gains (about 1 AP point on MTG-Jamendo) do not predict recsys gains
LAION-CLAP	Apache-2.0 weights	Yes	Moderate	Plausible	Repo "work in progress", 64 open issues

The key caution is Tamm and Aljanaki's finding of a "significant performance disparity" between MIR tasks and recommendation across nine pretrained backends ([arXiv 2604.23077](#), details secondhand), so a better MIR benchmark score is no reason to choose a bigger model. Tag-based representations beat audio CNNs for cold-start in a 2017 study (with tiny absolute numbers) ([arXiv 1706.09739](#)), and listening-based embeddings lose their advantage under artist-based splits ([ISMIR 2020 poster](#)). Legitimate audio means your own files or CC corpora (FMA, MTG-Jamendo, which is non-commercial research only). Spotify previews are out by contract, Deezer forbids local audio storage ([guidelines](#)), and Apple restricts previews to streaming for promotion without downloading or caching ([Apple](#)). For mood, valence is much harder to predict than arousal ([arXiv 2202.10453](#)), so use mood tags as soft features. Time-of-day features are cheap in a GBDT, but no effect size was found, so test them on your own plays.

Diversity, LLM steering and the session layer: adopt the cheap parts

Diversity re-ranking is the best-evidenced add-on. Greedy MAP DPP has a published fast algorithm and a Hulu online A/B ([arXiv 1709.05135](#)), and Steck-style calibration reranks to match your historical category distribution ([survey source](#)). Field studies show effects vary by user: personalization raised podcast streams 28.9% but cut individual diversity 11.5% ([arXiv 2003.08203](#)), while Deezer found the effect "depends on users" ([arXiv 2109.03915](#)). Cross-user homogenization results (Chaney et al., [arXiv 1710.11214](#)) do not apply directly to one

user. Track narrowing with unique-artist share, artist/genre entropy per window, median popularity percentile, the recommender-sourced share of plays, and first-ever-play rate; those metric choices are inferences.

LLMs are best as translators and explainers. Zero-shot LLM rankers show popularity and position bias ([arXiv 2305.08845](#)); RecBench reports LLM gains (up to 5% AUC, up to 170% NDCG@10 on sequential recommendation) alongside inference too slow for real-time use ([arXiv 2503.05493](#)); a Criteo meta-review of uncertain peer-review status reports a 15 to 60% accuracy gap versus the best classical systems ([PDF](#)); and no head-to-head on a single user's history exists. TalkPlay-Tools is the closest published analogue to "LLM turns a request into filters over your own table" ([arXiv 2510.01698](#)). So: parse "darker this week" into a validated schema (tag filters, weights, expiry), let the classical model score, generate explanations from the features the ranker actually used, and match every LLM-named track against your candidate set. Sending listening history to a hosted API leaks taste; send aggregate summaries or use a local model. These designs are inferences, not tested results.

The session layer needs no library. Use an exponentially weighted state over recent positives and negatives with a half-life of minutes to hours, re-weighting candidates at queue-fill time. No source validates a particular "N skips then pivot" rule, so treat N as a parameter tuned on your skip logs. Online learning of the main model is unsupported; River's own docs say the answer to whether you need online ML is "likely no".

For queue control, `POST /v1/me/player/queue` is Premium-only, append-only as documented, and order of execution is "not guaranteed" alongside other Player calls ([docs](#)). Hence the just-in-time pattern: keep one or two tracks queued, poll, and add the next near track end; do not issue skip and queue-add simultaneously. Multi-device handoff, Private Session and offline plays are undocumented, so test empirically. Recently-played is capped at 50 items ([docs](#)), and the HANDOFF already notes Private Session plays are invisible.

Legal and API constraints in October 2026: workable for private use, with three gray zones

The primary Spotify texts were read only through a summarizing fetch, so quotes and clause numbers need verification against the live pages. The Developer Terms are Version 10, effective 15 May 2025, with no later revision found ([Terms](#), [Policy](#)). The reported provisions are: Terms IV.2.a and Policy III.14 bar using Spotify Content "to train a machine learning or AI model" and ingesting it into one; IV.3.b limits caching to what is "strictly necessary"; IV.3.a bars indefinite storage and "compilations or databases"; and III.1.a licenses "private personal use", a phrase that does not define hobby apps. Spotify describes Development Mode as for learning, experimentation and personal non-commercial projects ([blog](#)). This supports the HANDOFF's [VERIFY] flag: the ML clause is now reported from a Spotify primary page, but through a summarizer, so read the text yourself.

Three gray zones follow. First, the clause targets Spotify Content obtained through the platform; the Extended Streaming History comes through a data-subject export (lifetime of the account, up to 30 days to arrive per [Spotify's privacy page](#)), and plausibly falls outside it, but no Spotify text says so. Second, joining export history to API-fetched metadata and persisting it sits near the "no databases" and ML clauses, so keep API-derived metadata minimal (URI, ISRC, names) and short-lived. Third, your own behavior labels as a training signal are a gray area the HANDOFF correctly leaves unresolved. The clearest prohibited design is training on API-fetched metadata or audio features; the safest is using the API only for live lookup, playback and playlist writing. The practical enforcement risk is key revocation, not litigation: documented cases are access revocations, sometimes with no appeal, and a reported 2020 threat over playlist transfer to competing services ([AppleInsider](#), [community thread](#)). Combining Spotify data with other services was cited as a reported trigger, which is exactly what this design does, so keep Spotify-sourced and open-data tables logically separate.

Development Mode rules: from 11 February 2026 new Client IDs need a Premium owner and allow five authorized users, and existing integrations were migrated on 9 March 2026 ([blog](#)). The 23 July 2026 update raised the cap to 25 Client IDs per developer account with one shared quota, and 429 responses now carry "reason": "QUOTA_EXCEEDED" ([blog](#)). No numeric rate limit is published beyond a rolling 30-second window

(docs). Extended quota reportedly requires about 250k MAU, which is user-reported only. The six-month refresh-token lifetime in the HANDOFF rests on the third-party vorplabs summary; the official July changelog did not list it, so keep it flagged.

For open data: Last.fm permits "solely non-commercial" use with a 100 MB stored-data cap, a duty to follow cache headers, and deletion on termination (ToS); the ToS does not mention ML. MetaBrainz states "Personal use of our datasets will always be free" (MetaBrainz). Deezer's API terms limit use to non-commercial and say nothing about ML, which is ambiguity, not permission (terms). Discogs terms could not be fetched.

Contradictions and open questions, each with a quick test

Question	What the notes say	Test
Was <code>POST /me/playlists</code> removed or is it the replacement?	One note lists it among Feb 2026 removals (changelog) and infers that a replacement may be needed. Two notes read the same changelog as removing <code>POST /users/{user_id}/playlists</code> in favor of <code>POST /me/playlists</code> , with <code>/tracks</code> renamed <code>/items</code> .	Create a playlist, add items with <code>POST /playlists/{id}/items</code> , and replace with <code>PUT /playlists/{id}/items</code> . One call each, today.
Do HANDOFF notes on playlist writes hold?	HANDOFF predates the renames and says Phase 2 writes via "replace items".	Re-test as above; treat the HANDOFF as unverified here.
Last.fm rate limit	Official ToS: discretionary; secondary source: about 5 req/s per IP averaged over 5 minutes, similar-artist cache at least a week; another source: "be reasonable".	Honor <code>Retry-After</code> errors, cap yourself at 1 to 2 req/s, and cache for a week.
Is LFM-2b still downloadable?	Host page says no; the modeling note says "check current status".	Open the JKU page. Assume unavailable.
Is the Million Playlist Dataset still downloadable?	Alcrowd says not hosted, request via Spotify Research (notice dated July 2024).	Same. It also conflicts with the no-Spotify-content-in-ML stance.
Yambda: Apache 2.0 vs "research only"	Page lists both; paper reportedly CC BY 4.0.	Read the license file on the page and ask the maintainers whether a personal hobby project counts. Until settled, use it only for architecture experiments.
Yambda collection window	Paper says 11 months; Yandex news says 10.	Immaterial; cite the paper.
Essentia model license	CC BY-NC-ND on one page, CC BY-NC-SA on another.	Check the per-model card before use.
<code>isrc:</code> search after Feb 2026	Plausible, no 2026 confirmation; limit is 10.	Test 20 known ISRCs and record the hit rate.
Does the Feb 2026 removal list hit top/follows/saved sync?	Library save/follow consolidated into <code>/me/library</code> ; the HANDOFF scopes use older paths.	Run sync steps once; check which fail. The HANDOFF already isolates them.
Does a single user's data move ListenBrainz CF?	Untested.	Mirror plays for two weeks and compare recommendations.

Question	What the notes say	Test
LightGBM vs kNN on your data	No music-specific replication.	Rolling-origin comparison on Extended History; the decisive experiment.
Does time-of-day or device help?	No effect sizes found.	Ablate in the LightGBM feature set.
Does Private Session/offline appear in recently-played?	Docs are silent.	Play in Private Session and offline once.

Recommended stack and build order

Add: an ISRC/MBID identity cache; a ListenBrainz mirror via multi-scrobber; a propensity-logging exploration policy; baseline models (decayed replay, implicit item-kNN); a time-split evaluation harness; a DPP or MMR re-ranker; and a heartbeat/health alert. **Replace:** per-track contextual bandits with Thompson sampling over clusters; "imitation learning" with labeled feedback plus exploration. **Drop for now:** SASRec pretraining, HSTU, Mamba, RL, audio embeddings, feature stores and orchestrators (plain DuckDB, OS scheduler and a runs table are enough; this tooling advice is entirely inference because no sources were retrieved).

Two-weekend MVP. Weekend 1: run the API smoke test; install and run the existing code against real Spotify and Windows; request the Extended Streaming History; build `ids.py` with ISRC lookup and a measured match rate; add ListenBrainz and Last.fm candidates with caching and User-Agent headers; resolve to Spotify via `isrc: search`; build the heuristic scorer (cluster similarity, familiarity, decayed replay) and write the nightly playlist with every pick's reason and sampling probability logged. Weekend 2: wire labels (saves, repeats, `start_kind`); build the rolling time-split harness and baselines (decayed replay, item-kNN); train LightGBM on the history import and compare; add Thompson sampling over clusters with a 10% budget; add the blind A/B harness (30 tracks per arm, source hidden, logged randomization).

Stretch version. Add DPP/MMR calibration and narrowing metrics; the exponentially weighted session layer and just-in-time queue; schema-constrained LLM steering with grounded explanations; the FastAPI dashboard; tag embeddings and Essentia MusiCNN features; then optional ListenBrainz-space pretraining with a light user vector, and only after that sequence models.

A blind A/B power note (the notes' own rough estimate, not computed from a source): at roughly 30 tracks per arm per week with a 30% base hit rate, ten weeks gives about 300 per arm, enough to detect a gap of about 10 points, not 3. A recommender limited to known tracks beats Discover Weekly trivially, so score unfamiliar tracks only. Discover Weekly also adapts to the plays you generate while testing. Treat offline metrics as a filter that removes bad models, not proof of superiority, because the target is confounded by exposure.

Evidence quality grading

Claim	Grade
Tuned simple baselines often beat neural recommenders	Replicated across two papers, mostly pre-sequence and pre-LLM
BERT4Rec/SASRec results depend on implementation and training budget	Single group plus a second study
ItemKNN and DecayPop competitive on Yambda	Single paper (numbers via summarizer)
HSTU and Mamba gains on one user's data	None found; HSTU gains are single-paper, industrial scale

Claim	Grade
LLM recommenders beat tuned classical baselines	Weak and hype-prone: benchmark claims from single papers; no single-user evidence; efficiency penalties
Imitation learning or offline RL beats supervised ranking	No credible evidence; evaluation critiques (single paper)
Leave-one-out distorts rankings	Multiple papers (2020 to 2025)
Pretrain on public data then fine-tune on one user helps	Not demonstrated; my inference
Personalization narrows diversity	Single studies at Spotify and Deezer, user-dependent
MIR benchmark rank does not predict recsys quality	Single group (two versions)
Spotify ML-training ban	Primary page via summarizer; verify
Dev Mode and endpoint changes	Official changelogs, with the playlist-creation conflict above
Exploration budget 5 to 15%; session half-life; "N skips" pivot	My inference
Power estimate for the blind A/B	My inference

Ten-item reading list, in order

1. Spotify February 2026 migration guide and the March, May and July 2026 changelogs: [migration guide](#), [March](#).
 2. Spotify Developer Terms and Policy, read in full: [Terms](#), [Policy](#).
 3. ListenBrainz API docs, recommendation and metadata pages: [recommendation](#), [metadata](#).
 4. Troi LB Radio reference, plus the repo for patch design: [reference](#).
 5. Yambda paper, for baselines, the temporal protocol and `is_organic`: [arXiv 2505.22238](#).
 6. Dacrema et al., "Are we really making much progress?": [arXiv 1907.06902](#).
 7. "Time to Split" evaluation paper: [arXiv 2507.16289](#).
 8. Anderson et al., algorithmic effects on listening diversity at Spotify: [paper](#).
 9. Chen et al., fast greedy MAP DPP inference: [arXiv 1709.05135](#).
 10. Tamm and Aljanaki on pretrained music audio representations for recommendation: [arXiv 2604.23077](#).
- Optionally follow with TalkPlay-Tools for the LLM-as-translator design: [arXiv 2510.01698](#).

Risks you have not considered

Exposure feedback is the biggest: once the playlist is your main source, your labels come from your own system, which is why propensity logging and recommender-sourced tagging matter. Premium dependency is next: the Premium subscription on the app owner is a single point of failure for Development Mode. Sleeping PCs create data gaps, and the user's Mac-versus-Windows split means the tracker's host is the weak link; SMTC covers Windows only, and multi-scrobber does not appear to offer an SMTC source. Open-data fragility is real, since ListenBrainz CF is experimental and Labs was archived. The Last.fm 100 MB cap rules out bulk mirroring. Identity mismatches can silently drop whole classes of tracks (remasters, live, regional variants), so report match rate by category. Quota is now shared across all Client IDs on your developer account, so a second test app can starve the first, and you should add `QUOTA_EXCEEDED` handling. Combining Spotify and other-service data was cited as an enforcement trigger, and keys can be revoked with no appeal. Hosted LLM calls leak taste data. Time-based subsets of public datasets change conclusions ([arXiv 2509.09685](#)), so beware over-trusting a short window of your own history. Finally, the Extended Streaming History can take up to 30 days to arrive, so request it now; otherwise the ranker has to wait 2 to 4 weeks for labels.

Conclusion

The notes shift the project's center of gravity from modeling to measurement and plumbing. The decisive assets are not a cleverer architecture but a clean identity layer, logged propensities, honest baselines and a blind A/B that scores only unfamiliar tracks, because every sophisticated alternative (sequence transformers, imitation learning, LLM ranking, audio foundation models) lacks single-user evidence and several face license or availability problems. The most valuable first hour is also the most uncertain one: a live test of playlist creation and ISRC search, since the sources disagree and the HANDOFF's assumptions predate the Feb 2026 renames.

What remains genuinely open is whether LightGBM on a few thousand labeled plays can beat decayed replay plus item-kNN, whether mirroring plays to ListenBrainz makes its CF useful for one listener, and how far the "private personal use" language stretches for Spotify's ML clause. Each is answerable with an experiment on your own data or a read of the primary text, and none requires more research. This report is not legal advice; confirm the quoted clauses against the live Spotify, Last.fm, MetaBrainz and dataset terms.

PART D — Research notes (evidence base)

Source: `research_notes/Personal music recommender methods/`. Grades: [R] replicated, [S] single paper, [B] blog/secondary, [I] inference. Each note lists its own Gaps.

D.1 prior_art.md

Prior art: single-user / self-hosted music recommenders (as of 2026-10-07)

Method note: repo stats (stars, license, last push) were pulled from the GitHub REST API on 2026-10-07 (cited as "GitHub API"). Time-boxed pass of about 12 tool calls; coverage is partial and gaps are listed.

Which open-source personal/self-hosted recommenders, playlist generators, LLM playlist builders and Spotify-replacement projects exist, and how did they survive API changes?

Takeaway

The maintained, well-starred projects all sit on open data (ListenBrainz, MusicBrainz, Subsonic/Jellyfin APIs, local audio analysis) rather than Spotify's removed endpoints. Spotify-centric LLM playlist builders are mostly small, abandoned, or pre-2024. I found no mature single-user project that combines LightGBM ranking, a bandit, and Spotify write-back.

Cited Findings

- Explo, "Spotify's Discover Weekly for self-hosted music systems", has about 2018 stars, MIT license, last push 2026-09-09. It pulls ListenBrainz Weekly Exploration, Weekly Jams and Daily Jams, can import Spotify/Apple Music playlists, then fetches tracks via YouTube, Sounseek or Lidarr. It is a thin consumer of LB's recommendations, with no own ranking. — [GitHub API](#), [README](#)
- DroppedNeedle: about 1468 stars, AGPL-3.0, pushed 2026-10-07. A self-hosted music request/discovery app built on MusicBrainz, with a built-in download engine and playback via Jellyfin/Navidrome/Plex. The README advertises "personalized recommendations". — [README](#), [GitHub API](#)
- JellyDJ: 4 stars, AGPL-3.0, pushed 2026-05-20, v1.4.0. A Jellyfin taste-profile and smart-playlist engine with Lidarr and audio analysis. It is very small, so treat it as an idea source, not a dependency. — [README](#), [GitHub API search](#)
- AudioMuse-AI: about 2693 stars, AGPL-3.0, pushed 2026-09-27. Containerized sonic analysis (Librosa + TensorFlow) for Jellyfin/Navidrome: similar songs, song-path playlists, and playlists from listening habits. A Navidrome plugin (AudioMuse-AI-NV-plugin: 518 stars, AGPL-3.0, pushed 2026-09-19) exposes it through Subsonic similar-song calls. A community install note reports about 2.2 GB of ML models, and analysis is CPU-heavy. — [GitHub API](#), [DEV post](#)
- Recommendarr (Shadowalker125): 49 stars, GPL-3.0, last push 2025-03-11. It adds artists from ListenBrainz "created for you" playlists to Lidarr. Stale. — [GitHub API search](#)
- LLM playlist builders found: dtpreda/llmusic (10 stars, MIT, last push 2023-05-16, prompt-to-Spotify playlist; effectively abandoned) and ChatGPTify (older Spotify/ChatGPT script). Newer tiny repos exist (SmartDiscover, 3 stars, MIT, 2026-08; Lyra-music-player, 2 stars). None is validated or widely used. spotify-playlist-curator (5 stars, no license, 2026-04) uses a 3-tier fallback: ReccoBeats, then audio-feature scoring, then Spotify-only search/genre/artist proximity. That fallback chain is a useful resilience pattern. — [llmusic](#), [GitHub API](#), [curator](#), [search](#)

- Embeat (gdstudio-org): 376 stars, license NOASSERTION, pushed 2026-09-24. Item-to-item recommender trained on the Spotify 45M-track / 1.8M-playlist dataset. Population-level, not personal. — [search](#)
- LMS (epoupon/lms): 1683 stars, GPL-3.0, pushed 2026-09-23. Self-hosted music server. I did not verify its recommendation features. — [GitHub API](#)
- Spotify replacement APIs: ReccoBeats (free, advertised as audio-feature substitute; one review says reliability/coverage has been inconsistent), Cyanite (paid B2B), and Essentia for self-hosted audio analysis. These come from vendor/blog sources, so verify. — [search result summary](#), [DEV](#)
- Official Spotify Feb 2026 Dev Mode changes (migration guide): Premium owner required, new apps get 1 Client ID and 5 users. Removed: batch GET tracks/albums/artists, browse, artist top-tracks, other-user profiles/playlists, POST /users/{id}/playlists (use POST /me/playlists), GET /markets. Search limit max 10 (default 5). Fields removed: track/album/artist popularity, artist followers, available_markets. Library save/follow is now generic PUT/DELETE /me/library. Playlist /tracks renamed /items, and GET items only works on playlists the user owns or collaborates on. external_ids (ISRC) was removed then reverted in March 2026. Playback state and recently-played are not listed as changed. — [Spotify migration guide](#), [Spotify blog 2026-02-06](#)
- Unverified (third-party only): a six-month refresh-token lifetime from June/July 2026 and a client-ID cap of 25 from 2026-07-23. — [vorplabs](#)

Inferences

- The user's design (open-data candidates, Spotify only for playback logging and playlist write) matches the survivors' pattern. Explo and Recommendarr show LB-derived candidates are the common substitute for Spotify recommendations.
- Dev Mode specifics matter: ISRC (external_ids) is kept, so ISRC to MBID mapping is feasible. popularity is gone, so use LB/Last.fm popularity. Playlist write needs POST /me/playlists then POST /playlists/{id}/items. The 5-user cap and Premium requirement are fine for a single user. Never depend on removed fields.

Gaps

- No verified end-to-end single-user project with learned ranker plus bandit. Not found does not mean it does not exist.
- Did not check Reddit/HN threads or awesome-lists, and did not assess Jellyfin/Plex-native "instant mix" internals.

What can be reused from scrobble-driven ecosystems (ListenBrainz stack, Maloja, Koito, Multi-scrobber, Pano Scrobber, Navidrome/Plex/Jellyfin)?

Takeaway

Reuse ListenBrainz as both a free scrobble store (via Multi-scrobber) and a candidate source (CF recommendations, LB Radio, similarity data via troi). It is actively maintained, but its recommendation endpoints are explicitly experimental.

Cited Findings

- troi-recommendation-playground: 163 stars, GPL-2.0, pushed 2026-09-28. Troi is a pipeline ("patches") playlist engine, API-first, outputting JSPF. It powers LB Weekly Jams/Exploration and LB Radio (artist/tag/stats/recommendation/playlist/collection seeds). It can resolve MBID-only playlists to a local library via Subsonic. Hosted datasets: collaborative-filtered recordings, user stats, popularity (listen counts), artist and recording similarity (Labs API). — [GitHub](#), [README raw](#), [GitHub API](#)

- listenbrainz-server: about 1030 stars, GPL-2.0, pushed 2026-10-07, release v-2026-09-28.0. — [GitHub API](#)
- LB CF endpoint `GET /1/cf/recommendation/user/{user}/recording` returns recording MBIDs with scores plus `last_updated`; HTTP 204 means not yet generated. The docs mark it experimental and "probably will change". Feedback endpoints exist (submit/delete/get ratings per recording MBID, needs user token), so explicit feedback can be written back. Docs do not describe the model. — [LB API docs](#)
- LB Radio (April 2024 release): prompt syntax with `artist:(...)`, `tag:(...)`, `#tag`, country element; easy/medium/hard modes; artist and tag playlists mostly generated in Postgres. Small countries and country origin are best-effort. — [MetaBrainz blog](#)
- ListenBrainz Music Neighborhood (Jan 2024) visualizes artist similarity. — [blog search](#)
- ListenBrainz-labs is archived (last push 2020); the old Spark-era experiments live elsewhere. Do not depend on it. — [GitHub API](#)
- Multi-scrobber: about 1255 stars, MIT, pushed 2026-10-07. Scrobbles from many sources (includes Spotify) to many clients (includes ListenBrainz/Last.fm). — [GitHub API](#)
- Koito: gabeht/Koito, about 1095 stars, MIT, pushed 2026-07-23. A self-hosted scrobble store that accepts ListenBrainz-compatible submissions. Maloja: krateng/maloja, about 1831 stars, GPL-3.0, last push 2026-08-13. — [Koito](#), [Maloja](#)
- Pano Scrobber: about 2314 stars, GPL-3.0, pushed 2026-10-06. Windows/Linux/Android scrobber for Last.fm, ListenBrainz and others. Web-scrobber about 3050 stars, MIT. Neither covers iPhone Spotify directly, so polling the Spotify API remains necessary. — [topic search](#)
- Navidrome: about 24023 stars, GPL-3.0, active. Sonic similarity comes via the AudioMuse plugin and Subsonic similar-song calls. Plexamp has built-in sonic analysis but needs Plex Pass (sources disagree on price). Jellyfin has no native sonic analysis per one comparison. — [GitHub API](#), [selfhosting comparison](#)
- Last.fm: no official deprecation found for artist/tag similarity methods (only the Radio API is deprecated). Rate limit guidance conflicts across third-party sources (about 5 req/s vs "be reasonable"); non-commercial use is free; the ToS requires rate limiting. — [Last.fm Radio API page](#), [API catalog](#)
- Python clients: pylast (772 stars, Apache-2.0, pushed 2026-10-06) and liblistenbrainz exist for the API layer. — [topic search](#), [liblistenbrainz docs](#)

Inferences

- Pull LB similar-artist/recording data and popularity as features for LightGBM, and use LB CF recs only as one noisy candidate source (experimental, may change, GPL-licensed code so call the API, do not copy).
- Mirror the user's plays to ListenBrainz (via Multi-scrobber) so LB's CF and similarity can personalize to them. Whether a single listener meaningfully shifts CF output is untested here.

Gaps

- Details of LB's similar-users feature, LB's CF algorithm, and rate limits for LB/MusicBrainz were not retrieved. The ListenBrainz blog search returned only partial results.
- Did not verify Maloja/Koito recommendation features (likely none).

What do academic/industry write-ups say about single-user or small-data personalization?

Takeaway

There is little replicated evidence specific to one user. The relevant literature is user cold-start (population priors plus few signals) and LLM-profile work, which is mostly preliminary. Plan on population priors plus content features plus implicit feedback, and evaluate offline carefully.

Cited Findings

- Deezer semi-personalized user cold-start (KDD-era 2021, arXiv 2106.03819): clusters users from heterogeneous signals and predicts new users' tastes with a DNN. It has released code and data. This is peer-reviewed with an industrial setting and is the best-matching foundational result: population-level transfer helps when per-user data is scarce. — [arXiv](#), [Deezer newsroom](#)
- LARP (arXiv 2024): language-audio relational pre-training for cold-start playlist continuation. It addresses item cold-start rather than single-user. — [search listing](#) (the ID 2406.14333 came from a search result; I did not open it to confirm the title, so treat as unverified)
- Sguerra et al., "Biases in LLM-Generated Musical Taste Profiles" (RecSys 2025, arXiv 2507.16708): 64 Deezer employees rated LLM profiles from their own data higher than random ones; the LLM choice mattered more than the time window; profiles show genre biases (rap rated lower); hallucinated tracks reported; link between profile rating and downstream recall was weak; no external baselines. Small internal sample. — [arXiv HTML](#)
- Epure et al., survey "Music Recommendation with LLMs: Challenges, Opportunities, and Evaluation" (arXiv 2511.16478, Nov 2025): organizes zero-/few-shot in-context uses for user modeling. A survey, not an experiment. — [arXiv](#)
- JKU, intent-aware LLM music recommendation: reports intent in prompts improves quality and that a Factorization Machine remains a competitive baseline (per the publication page; date not confirmed). — [JKU](#)
- TalkPlay-Tools (arXiv 2510.01698): LLM tool-calling over SQL, BM25, dense and generative retrieval for conversational recommendation. Conversational, not a single-user history setting. — [arXiv](#)

Inferences

- Evidence that LLM-only ranking beats a simple trained model on one user's history was not found. Keep LightGBM or simple baselines (recency-weighted popularity, item-item similarity) as the comparator, and treat LLM use as candidate generation or explanation.
- A single user yields few thousand labeled events at most; prefer features that transfer across items (artist similarity, tags, popularity, recency, skip history of related artists) over item IDs.

Gaps

- No replicated bandit or LightGBM-on-one-user study found. No Hacker News/Reddit real-world reports were collected (not searched). No meta-learning or MAML music papers verified.

What projects were shut down or broken by API/policy changes, and what are the resilience lessons?

Takeaway

Spotify's Nov 2024 and Feb 2026 changes broke anything relying on recommendations, audio features, popularity or other-user data, and the Dev Mode tightening shrinks apps to personal use only. Evidence of specific named shutdowns was thin in this pass.

Cited Findings

- Spotify deprecated `/recommendations`, related-artists, audio-features/analysis for apps created after 27 Nov 2024 (the endpoints return 404 with valid tokens per developer forum reports). — [DEV post](#), [Spotify community](#)
- Feb 2026: extended-quota apps unaffected, Dev Mode apps migrated on 2026-03-09, and Spotify states the motivation as risks from automation and AI. Developers objected to removals such as `external_ids` (reverted). — [Spotify blog](#), [rspotify issue](#)

- Repos that predate the removals are visibly stale (Ilmusic 2023, Spotify-Playlist-Generator 2016, moosic.me 2016, tutorial repos using audio features). — [search](#)
- Platform risk is also real for open data: ListenBrainz-labs was archived (2020), and LB's CF endpoint is flagged experimental. — [GitHub API](#), [LB docs](#)

Inferences

- Resilience tactics: store raw event logs in your own DB; key tracks by MBID/ISRC rather than Spotify ID; isolate each data source behind an adapter with a fallback (the 3-tier pattern above); cache all API responses; pin to endpoints Spotify still lists (playback, recently-played, library, playlists items) and handle `/items` rename; handle token expiry (possible 6-month refresh limit, unverified); keep the ranker able to run with missing features; and monitor Spotify's changelog.
- Dev Mode requires the owner's Premium subscription; a lapse stops the app, so the nightly job needs a health check and alert.

Gaps

- No named, verified list of shut-down projects (e.g., Spotify-dependent analytics sites) was gathered; the Medium piece on "API lock-down" was surfaced but not read: [Medium](#).
- Reddit/HN threads were not searched.

D.2 open_data_datasets.md

Open data sources, public listening datasets, and track identity resolution (as of Oct 2026)

Note: ~18 tool calls; some pages (Discogs, Spotify full changelog, MB ISRC docs, ListenBrainz Labs endpoints) were not directly retrieved. Items from my own background knowledge are labelled UNVERIFIED and carry no link.

1. Open sources for similar tracks/artists (candidates and features)

Takeaway

ListenBrainz (CF recommendations, LB Radio, Labs similarity) plus MusicBrainz (identity, tags) are the strongest open, non-commercial-friendly backbone. Last.fm similar/tag endpoints are usable for non-commercial purposes but with a 100 MB storage cap and caching duties. AcousticBrainz is shut down with no direct successor for audio descriptors, so audio features must be computed locally (e.g. Essentia) or via open embeddings.

Cited Findings

- ListenBrainz API: keep to "ONE call per second" per client; headers `X-RateLimit-Limit/Remaining/Reset-In`; 429 on exceed; a valid user token "may receive higher rate limits"; auth via `Authorization: Token ...`; a descriptive User-Agent with contact is required or requests "may be blocked without further notice"; HTTPS only; root <https://api.listenbrainz.org> — [LB API docs](#)
- LB CF recommendations: `GET /1/cf/recommendation/user/{mb_username}/recording` returns recording MBIDs + scores, supports count/offset; marked experimental; 204 if not yet generated. Recommendations are MBID-only (need resolution to a playable catalog ID). Feedback endpoints exist (`/1/recommendation/feedback/submit` etc., token required) — [LB recommendation API](#)
- LB metadata: `GET/POST /1/metadata/recording/` returns recording/artist/release/tag data for recording MBIDs; `GET/POST /1/metadata/lookup/` finds MBIDs from artist + recording (+ release) name and requires a token; manual msid->MBID mapping endpoints exist. Numeric batch limits

(MAX_ITEMS_PER_GET, MAX_LOOKUPS_PER_POST, MAX_MAPPING_QUERY_LENGTH) are named but values not given in docs page — [LB metadata API](#)

- LB Radio (troi): prompts of form `entity:values:weight:option`; entities include artist (artist + similar artists), tag, collection, playlist, stats, recs; modes easy/medium/hard control distance; global playlists contain only MBIDs; "LB Radio Local" resolves against a local collection (only artist and tag entities supported at time of source) — [Troj LB Radio reference](#), [Troj project](#)
- ListenBrainz Labs hosts similarity datasets for artists and recordings (artist-similarity returns MBID, name, strength score per a 2023 GSoC description; live endpoint paths not verified) — [GSoC 2023 ListenBrainz](#)
- ListenBrainz data dumps: public dump (`-db`), listens dump (`-full` , monthly .listens files), Spark dump, plus incremental dumps (deleted listens not removed in incrementals); full dumps twice a month; docs are inconsistent on incremental frequency (twice a week vs daily); docs page gives no license or sizes; download via [listenbrainz.org/data](#) — [LB dumps docs](#)
- MusicBrainz rate limiting: source IP limited to ~1 request/s average unless agreement; exceeding causes all requests to be declined (503); anonymous/generic User-Agents limited more tightly; every request must carry a User-Agent identifying app with contact (e.g. `MyApp/1.2.0 ( me@example.com )`); global ~300 req/s. The page states "as of 2012-01-08, and subject to change" — [MB rate limiting](#)
- Last.fm API ToS: data usable "solely for non-commercial purposes" (cl. 3.1); rate limits set at Last.fm's discretion, don't circumvent (4.4); must cache per HTTP headers (4.3.4); stored/distributed Last.fm data capped at "Reasonable Usage Cap" of 100 MB (4.3.4); delete data on termination (9.3). ToS does not mention ML/AI training — [Last.fm API ToS](#)
- Last.fm ~5 requests/s per IP averaged over 5 min, and caching of similar-artist data for at least a week — only from secondary sources (not found on the official page I fetched): [lastfm-java wiki quoting terms](#)
- AcousticBrainz: MetaBrainz ended it (data collection stopped 2022; discontinued 16 Feb 2022 per one summary); reasons: data quality not good enough, no resources to reboot; content-based similarity gave poor recommendations; site/API remained available for a time (current status of site/dumps in Oct 2026 not verified). No MetaBrainz successor for audio descriptors found; beets docs mark the AcousticBrainz plugin deprecated, suggesting beets-xtractor; Essentia (MTG/UPF) is the underlying open toolkit for local analysis — [MetaBrainz blog \(mirror\)](#), [beets plugin docs](#)
- Yambda ships open precomputed audio embeddings (CNN contrastive) for 7.72M tracks — [HF Yambda](#) (usable only for Yambda's own item IDs).

Inferences

- With no `preview_url` and no Spotify audio features, any audio-derived features need either a local audio library (Essentia/MusiCNN-type models on audio the user owns) or pretrained embeddings keyed to the dataset's own IDs; none is a drop-in AcousticBrainz replacement.
- At ~1 req/s per API, a single-user pipeline is feasible if candidates are cached aggressively (a few thousand lookups per hour per service).

Gaps

- Discogs: developer page returned HTTP 403; rate limits (UNVERIFIED from memory: 60 req/min authenticated, 25/min unauthenticated; monthly data dumps released under CC0) and API terms not verified.
- Wikidata, Bandcamp tags, Cover Art Archive: not researched (budget). Bandcamp has no public API (UNVERIFIED).
- Last.fm official rate-limit number and current key/auth requirements (API key needed for read calls — UNVERIFIED) not confirmed.
- Exact Labs API endpoints for similar artists/recordings, MB tag/relationship coverage stats, and ListenBrainz dump license/sizes not retrieved. ListenBrainz data is described by MetaBrainz as open (UNVERIFIED; check [listenbrainz.org/data](#)).

2. Public datasets for pretraining collaborative/sequential models

Takeaway

Most classic big listening datasets are no longer downloadable (LFM-1b, LFM-2b, MPD, Sequential Skip all withdrawn for license reasons). Currently accessible: Yambda (Apache 2.0, research use), ListenBrainz dumps, Music4All(-Onion) (Zenodo), and older MSD/Taste Profile. I found no study demonstrating pretrain-then-fine-tune for a single-user music recommender; the closest evidence is general transfer-learning and new-user projection work.

Cited Findings

- Yambda (Yandex): 3 sizes — 50M (10k users, 934k items, 46.5M listens), 500M (100k users, 3.0M items, 466.5M listens), 5B (1M users, 9.39M items, 4.65B listens; 4.79B events total); ~195 GB total; Apache 2.0 but "published exclusively for scientific and research purposes"; has `is_organic` flag, likes/dislikes, 7.72M audio embeddings, album/artist mapping files; `item_id` is a Yandex ID, no MusicBrainz/ISRC/Spotify mapping documented; Yandex Music catalog skews Russian/CIS — [HF dataset](#), [arXiv 2505.22238](#). Source discrepancy: collection window 11 months (paper) vs 10 months (Yandex news) — [Yandex news](#)
- LFM-2b: 2,014,164,872 listening events, 120k+ users, 2005-2020 — now "not available for download anymore due to license issues" — [JKU LFM-2b](#)
- LFM-1b: >1 billion events — "not available for download anymore due to license issues"; page does not mention MBIDs (the original paper reportedly provides MBIDs in the track/artist tables — UNVERIFIED here) — [JKU LFM-1b](#)
- Spotify Million Playlist Dataset: 1M playlists (Jan 2010-Oct 2017); Alcrowd page says dataset not available for download; contact Spotify Research (notice dated July 2024); a forum reply attributes removal to licensing (community reply, not official) — [Alcrowd MPD](#), [forum](#). Spotify IDs only (track URIs) — and would be Spotify-derived content, conflicting with the user's constraint.
- Spotify Sequential Skip Prediction: ~130M public sessions (+~30M leaderboard); not downloadable anymore, contact Spotify Research; terms not retrieved — [Alcrowd](#)
- Music4All-Onion: 109,269 tracks with 26 extra audio/video/metadata features; 252,984,396 listening records from 119,140 users (Last.fm-derived); Zenodo access category "Open"; specific license and Spotify-ID presence not confirmed — [Zenodo 6609677](#), [CIKM'22 paper](#)
- Pretrain-then-fine-tune evidence: surveys describe pretraining on interaction data then fine-tuning for downstream/cold-start tasks — [Zeng et al. survey](#); Deezer projects new users into an existing latent space instead of retraining — [Deezer](#); pretrained audio representations matter for cold-start items (self-supervised/generative backbones stronger in cold-start) — [arXiv 2604.23077](#). No source found showing single-user fine-tuning of an interaction model pretrained on a public music dataset.
- Million Song Dataset / Taste Profile (~48M user-song-count triplets, ~1M users, MSD song IDs mappable to MusicBrainz via metadata and to 7digital) — UNVERIFIED, from memory; not retrieved. MSD is from 2011 so lacks post-2011 music.
- ListenBrainz dumps: license not stated on docs page (see section 1); IDs are MBIDs when mapped (unmapped listens only have names/MSIDs).

Inferences

- The only mappable-to-MBID, current, large data are ListenBrainz dumps (native MBIDs) and Last.fm-derived Music4All-Onion (names/IDs need mapping). Yambda is the best-documented modern sequence benchmark but its IDs cannot be tied to the user's catalog except via audio embeddings or title matching (title metadata not documented), so it is useful mainly for architecture pretraining/ablation, not for item-level transfer.

- A realistic single-user design: pretrain item/sequence embeddings on LB dumps (MBID space), then fit a light user vector (projection/few-shot) rather than full fine-tuning. This is my inference, not demonstrated in the literature found.

Gaps

- Licenses for LB dumps, Music4All-Onion; whether Music4All carries Spotify IDs; Yambda track metadata (titles) availability; Taste Profile current hosting; LFM-1b MBID fields; any empirical single-user transfer study.

3. Track identity resolution (Spotify URI <-> ISRC <-> MB recording MBID)

Takeaway

ISRC is the best join key but is imperfect: MusicBrainz docs say remasters/remixes/edits get their own ISRC, and ISRC data is messy at scale. Use a layered approach: ISRC -> MB (and LB MBID mapper) -> fuzzy artist+title(+duration) fallback, with human-reviewable confidence. I found no authoritative published match rates.

Cited Findings

- MusicBrainz ISRC doc: same recording should carry the same ISRC across territories; different recordings, edits, remixes and remasters get their own ISRC — [MB ISRC terminology](#)
- UK government metadata report: 4-11% of recordings had more than one ISRC in a ~5M-recording dataset — [GOV.UK report summary](#)
- Practitioner write-up on matching ~90M tracks across six platforms with ISRCs plus fuzzy matching and "what breaks" (not read in full) — [DEV Community](#)
- LB `/1/metadata/lookup/` resolves artist/recording/release names to MBIDs (token required) — [LB metadata API](#). (LB's MBID mapper is a name-based matcher; ISRC/Spotify-ID reverse lookup not found in the docs page — UNVERIFIED.)
- MB `GET /ws/2/isrc/{ISRC}` returns linked recordings, possibly several (UNVERIFIED from memory; not retrieved). MB ISRC coverage % not found.

Inferences

- Failure modes to expect: remaster/reissue ISRC differs from original MB recording; live/remix/regional variants have different ISRCs and may be missing in MB; one ISRC linked to multiple MB recordings; compilation/"feat." title formatting differences hurting fuzzy matching.
- Cache all mappings locally; log unresolved tracks and measure own match rate on the user's library (the only reliable estimate).

Gaps

- Published match rates for Spotify->MBID; ListenBrainz mapper accuracy; MB ISRC coverage; handling guidance in MetaBrainz forums/blog (not searched).

4. Does Spotify still allow `isrc:` search after Feb 2026?

Takeaway

Search `limit` max was cut from 50 to 10 and default from 20 to 5 (Feb 2026 migration guide); the guide does not mention removing `isrc:` filtering, and `external_ids` (which carries the ISRC on tracks) was removed in

February but reverted in March 2026. So ISRC lookup via `q=isrc:<code>&type=track&limit=10` is plausibly still possible, but I found no 2026-dated confirmation; test it directly.

Cited Findings

- Search: `limit` max 50 -> 10, default 20 -> 5; paginate with `offset` — [Feb 2026 migration guide](#), [Feb 2026 changelog](#)
- Track/album `external_ids` removal was reverted: "will continue to be available" — [March 2026 changelog](#)
- Same guide also removed batch `GET /tracks`, `/artists`, `/albums` (fetch one by ID), artist top-tracks, browse endpoints; removed Track `popularity`, `available_markets`, `linked_from`; Artist `popularity`, `followers`; playlist items readable only for user-owned/collaborative playlists. Dev Mode: owner needs Premium; new apps limited to 5 users per app; Client ID limit raised to 25 in July 2026 — [migration guide](#)
- Community note that the search cut was not in the changelog page itself (only the migration guide) — [Spotify community thread](#)
- Historical `isrc:` behaviour (pre-2026, undated threads): sometimes empty results for valid ISRCs, filter not a free-text term, OR of multiple ISRCs fails, dashed/undashed formatting issues, quotes sometimes help — [thread 1](#), [thread 2](#)

Inferences

- Because batch `GET /tracks` is gone, mapping N tracks costs N calls either way; for MBID->Spotify, use one `isrc:` search per track (limit 10 is enough since ISRC matches few results), verify `external_ids.isrc` in the response, fall back to `track:... artist:... search`.

Gaps

- No 2026-confirmed statement that `isrc:` search works; recommend a live test. The changelog states not seen: whether `/search` itself is restricted in Dev Mode beyond the limit change.

D.3 modeling_methods.md

Modeling methods for a single-user music recommender (as of Oct 2026)

Evidence grades: **[R]** replicated or multi-source, **[S]** single paper, **[B]** blog or secondary source, **[I]** inference by researcher. Items marked "(background, unverified this session)" come from my prior knowledge and were not confirmed by a source in this research pass. Treat them as leads. Search tooling was weak: it often returned only abstracts or summaries, and my budget was about 15 calls. Coverage is uneven, and the Gaps sections say where.

Sequence and session-based recommenders (SASRec, BERT4Rec, GRU4Rec, HSTU, Mamba, LLM), and simple baselines

Takeaway

There is solid evidence that reported neural gains are fragile, and that well-tuned simple baselines (item-kNN, decayed popularity) are competitive. On music specifically, Yambda's own baseline table shows ItemKNN beating or matching SASRec at the 50M scale. I found no source showing HSTU, Mamba or LLM recommenders work on a single user's data, so for one user the practical stack is kNN/popularity-decay candidates plus a small learned ranker, with an optional transformer pretrained on public data.

Cited Findings

- Dacrema et al. (the "worrying analysis" paper) examined 18 neural top-n algorithms from top conferences. Only 7 could be reproduced with reasonable effort, and 6 of those 7 were often beaten by simple nearest-neighbor or graph-based heuristics. The seventh did not consistently beat a well-tuned non-neural linear model. Code is on GitHub (MaurizioFD/RecSys2019_DeepLearning_Evaluation, AGPL-3.0, last push 2023-05). Grade [R] for the general pattern, and it is predominantly non-sequential models. — [alphaxiv summary](#); [DeepAI](#); [repo](#)
- Petrov and Macdonald (RecSys 2022) found that BERT4Rec's results varied across publications. Default configurations of the available implementations did not reproduce the original numbers. The original code needed up to about 30x longer training. Their Hugging Face-based reimplementation reproduced the original results on 3 of 4 datasets with up to 95% less training time. They conclude BERT4Rec is state of the art only when trained long enough. Grade [S] with corroborating context. Training budget and objective matter more than the architecture. — [arXiv 2207.07483](#); [Glasgow](#)
- gSASRec (Petrov and Macdonald, RecSys 2023) targets the overconfidence of sequential recommenders trained with negative sampling. An official PyTorch port exists: asash/gSASRec-pytorch, Apache-2.0, last push 2024-02, about 60 stars. — [repo](#)
- Betello et al. (reproducibility study, 2024; IEEE Access 2025) found SASRec does not consistently beat GRU4Rec. SASRec dominates only once parameter counts become substantial. They stress that non-standardized experimental design leads to flawed conclusions. — [arXiv 2408.03873](#)
- Yambda (Yandex, 2025) is a music dataset with 4.79B interactions from 1M users and 9.39M tracks. It comes in 50M, 500M and 5B versions: 10k users and 46.5M listens, 100k users and 466.5M listens, and 1M users and 4.65B listens respectively. It has timestamps, likes and dislikes, audio embeddings, and an `is_organic` flag that separates user-initiated plays from recommendation-driven ones (about 48.7% of listens are recommendation-driven). The protocol is a global temporal split: 300 days train, 30-minute gap, 1 day test. The paper's license is CC BY 4.0, and the dataset page says research use only. I did not verify the dataset license text. — [arXiv 2505.22238](#); [HF dataset](#)
- Yambda baselines, NDCG@10, Listen+ task:

Model	50M	500M	5B
MostPop	0.0186	0.0173	0.0175
DecayPop	0.0260	0.0267	0.0271
ItemKNN	0.0781	0.0708	not run
iALS	0.0407	0.0384	0.0388
BPR	0.0389	0.0400	0.0408
SASRec	0.0748	0.0754	0.0647

On the Like task, DecayPop was the strongest at NDCG@10 (0.0165 at 5B, vs SASRec 0.0136). ItemKNN and SANSA were computationally intractable at the larger scales. The authors' conclusion that transformers are needed is only partly supported by their own tables. Grade [S], but it is a music dataset, which is directly relevant. — [arXiv 2505.22238](#)

- HSTU ("Actions Speak Louder than Words", Meta 2024) claims up to 65.8% NDCG gains over baselines on synthetic and public data. Its headline claims concern trillion-parameter scaling and industrial data. I found no replication on small data. The code is at facebookresearch/generative-recommenders (now

meta-recsys/generative-recommenders), Apache-2.0, actively pushed as of 2026-10-07. — [arXiv 2402.17152](#); [repo](#)

- A 2026 paper reports that HSTU trained on a random 10% of data retains NDCG@10 of 0.700 relative to full data. This is shrinking subsets, not intrinsically small datasets. — [arXiv 2604.07739](#)
- A 2026 reproducibility study of generative recommenders under cold-start protocols says gains are hard to interpret because model scale, item ID design and training strategy are changed together. — [arXiv 2603.29845](#) (found via a search listing only, so the details are thin).
- Synerise's BaseModel-vs-HSTU post argues MovieLens is a poor benchmark for sequential recommendation because its temporal signal is noisy. Grade [B], vendor with a competing product. — [post](#)
- Mamba4Rec has an implementation at chengkai-liu/Mamba4Rec (MIT, last push 2025-04), built on RecBole. I found no independent replication. — [repo](#)
- LLM-based recommenders: [arXiv 2506.15833](#) and [2507.05733](#) use SASRec as a strong baseline. The latter notes fair comparison needs matched backbones, prompts and repeated runs. Replication of LLM baselines is described as incomplete. No source I found supports an LLM recommender beating SASRec or kNN on one person's music history. — [2506.15833](#); [2507.05733](#)
- LFM-2b (Schedl et al., CHIIR 2022) has more than 120k users and 15 years of Last.fm history, with Spotify ID mappings. The host page says it is no longer downloadable because of license issues. Check the current status before relying on it. A follow-up analysis of LFM-1k, LFM-2b and the Spotify Music Streaming Sessions Dataset reports that results on time-based subsets deviate from the full dataset. — [CHIIR paper](#); [dataset page](#); [arXiv 2509.09685](#)
- Open-source library status, from the GitHub API on 2026-10-07 (last push; license):

Library	License	Last push	Notes
RecBole	MIT	2025-02-24	About 4.6k stars; slow-moving
Microsoft Recommenders	MIT (per prior knowledge)	pushed today per search listing (about 21.9k stars)	License not verified via API; rate-limited
LensKit (Ilkpy)	NOASSERTION	2026-09-30	Active
NVIDIA Merlin	Apache-2.0	2026-07-22	
Transformers4Rec	Apache-2.0	2026-08-23	
implicit	MIT	2026-05-08	About 3.8k stars; ALS/BPR/kNN
TorchRec (meta-pytorch)	BSD-3-Clause	2026-10-07	Built for large-scale embedding sharding; overkill for one user
ReChorus	MIT	2026-02-11	
Elliot	Apache-2.0	2026-09-22	
EasyRec	Apache-2.0	2026-04-15	
kang205/SASRec (original TF)	Apache-2.0	2023-08	Stale
hidasib/GRU4Rec_PyTorch_Official	none declared	2023-08	Stale

RecPack: I could not locate the repo under the names I tried, so its status is unknown.

Inferences

- [I] At one user's scale (10^3 to 10^5 plays, maybe 10^3 distinct tracks heard repeatedly), a transformer has far too little data to learn item embeddings from scratch. Yambda shows SASRec only matches ItemKNN at 10k-user scale. The practical options are (a) item-kNN or co-occurrence on your own sessions, (b) decayed popularity or recency of plays, (c) a gradient-boosted ranker over features (recency, play count, time of day, audio embeddings, artist), or (d) a SASRec-like model pretrained on a public dataset, with only a light fine-tune or a user-state head on your data.
- [I] The pretraining route needs item-ID alignment: map your tracks to the public catalog via Spotify/ISRC IDs, or use content embeddings (Yambda ships audio embeddings). Items outside the catalog need a content-based fallback. I found no evidence of the benefit of this transfer for a single user.
- [I] The Yambda finding that DecayPop is best for Likes suggests recency-weighted repeat consumption is a very strong baseline. Personal listening is highly repetitive, so always test "replay what I recently played" first.
- [I] "LightGBM beats neural" is plausible but I found no music-specific replication in this pass. (Background, unverified: LightGBM-based rerankers were common in top RecSys Challenge entries.)

Gaps

- No source on HSTU, Mamba or LLM recommenders trained on single-user data, or on fine-tuning a pretrained sequential model for one user.
- GRU4Rec vs SASRec replication details, and the "GRU4Rec implementations are flawed" debate (Hidasi and Czapp), were not retrieved. (Background, unverified.)
- Exact license and last-commit for Microsoft Recommenders, RecPack, and LightGBM were not confirmed because of API rate limiting. The cited Yambda numbers come from an LLM-summarized fetch, so check them against the paper tables before final use.

Skip prediction and implicit-feedback modeling

Takeaway

Skip prediction was well studied through the WSDM Cup 2019 (Spotify), where RNN-based solutions with interaction and track-metadata features won, and later analysis found a temporal data leakage problem in that dataset. Spotify's recommender work treats a 30-second stream as the success threshold, so skips act as noisy negatives rather than true dislikes.

Cited Findings

- The first-place team (Zhu and Chen, Ctrip, "ekffar") used RNNs with user-interaction features and song metadata. They report a single-network Mean Average Accuracy of 0.648 on the withheld test set, and ensembling variants improved it further. Self-reported. — [arXiv 1902.04743](#)
- DIKU-IR (Copenhagen) placed 2nd of 45 teams with a Multi-RNN design and MAA 0.641. — [arXiv 1903.08408](#)
- Sainath Adapa's solution placed 7th. — [repo listing](#)
- Meggetto et al. (CHIIR 2023), "Why People Skip Music?", uses deep RL for track-by-track skip prediction on Spotify data. User behavior features were the most discriminative, with content and context features less so. The analysis reveals a temporal data leakage problem in the dataset. Grade [S]. — [arXiv 2301.03881](#)
- A later contrastive-learning approach treats skips as informative negative signals for sequential music recommendation (same research line as the LFM-2b analysis). Grade [S]; I only saw its abstract summary. — [arXiv 2509.09685](#)

- Spotify's Home bandit (BaRT, McInerney et al., RecSys 2018) defines reward as a stream of at least 30 seconds, with the reward personalized by content type. This comes from slide-deck and blog summaries of the paper. Grade [B] for the details. — [Lalmas slides](#)

Inferences

- [I] For one user, a skip label is only useful once conditioned on context (position in queue, whether the track was autplayed vs chosen, time of day, mood). The skip-prediction winners mostly used session-position and previous-skip features, which is a strong autoregressive signal. That is partly "skip begets skip", not taste.
- [I] Practical label design: use a graded target (completed, played over 30s, skipped under 30s, skipped under 5s) with a hierarchical or ordinal loss. Consider three states: positive (replayed, saved, completed repeatedly), neutral and negative (early skip in a non-shuffled context). Treat single early skips as weak evidence.
- [I] Check for temporal leakage in any skip features you build: do not use future-in-session information.

Gaps

- Spotify's own papers on skips and satisfaction (e.g., on user satisfaction metrics, "implicit negative feedback") were not retrieved. My search for a satisfaction paper surfaced only Meggetto. I could not verify specific Spotify publications beyond BaRT. Hence no firm quantitative view on skip-as-negative noise rates.
- The skip-challenge feature lists (e.g., top features, GBDT vs RNN comparisons) were not extracted.

Exposure and position bias (IPW, counterfactual LTR, doubly robust)

Takeaway

I did not retrieve direct evidence on feasibility with one user's logs. The reasoning below is inference. Yambda's `is_organic` flag is the one concrete, public handle on exposure confounding in music data.

Cited Findings

- Yambda separates organic from recommendation-driven events, with about 48.7% of listens recommendation-driven. — [arXiv 2505.22238](#)
- The Open Bandit Dataset provides true behavior-policy propensities and is built to benchmark OPE estimators. A summary says one well-established estimator fails there, with the name not confirmed. Grade [S]. — [arXiv 2008.07146](#); [docs](#)

Inferences

- [I] IPW and counterfactual LTR need logged propensities, i.e. the probability the system showed an item. For a single user listening via Spotify you cannot observe Spotify's propensities. IPW therefore works only for a system you control: if your recommender picks the tracks, you log its own probabilities.
- [I] To get usable propensities, randomize a small fraction (epsilon-greedy or softmax with logged probabilities). This also provides unbiased data for evaluation. This ties exploration and evaluation together.
- [I] With a small sample, clipped or self-normalized IPS and doubly robust estimators trade bias for variance. With a few hundred exploratory plays, confidence intervals will be wide. (Background, unverified.)
- [I] Position bias within your own interface (queue order, first-slot effects) can be estimated cheaply with a randomized-order swap, but only if you present lists.

Gaps

- No retrieved source on position-bias estimation or counterfactual LTR in music, or with $N=1$ users. Nothing found on propensity estimation from passive logs of a platform you don't control.

Exploration (bandits and libraries)

Takeaway

Bandit libraries are available and licensed permissively. For one user, the main constraint is statistical (few exploratory plays), not software. Thompson sampling or epsilon-greedy over a candidate pool with logged propensities is enough. Neural bandits are not justified at this scale.

Cited Findings

- BaRT (Spotify, RecSys 2018, "Explore, Exploit, Explain") is a contextual bandit for Home with explanation features. A coin-flip exploit/explore description appears in a slide deck, a simplification. Grade [B]. — [Lalmas slides](#)
- Library status (GitHub API, 2026-10-07):

Library	License	Last push	Notes
Open Bandit Pipeline (st-tech/zr-obp)	Apache-2.0	2024-06-03	About 712 stars; effectively unmaintained for 2 years
MABWiser (fidelity)	Apache-2.0	2024-09-05	sklearn-style contextual bandits; stale
Vowpal Wabbit	NOASSERTION per API (BSD-3-Clause per prior knowledge, unverified)	2026-09-28	Active; contextual bandit and CB-explore
River	BSD-3-Clause	2026-10-07	Active; online learning, includes bandit module
VowpalWabbit/reinforcement_learning	MIT	2026-03-03	

RecoGym: not checked. — [OBP docs](#); [MABWiser](#)

Inferences

- [I] Practical budget: reserve roughly 5 to 15% of slots for exploration, which is my suggestion and not a sourced figure. At 20 plays per day and 10% exploration you get about 2 exploratory plays per day, so only a few hundred after a year. This supports simple Beta-Bernoulli Thompson sampling over clusters, genres or artists rather than per-track contextual models.
- [I] Explore on the unplayed side of the catalog: new tracks and artists adjacent to your taste cluster, since repeat tracks need no exploration.
- [I] For a personal system, VW or River is the lowest-friction option because they are active. OBP is mainly useful for its OPE estimators, and its age is a risk with newer numpy/sklearn.

Gaps

- No Spotify papers beyond BaRT were retrieved (e.g., Home explore/exploit follow-ups, slate bandits). No evidence found on neural bandits at small N , or a recommended exploration budget from a source.
- RecoGym status and license were not checked.

Imitation learning, inverse RL, offline RL

Takeaway

I found no credible evidence that RL or imitation beats supervised ranking on implicit feedback, and the evaluation literature explains why the comparisons are unreliable. Behavior cloning on your own history is just next-item prediction (what SASRec already does), with the same exposure-bias limitation.

Cited Findings

- Deffayet et al. (SIGIR Forum 2022): offline evaluation of RL-based recommenders typically uses a next-item prediction protocol with three listed shortcomings. It cannot show the benefit RL is meant to bring and it hides deficiencies of some offline RL agents. — [arXiv 2301.00993](#)
- Xiao and Wang (2023) report offline RL beating supervised and RL baselines on two public datasets. The supervised baseline is next-item prediction, which does not maximize cumulative reward, so the comparison favors the RL objective. Grade [S], self-reported. — [arXiv 2310.00678](#)
- Meggetto et al. use deep RL for skip prediction and report beating prior models, on a dataset they found had temporal leakage. — [arXiv 2301.03881](#)

Inferences

- [I] Imitation learns "what I would have listened to under the exposure I had". If most of your plays came from algorithmic feeds, you clone Spotify's policy as filtered by your acceptance. That inherits exposure bias, and it cannot tell you about items you were never shown. Yambda's organic flag lets a public-data experiment separate the two.
- [I] The one real argument for RL-style objectives is long-horizon session value (e.g., arc of a playlist). Validating that needs a simulator or live testing, which is expensive for one user. I would defer it.

Gaps

- No search for inverse RL or offline RL on playlists/sessions specifically (e.g., playlist continuation RL). No head-to-head replication study of RL vs supervised ranking found.

Multi-interest and short/long-term fusion, time decay

Takeaway

I did not research this area beyond one dataset result, so there is little sourced evidence. The only sourced signal is that DecayPop clearly beats MostPop, which supports time decay as a cheap and strong component.

Cited Findings

- On Yambda, DecayPop (NDCG@10 0.0271 at 5B) is roughly 1.5x MostPop (0.0175) on Listen+, and about 6x on Like (0.0165 vs 0.0026). — [arXiv 2505.22238](#)
- Time-based subsets of LFM-2b yield results that deviate from the full dataset, so the choice of history window changes conclusions. — [arXiv 2509.09685](#)

Inferences

- [I] MIND and ComiRec are designed for millions of users and item corpora. For one user, a cheaper equivalent is clustering your listened-track embeddings (audio or co-listening) into a handful of taste

clusters and scoring candidates by max similarity to cluster centroids weighted by recency. This is untested here and not sourced.

- [I] Time decay: tune the half-life per feature with a time-split validation, and expect a short half-life for mood and context and a long half-life for stable taste.

Gaps

- MIND, ComiRec and long/short-term fusion papers were not retrieved. No evidence on whether multi-interest models help N=1.

Offline evaluation for one user and a blind A/B versus Discover Weekly

Takeaway

Use a global temporal split, not leave-last-out. Leave-one-out lets training data overlap the test period and can distort model rankings. With one user, statistical power is low, so compare few methods with pre-specified metrics and paired designs.

Cited Findings

- Ji et al. (ACM TOIS 2023, arXiv 2010.11060): data leakage arises from not respecting the global timeline. Leave-last-one-out splits with BPR, NeuMF, SASRec and LightGCN showed accuracy and relative model order changing unpredictably depending on how much future data was in training. — [arXiv 2010.11060](#)
- "Time to Split" (RecSys 2025): leave-one-out has lower correlation with real-world evaluation and can distort model rankings. The authors recommend a global temporal split with Last or Random targets, and note Successive is more expensive. — [arXiv 2507.16289](#)
- Sun (2023) argues that ignoring the global timeline causes leakage and oversimplified preference modeling. — [arXiv 2210.04149](#)
- Yambda uses a global temporal split with a gap between train and test, with model state frozen at test start. — [arXiv 2505.22238](#)

Inferences

- [I] For one user, use rolling-origin evaluation: train on weeks 1..k, test on week k+1, repeat. Report mean and spread across folds and use a paired bootstrap over days or sessions (not over plays, which are correlated within sessions). Include a gap to avoid session spill-over.
- [I] The offline target is confounded: plays you made are plays you were exposed to. Offline metrics reward recommending what Spotify would have shown. Treat offline numbers as a filter that removes bad models, not as proof a model is better than Discover Weekly.
- [I] Minimum data (my estimate, unsourced): a few thousand plays with 10+ weeks gives crude model ordering of only large effects (e.g., kNN vs random). Distinguishing similar models needs far more.
- [I] Blind A/B vs Discover Weekly: put 30 tracks in each arm (Discover Weekly's 30 per week; match length), exclude tracks you already know from both arms, shuffle the combined order with randomization logged, and strip the source label (a helper can queue tracks from a hidden list). Measure the pre-specified outcome per track (completed >30s, explicit rating, save). Use paired weekly comparisons over 8 to 12 weeks, analyzed with a permutation test or a mixed-effects model with track and week effects. With roughly 30 tracks per arm per week and a base hit rate of 30%, ten weeks gives only about 300 per arm, enough to detect a gap of roughly 10 points or more, not 3. (Rough power estimate, not computed from a source.) Pre-register the metric. Note Discover Weekly contents change each Monday and are already personalized to your history, including any plays you generate while testing.
- [I] Novelty confound: a recommender restricted to known tracks beats DW on hit rate trivially. Compare on unfamiliar tracks only, or report novelty-adjusted hit rate.

Gaps

- No source on OPE sample-size requirements or on minimum data for a reliable single-user offline evaluation. The power numbers are my own approximation.
- I did not find an existing published blind-comparison design versus Discover Weekly.
- Details on SNIPS and doubly robust performance on small samples were not retrieved from sources.

D.4 representations_diversity.md

Music representations, context/mood, and diversity for a personal recommender (as of Oct 2026)

Grading tags: [replicated] [single paper] [anecdote/secondary] [inference]. Many facts came via search-summary or mirrors; items marked (unverified) were not read on a primary page. Not legal advice.

Q1. Audio embedding models: licenses, size, quality, legit audio sources

Takeaway

All strong open music encoders (MERT, MuQ, MuQ-MuLan, Essentia models) ship weights under non-commercial licenses (CC-BY-NC family), which is fine for a private personal tool; Apache-2.0 LAION-CLAP and the MIT MusicFM code are the permissive exceptions (MusicFM weights license not stated). Spotify content is out for ML by contract, Deezer/Apple previews are effectively unusable for caching/embedding, so the legitimate audio is the user's own files plus CC-licensed corpora. Pretrained-embedding quality on MIR benchmarks does not predict recommendation quality.

Cited Findings

- MERT-v1-330M: license cc-by-nc-4.0, 330M params, 24 kHz, 1024-dim hidden states over 25 layers, needs `trust_remote_code=True` [single source: model card] — [HF MERT-v1-330M](#)
- MuQ: code MIT; "model weights (MuQ-large-msd-iter, MuQ-MuLan-large) are released under the CC-BY-NC 4.0 license"; ~0.3B params; public checkpoint trained on Million Song Dataset and may underperform paper numbers — [HF MuQ-large-msd-iter](#); paper [arXiv 2501.01108](#)
- MusicFM (minzwon): code MIT; weights license not stated on README; README says released model was trained on FMA-large (MSD variant also exists) "to avoid licensing complications"; 24 kHz input, 25 Hz frame output; maintenance unclear (25 commits, checkpoint fix Feb 2024) — [GitHub musicfm](#)
- LAION-CLAP: repo CC0-1.0; music checkpoint `laion/larger_clap_music` Apache-2.0 on HF; repo self-described as work in progress, 64 open issues; zero-shot GTZAN 71% for music-audioset model — [GitHub LAION-AI/CLAP](#), [HF larger_clap_music](#)
- Essentia: library AGPLv3 "for non-commercial applications" (commercial license from UPF); pretrained models CC BY-NC-ND 4.0 per licensing page, CC BY-NC-SA 4.0 per models page (the two pages disagree) — [Essentia licensing](#), [Essentia models](#)
- Essentia model zoo offers MSD-MusiCNN, Discogs-EffNet (artist/label/release/track variants), MAEST, OpenL3 (8 variants, 512 or 6144-d), AudioSet-VGGish; mood classifiers sit on top of these embeddings — [Essentia models](#)
- Layer-wise study of 12 public music foundation models (MERT x2, MusicFM, MuQ, OMAR-RQ, MusicGen, YuE, CLAP, Myna) exists (2026) [single paper, only abstract-level read via search] — [arXiv 2608.14819](#)
- MTG-Jamendo linear probe (MERT v2 card): MuQ top-50 AP 30.2, MERT-Large 29.1, MusicFM best on genre AP 19.4; differences about 1 point [single source, model card] — [HF MERT-v2-30s](#)

- Recommendation-specific benchmark: Tamm & Aljanaki evaluate 9 pretrained backends (MusicFM, Music2Vec, MERT, EncodecMAE, Jukebox, MusiCNN, MULE, MuQ, MuQ-MuLan) with KNN, shallow NN, contrastive projection, hybrid, BERT4Rec in hot and cold start; finding: "significant performance disparity" between MIR tasks and recommendation. Search summary of the paper reports supervised MusiCNN is strong in hot start while generative/self-supervised (Jukebox, MusicFM) are better cold-start; MERT strong but wide CIs (secondhand, verify in full text). Extended from RecSys'24 work; accepted ACM TORS — [arXiv 2604.23077](#), [arXiv 2409.08987](#) [single group, two versions]
- CLAP embeddings fed into graph-based CF for cold start reported promising (RecSys'24 Music Recommender Workshop) [single paper] — [arXiv 2409.09026](#)
- Spotify Developer Terms IV.2.1.a prohibit "to train a machine learning or AI model" with Spotify Content; previews are defined as 30-second clips — [Spotify Developer Terms](#)
- Deezer guidelines: API gives "30s. extract"; "Local Storage/Offline Storage of audio data is strictly forbidden." Page does not address ML; I could not retrieve Deezer's full ToU (404 on /termsofservice) — [Deezer guidelines](#). Deezer publicly signed a statement opposing unlicensed AI training (press stance, not a contract term) — [Deezer newsroom](#)
- Apple promo content (iTunes Search API previews): must be "streamed only" and never "downloaded, saved, cached", only on pages promoting the content with a store badge, and no independent entertainment value (paraphrased) — [Apple Performance Partners Search API](#)
- MTG-Jamendo: audio under per-track Creative Commons licenses; metadata CC BY-NC-SA 4.0; "solely for non-commercial research and academic use"; 59 mood/theme tags — [GitHub mtg-jamendo-dataset](#)

Inferences

- Personal, non-redistributed use makes NC licenses (MERT, MuQ, Essentia models) low risk; the NC terms only bite if the tool is shared/commercialized. If it might ship publicly, prefer LAION-CLAP music (Apache-2.0) and verify MusicFM weights.
- Best legit audio route: embed the user's own purchased/ripped-by-them local files (user's own copy; I found no source specifically addressing this and it is jurisdiction dependent) and CC corpora (FMA, Jamendo). Streaming-service previews (Spotify, Deezer, Apple) should be treated as unusable for embedding: Spotify bans ML, Deezer bans audio storage, Apple bans caching/entertainment use.
- Compute: ~0.3B-parameter encoders on 30 s clips are feasible on a laptop/consumer GPU (inference only; CPU slow but workable for a few thousand tracks). No benchmarked timing found; this is inference.
- Start with a cheap model (MusiCNN/Discogs-EffNet via Essentia) before a large foundation model, given the MIR-vs-recsys disparity.

Gaps

- No primary read of the Tamm & Aljanaki results tables (datasets, exact rankings); only abstract plus search summaries.
- Newer 2025-26 models beyond those listed (e.g., OMAR-RQ, Myna appear only as names in the layer-wise study); I did not verify their licenses or recsys performance.
- Deezer's full Terms of Use and Apple's/Deezer's exact ML language not retrieved. OpenL3 and MuLan license not verified.
- Actual laptop timings/VRAM figures not found.

Q2. No-audio fallbacks (tags, graphs, lyrics, collaborative)

Takeaway

Evidence says tags/listening data are often as good as or better than audio alone, and hybrids (tags/CF + audio) beat either alone, especially for long-tail/cold start; results depend heavily on split (by artist vs random). For a

single user, public collaborative data (ListenBrainz) and tag/metadata embeddings are the practical no-audio substrate.

Cited Findings

- Tag-derived artist embedding beat audio CNN for cold-start song recommendation (MAP 0.0049 vs 0.0015; late-fusion ~0.0036), but absolute numbers are tiny [single paper, 2017] — [arXiv 1706.09739](#)
- Hybrid tag+audio (LSA) outperformed tag-only on quality and long-tail discovery, with Last.fm user data [single group, older] — [RGU repository](#)
- ISMIR 2020 mood-recommendation poster: listening-based embeddings beat audio (MusiCNN), but with artist-based splits the listening embeddings lose their advantage over audio [single paper] — [ISMIR 2020 poster](#)
- Comparison of pretrained audio representations: content+collaborative tends to beat pure collaborative; KNN results vary widely across embeddings — [arXiv 2409.08987](#) is a different (2018) comparison of representation strategies; the 2024 one is [arXiv 2409.08987](#)
- Deep content-user model paper: choice of loss/learning strategy seems to matter more than input modality [single paper, 2018] — [arXiv 1807.06786](#)
- ListenBrainz Listens Dataset: >800M entries with timestamp, pseudonymous user id, track metadata, optional MusicBrainz IDs (ISMIR 2024) — [ISMIR 2024 poster](#). Exact license not verified (community posts only).
- Musical Word Embedding links listening contexts (text) with music [single paper] — [arXiv 2008.01190](#)
- Playlist continuation study claims audio helps in cold start versus metadata (venue/year unconfirmed) — [CEUR Vol-4045](#)

Inferences

- MusicBrainz is useful mainly as an ID bridge (MBIDs link ListenBrainz, tags, relationships); Last.fm tags are the richer tag source. No head-to-head of MusicBrainz tags vs audio found.
- For one user with few hundred to few thousand plays, node2vec/LightGCN on personal data alone is data-starved; pretraining/borrowing item embeddings from public ListenBrainz co-listening is the logical route (my inference; not tested in any source found).

Gaps

- No source on lyric/theme text embeddings for recommendation retrieved; no direct evidence for node2vec/LightGCN on music listening graphs retrieved (only that LightGCN is a RecBole built-in).
- Million Playlist Dataset availability/license not verified (Spotify-origin; likely restricted given ML ban, unverified).
- Last.fm API terms for tags not checked.

Q3. Mood/context features and context-aware rankers

Takeaway

Time-aware and session-aware modeling reliably helps somewhat in older studies, but I found little modern, replicated evidence for which specific context features (time of day, day of week, device) move metrics for music; mood regression is easier for arousal than valence.

Cited Findings

- Session-based CF with temporal information significantly increased hit ratio and MRR vs plain session CF [single paper, 2013] — [IEEE 6735331](#)

- Time-aware CF with per-period "micro-profiles" via contextual pre-filtering, validated on Last.fm [single paper; numbers not read] — [arXiv 2008.11432](#)
- Context-aware recsys difficulties: hard-to-detect factors and sparsity when pre-filtering on many factors [secondary] — via search summary of a 2021 paper (not independently opened)
- Time zone inference from Last.fm scrobbles: ~75% correct within +/-1 hour using sleep-gap assumption (N=594K, 2014) — [UPF MTG](#) (likely source; unverified link match)
- Music2Emo (Kang & Herremans 2025): frozen MERT (layers 5-6, ~95M param variant) plus chord/key features, trained jointly on 56 MTG-Jamendo tags and valence-arousal regression; used DEAM, EmoMusic (744 clips), PMEmo — described in [arXiv 2608.25621](#) (a follow-on paper reporting small R2 gains of ~0.005-0.012)
- Across the literature arousal is easier to predict than valence [secondary] — [arXiv 2202.10453](#)

Inferences

- For a single user, hour-of-day/day-of-week as features into a GBDT or FM over track-embedding similarity is cheap and plausible, but I have no evidence quantifying the lift; plan an offline test on own scrobbles.
- Mood tags from Essentia classifiers or MTG-Jamendo-trained probes are noisy, especially valence; use as soft features, not hard filters.

Gaps

- No evidence retrieved on DeepFM/FM/contextual GBDT in music specifically, activity inference, or device context. No effect sizes for time-of-day features.
- No MERT linear-probe numbers on EmoMusic found.

Q4. Diversity, novelty, serendipity, anti-narrowing

Takeaway

Re-ranking is well-established (MMR, DPP greedy MAP, Steck calibration); music-specific field evidence (Spotify, Deezer) shows effects are user-dependent, recommendation-driven listening correlates with narrower consumption, and homogenization is measurable in simulation. Long-term narrowing metrics for an individual user are mostly inference.

Cited Findings

- Anderson et al. WWW 2020 (Spotify): higher consumption diversity associated with retention/conversion (observational); algorithm-driven listening associated with reduced diversity; users who diversified mostly did so via organic consumption; recsys more effective for low-diversity users [single study] — [Anderson et al.](#)
- Holtz et al. (Spotify podcasts, field experiment): personalization raised streams 28.90% but cut individual diversity 11.51% (aggregate diversity +5.96%); retention not measured [single experiment, podcasts] — [arXiv 2003.08203](#)
- Deezer/CNRS (RecSys 2021, ~9,000 users): effect of recommendation on diversity "depends on users"; propose "filter niches" over bubbles; algorithmic vs editorial recommendations play different roles — [Deezer research](#), [arXiv 2109.03915](#); follow-up in Sci. Reports (Shakespeare, Chareyron, Roth, 2025), findings not read — [DOI 10.1038/s41598-024-75967-0](#)
- Chaney et al. RecSys 2018: algorithmic confounding amplifies homogenization without utility gains; replicated/critiqued in simulation papers (T-RECS, "Simulation as Experiment") — [arXiv 1710.11214](#), [arXiv 2107.14333](#), [arXiv 2107.08959](#) [replicated in simulation; definition of homogenization is contested]
- Fast greedy MAP DPP (Chen et al., NeurIPS 2018): exact greedy in $O(M^3)$ total with sliding-window variant, better relevance-diversity tradeoff and online A/B at Hulu — [arXiv 1709.05135](#)

- Calibrated recommendations (Steck, RecSys 2018): post-hoc greedy rerank minimizing KL divergence between user's historical category distribution and list's; later work shows LP/exact optimization beats greedy; extended beyond single-category in 2026 work (secondary) — [Calibration survey-ish source](#), [ExCalibR](#)
- Kaminskas & Bridge (ACM TiiS 2016) survey diversity, serendipity, novelty, coverage; re-ranking trade-offs measured [survey] — [UCC](#); serendipity is hard to define/measure — [Kotkov et al.](#)
- Deezer metrics: repetition, distinct/total tracks, popularity; researchers distrust genre labels for diversity — [CNRS news](#)

Inferences

- Practical stack for one user: retrieval by embedding similarity -> score with context-aware ranker -> rerank with DPP (kernel from embeddings x relevance) or MMR; calibrate against user's 90-day genre/artist/mood mix with a floor on exploration slots (e.g., reserve k% for unseen artists).
- Track narrowing over time with: share of unique artists, entropy of artist/genre plays per window, median popularity percentile, fraction of plays from recommended vs organic sources (as in Anderson/Deezer), and novelty rate (first-ever plays). Chaney-style homogenization is across-user so it does not directly apply to a single user.
- Because feedback from own recs contaminates training data (algorithmic confounding), log which plays came from the recommender and downweight or use propensity weighting.

Gaps

- Did not retrieve exact formulas (ILD, coverage) or the primary Steck paper; no Spotify/Deezer 2023-26 long-term-outcome paper found beyond those above; no empirical comparison of MMR vs DPP on music.

Q5. Open-source implementations and maintenance

Takeaway

Core pieces exist under permissive licenses for code (MERT/MuQ/MusicFM code MIT or similar, CLAP CC0, RecBole MIT), but pretrained weights are mostly NC, and Essentia is AGPL.

Cited Findings

- Essentia: AGPLv3, models NC — [licensing](#)
- MuQ: `muq` package on PyPI, code MIT — [PyPI muq](#)
- LAION-CLAP: CC0 repo, 871 commits, 64 open issues, "work in progress" — [GitHub](#)
- MusicFM: MIT code, unclear maintenance — [GitHub](#)
- RecBole (LightGCN etc.): MIT per PyPI; third-party health score 38/100 (low maintenance signal, Feb 2025); issue activity visible to mid-2024 — [depscope](#), [paper](#)
- DPP greedy MAP: algorithm published (Hulu); I did not locate an official maintained repo.

Inferences

- A minimal stack: Essentia or MuQ/CLAP for embeddings, `faiss`/numpy for kNN, a small custom DPP/MMR (~30 lines), LightGBM for context ranker. None of the latter were checked in sources.

Gaps

- Did not check `implicit`, node2vec, LightFM, Merlin, or maintenance dates for MERT's repo. Last-commit dates not verified for any repo.

D.5 llm_and_systems.md

LLM-assisted recommendation and systems engineering for a single-user music recommender (as of Oct 2026)

Evidence grades: [replicated] / [single paper] / [anecdote] / [inference]. Research budget was limited (~16 tool calls). Several areas (tooling, steering designs, capture internals) are thinly sourced and flagged under Gaps.

1. LLM-assisted recommendation: steering, reranking, explanation, and evidence vs classical baselines

Takeaway

Evidence supports LLMs as a translator or reranker over a candidate set produced by a classical model, not as the primary recommender. LLM gains in benchmarks come with large inference cost, and an old, replicated lesson (tuned simple baselines often beat fancy models) applies. A design that keeps the LLM off the hot path is an inference from this evidence, not a proven result.

Cited Findings

- Zero-shot LLM rankers (Hou et al., arXiv 2305.08845): LLMs struggle to perceive the order of history and are biased by popularity and prompt position. Prompting and bootstrapping reduce this. They can be competitive with or better than conventional models when ranking candidates retrieved by multiple generators. Code at RUCAIBox/LLMRank. [single paper] — [arXiv 2305.08845](#)
- RecBench (arXiv 2503.05493) tests up to 17 LLMs on 5 datasets (incl. music). It reports LLM recommenders beat conventional ones by up to 5% AUC (CTR) and up to 170% NDCG@10 (SeqRec). The authors say inference efficiency is "significantly reduced" and impractical for real-time settings. [single paper] — [arXiv 2503.05493](#)
- Criteo meta-review: a 15-60% gap in offline ranking accuracy remains between LLM rerankers and the best classical recsys, but it excludes fine-tuned LLMs from its aggregated table. [meta-review of mixed sources; not peer reviewed as far as I could tell] — [Criteo PDF](#)
- Negative results: in MSL (arXiv 2504.04178), the LLM-enhanced baseline LLM-CF shows negative gains on 3 of 4 datasets, and S-DPO sometimes negative. The authors attribute this to a gap between LLMs and traditional models. [single paper, tangential: it is a paper proposing its own method, so baseline numbers are self-reported] — [arXiv 2504.04178 \(HTML\)](#)
- Classical-baseline lesson: Dacrema, Cremonesi, Jannach (RecSys 2019) tried to reproduce 18 neural top-n algorithms; only 7 reproduced, and 6 of those were often beaten by simple kNN or graph-based heuristics. The seventh did not consistently beat a well-tuned linear method. A follow-up extends this. This predates LLM recommenders; I found no equivalent LLM-specific replication in my searches. [replicated across two papers, pre-LLM] — [Dacrema et al. summary](#)
- Tool-calling design: TalkPlay-Tools (arXiv 2510.01698) has an LLM orchestrate retrieval (SQL boolean filters, BM25, embedding search, semantic-ID generation) in a retrieval-then-rerank pipeline. This is the closest published analogue to "LLM translates a request into filters over your own table". [single paper; I only saw the abstract-level description] — [arXiv 2510.01698](#)
- TalkPlay (arXiv 2502.13713) is an end-to-end approach: a single model tokenizes audio, lyrics, metadata, tags, and playlist co-occurrence for next-token prediction. [single paper] — [arXiv 2502.13713](#)
- WeMusic-Agent (arXiv 2512.16108) trains an LLM to decide between internal knowledge and external music tool calls; MuChator (arXiv 2605.27103) is a conversational music LLM deployed in Douyin Music, which reports a 46.49% rise in user active days in an A/B test. Both are industry systems with proprietary data and catalogs, so they do not transfer directly to a single-user setup. [single paper each; self-reported by authors] — [WeMusic-Agent](#), [MuChator](#)

- Constrained LLM reranking (arXiv 2608.23484, a conversational music rec challenge entry) used non-LLM post-retrieval rules (excluding already-heard tracks, a popularity bonus, an over-recommended penalty). The authors dropped an album-continuation signal from the official submission because it may reflect dataset artifacts. [single paper, only seen via search summary] — [arXiv 2608.23484](#)
- LP-MusicDialog (arXiv 2411.07439) generates synthetic music-discovery dialogues with an LLM, guided by intents and musical attributes, over the Million Song Dataset (288k+ conversations). It is relevant for text-to-attribute mapping data, but I did not verify its evaluation. [single paper] — [arXiv 2411.07439](#)
- RecSys 2025 talk on an LM-based playlist generator generating tracks from playlist titles (cold start): GPT-4o prompting gave strong qualitative but weaker quantitative results. [single talk summary; no arXiv link found] — [SlidesLive](#)

Inferences

- Safest steering pattern: the LLM parses text like "darker this week" into a structured, validated object (e.g., filter ranges on audio/tag features, a weight vector, an expiry date) using a constrained schema. The classical model still does the scoring. [inference; consistent with TalkPlay-Tools' SQL-filter tool, but no paper tested this in a single-user setting]
- Keep the LLM off the hot path: call it only (a) when the user types a steering request, (b) in an optional nightly or weekly batch to rerank the top ~50-100 candidates or write explanations, and cache results in DuckDB. Latency and cost are then irrelevant to playback. [inference from RecBench's efficiency finding]
- Explanations are the lowest-risk use: generate them from features that the ranker actually used, so the text cannot change the ranking. Explanations that are not grounded in model features can be fabricated; I found no music-specific evidence on faithfulness. [inference]
- Hallucination guard: any track or artist the LLM names must be matched against your catalog or the Spotify catalog before use; the reranker should only permute a given candidate list. [inference; popularity and position bias in Hou et al. motivate shuffling the candidate order and averaging]
- Privacy: sending listening history to a hosted API leaks taste data. Options are sending only aggregate or anonymized feature summaries, or using a local model. I found no source quantifying this tradeoff. [inference]

Gaps

- No search of open-source LLM playlist generators or music-LLM repos was done, so I cannot name any with verified license or activity. Do not treat any as vetted.
- No evidence found on text-to-audio-attribute mapping accuracy (e.g., "darker" to valence/mode/tags). Note also that Spotify's audio-features endpoints were not checked here and may be unavailable to new apps (see section 4); this needs verification.
- No head-to-head of LLM reranking vs a tuned classical model on single-user data was found.
- The Criteo meta-review's authorship and review status were not confirmed.

2. Real-time/session layer

Takeaway

Direct evidence on "online learning of the big model isn't worth it" was not found in my searches; the main supported point is that River itself warns batch learning is usually sufficient. A cheap session layer (exponentially weighted state plus a small online model) over a nightly-retrained model is a reasonable inference, not a verified result.

Cited Findings

- River (online-ml/river) is a BSD-3 library for learning one observation at a time (linear models, trees, drift detection, recsys utilities); it merges creme and scikit-multiflow. Its own docs say that you should ask whether you need online ML and "the answer is likely no" because batch learning is usually sufficient. [project documentation] — [PyPI river](#); search summary of docs at [docsearch](#). I did not check the current release date or commit activity (verify before depending on it).
- Spotify Sequential Skip Prediction Challenge (WSDM Cup 2019): ~130M sessions of 10-20 tracks, predict skips in the second half of a session from the first half. DIKU-IR's stacked-RNN ensemble ranked 2nd of 45 with mean average accuracy 0.641 and first-skip accuracy 0.807; another team found sequence learning significantly beat metric learning and that full user log information helped. [challenge results; single papers] — [DIKU-IR arXiv 1903.08408](#), [few-shot paper arXiv 1901.08203](#)

Inferences

- A "pivot after N skips" heuristic (e.g., after 2-3 consecutive early skips, shift the next queue picks away from the recent cluster and toward the exploit side) is plausible, but I found no paper that specifies a validated N. Treat N as a tunable parameter and evaluate it on your own skip logs. [inference]
- Implement session state as an exponentially weighted average of embeddings or features of recently liked and skipped tracks, with a half-life of minutes to hours, that re-weights candidates at queue-fill time. It needs no library. River's logistic regression is an option for a per-user "will I skip this" model, but with one user the data is small enough that periodic batch refits would likely do. [inference]
- Skip logging requires capture of play duration, so session adaptation depends on section 3's capture quality. [inference]

Gaps

- No source found directly comparing online updating vs nightly retraining of a large recommender, and none on a validated "N skips" pivot rule.
- River's current maintenance status and a concrete music-recsys example were not checked.

3. Capture robustness

Takeaway

Spotify's own "recently played" is a coarse record (max 50 items, no stated private-session behavior), so a redundant record from an OS-level or scrobbler source is worthwhile. Note multi-scrobbler does not appear to support Windows SMTC as a source.

Cited Findings

- Get Recently Played: limit max 50 (default 20), `after / before` Unix-ms cursors (not both), scope `user-read-recently-played`, no podcast episodes. The docs do not say how private sessions affect results or how long a track must play to count. [official docs] — [Spotify reference](#)
- multi-scrobbler (FoxxMD): MIT license, 30+ sources including Spotify, Plex, Jellyfin, VLC, YouTube Music, and "MPRIS (Linux Desktop)", scrobbling to Last.fm, ListenBrainz and Maloja without duplicating tracks (mechanism not documented on the fetched page). No Windows SMTC source was found in the README or search results. I could not confirm release date or recent activity from the fetched page. [project README] — [GitHub](#), [search summary](#)
- Pano Scrobbler (kawaiiDango): GPL-3.0, scrobbles to Last.fm, ListenBrainz, Libre.fm, Pleroma on Android and desktop; has an interactive notification for editing/cancelling/blocking scrobbles; can scrobble from Pixel Now Playing; 4.41 release noted. Its internals (notification listener / media session) were not confirmed from source. [project listings, secondary] — [LinuxLinks](#), [newreleases 441](#)

- Windows SMTC: `GlobalSystemMediaTransportControlsSessionManager` exposes sessions system-wide for apps integrated with SMTC; per session you get media properties (title, artist, album, track number, etc.), timeline, playback info and `SourceAppUserModelId`. Python wrapper `py-now-playing` builds on `winsdk` and has a media-properties-changed callback. `winsdk` method names are snake_case of the WinRT names (e.g. `try_get_media_properties_async`) — my inference, not confirmed from docs. [official/docs; secondary for Python] — [Microsoft Learn](#), [Raymond Chen blog](#), [py-now-playing](#)
- ListenBrainz: POST `/1/submit-listens` with user token; guidance is to submit a listen only after half the track or 4 minutes, whichever is lower; `/1/latest-import` supports incremental imports; adaptive rate limiting (numbers not retrieved); `liblistenbrainz` client warns multiple client instances can trigger 403s. [official docs] — [ListenBrainz API docs](#), [liblistenbrainz](#)

Inferences

- Run two independent capture paths into a single DuckDB `plays` table keyed by (track, start-time bucket): Spotify polling for ground truth of identity, and either an OS-level source (SMTC on Windows, MPRIS on Linux) or a scrobbler server (ListenBrainz/Last.fm) as a redundant record. Dedupe on a time window. [inference]
- Because the Spotify docs are silent on private session and offline playback, verify empirically by testing private session and offline plays and seeing whether they appear in recently-played. [inference/gap]
- The user's environment is macOS (Darwin); SMTC applies only if the player runs on Windows. macOS equivalents (MediaRemote) were not researched.

Gaps

- Private Session, offline, and multi-device handoff behavior: no source found; needs an experiment.
- Spotify currently-playing polling interval guidance and rate limit costs: not researched beyond the 30-second rolling window in section 4.
- Not verified: Pano Scrobbler's capture mechanism, multi-scrobbler's dedupe mechanism and release cadence, ListenBrainz's rate-limit numbers.

4. Spotify control surface (Oct 2026)

Takeaway

Spotify tightened Development Mode in 2026: Premium required for the app owner, small user/client limits, renamed playlist endpoints (`/items`), reduced search limits, and removed fields. Queue and playback endpoints were unchanged. The Add-to-Queue endpoint is append-only as documented.

Cited Findings

- February 2026 changelog (blog slug suggests 6 Feb 2026): POST `/users/{user_id}/playlists` removed, use POST `/me/playlists`. Playlist `/tracks` endpoints (add, get, remove, update) replaced by `/playlists/{id}/items`; fields renamed tracks → items, track → item; only the user's own playlists return an `items` object. Library save/follow endpoints consolidated to PUT/DELETE `/me/library` and GET `/me/library/contains`. Removed: Get Artist's Top Tracks, Get New Releases, Get Several (albums/artists/tracks etc.), Browse Categories, Get User's Playlists (`/users/{id}/playlists`), Get User's Profile. Search `limit` max cut from 50 to 10 and default from 20 to 5. Removed fields include track `popularity` and `available_markets`, artist `followers/popularity`, album `popularity/label`. No playback or queue endpoints removed or renamed. [official] — [Feb 2026 changelog](#)
- March 2026: `external_ids` on album and track, marked removed in February, was reverted and stays available. [official] — [March 2026 changelog](#)

- May 2026: `account_id` (public, immutable, pseudoanonymous) added to Get Current User's Profile. [official] — [May 2026 changelog](#)
- July 2026: Client ID cap raised to 25 per account; development-mode quotas now counted per developer account (shared across Client IDs); 429 body now includes reason `QUOTA_EXCEEDED`. [official] — [July 2026 changelog](#)
- Dev Mode migration: app owner needs active Premium (app stops working if it lapses); new apps limited to 5 users (and 1 Client ID before the July raise); new apps affected from 11 Feb 2026, existing Dev Mode apps migrated 9 Mar 2026; `PUT /playlists/{id}/items` replaces `PUT .../tracks` for replace/reorder. Extended Quota Mode apps unaffected; criteria not stated. [official] — [Migration guide](#)
- A developer reported Dev Mode apps got 403 on the old `/tracks` path and 200 on `/items` after the 9 March 2026 migration. [anecdote] — via [search summary citing GitHub issue](#)
- Rate limits: rolling 30-second window; 429 may carry `Retry-After`; Dev Mode lower than Extended Quota; no numeric values on the page. [official] — [Rate limits](#)
- Add Item to Queue: `POST /v1/me/player/queue`, Premium only, scope `user-modify-playback-state`, track/episode URIs only, optional `device_id` (else active device), returns 204. "The order of execution is not guaranteed when you use this API with other Player API endpoints." The page does not describe removal or editing of queued items. [official] — [Add to Queue](#)

Inferences

- Since queued items cannot be removed via the documented API, use a just-in-time queue: keep only 1-2 tracks ahead in the Spotify queue, poll the playback state, and add the next pick shortly before the current track ends, so a changed state (skips, steering) can still influence upcoming picks. Because ordering across Player calls is not guaranteed, avoid issuing pause/skip and queue-add calls simultaneously. [inference]
- For a pre-built list, create/replace a dedicated playlist (`POST /me/playlists`, then `PUT /playlists/{id}/items`) and start playback with it as context; the playlist can be rewritten, unlike the queue. [inference from endpoints]
- Features dropped (popularity, artist followers) must come from other sources; audio-features/recommendations endpoints were not checked in this session and should be verified against the changelog before being relied on. [gap]

Gaps

- Did not retrieve the root changelog page (404 on the first URL tried), nor an April or June 2026 page; changes in those months, if any, are not covered.
- A July 2026 "refresh-token lifetime limit" was mentioned in a secondary search summary but the official July changelog did not list it; unverified.
- Get User's Queue behavior, playback prerequisites beyond Premium + active device, and the Quota-modes numeric limits were not retrieved.

5. Single-machine tooling

Takeaway

No sources were retrieved for this section. The recommendations below are inferences and should be treated as unverified opinion.

Cited Findings

- None retrieved (DuckDB, MLflow, DVC, Prefect, Dagster, observability not searched due to budget).

Inferences

- Keep everything in one DuckDB file (plays, features, candidate scores, LLM cache, steering log) with SQL/Python scripts; for one user, skip feature stores and orchestrators with a server. [inference]
- Scheduling: OS scheduler (cron/launchd/Task Scheduler) for the nightly job; a long-lived poller run under a supervisor (launchd/systemd/NSSM) with a heartbeat row written each poll, and an alert if the heartbeat is stale. [inference]
- Experiment tracking: a plain table of runs (config hash, metrics, date) is probably enough; MLflow or DVC are optional rather than needed. [inference]
- Skip until needed: feature stores (Feast etc.), Dagster/Prefect services, online-learning the main model, vector DBs beyond DuckDB's own capabilities. [inference]

Gaps

- All claims here are unsourced; library activity, licenses, and DuckDB-specific patterns need a follow-up pass.

D.6 legal_terms.md

Legal/policy terms for a private, non-commercial personal music recommender (as of Oct 2026)

Not legal advice. Fetched via a summarizing tool (WebFetch), so quotes and clause numbers should be verified against the live pages. Discogs ToU and the Apple terms could not be fetched (403 / not found).

Spotify Developer Terms and Policy: ML/AI, storage, derived data, own history, personal use

Takeaway

Spotify's Developer Terms (Version 10, effective 15 May 2025) and Policy prohibit using the Web API or Spotify Content to train or feed ML/AI models, and restrict indefinite storage and aggregated databases of Spotify Content. The "private personal use" framing is ambiguous for hobby projects, and the terms are silent on a user's own GDPR/privacy-page data export.

Cited Findings

- Terms are "Version 10, effective as of 15 May, 2025"; the Policy is also "Effective as of 15 May, 2025." No later revision of either was found. — [Developer Terms](#), [Developer Policy](#)
- ML/AI: Terms IV.2.a (and Policy III.14) bar using Spotify Content "to train a machine learning or AI model", and "ingesting Spotify Content into a machine learning or AI model". — [Terms](#), [Policy](#)
- Caching: "Do not locally cache any Spotify Content, except as strictly necessary" (IV.3.b); paraphrase: allowed for performance, limited to metadata, cover art and Premium offline downloads. — [Terms](#)
- Storage: "Do not store Spotify Content indefinitely" and no storing, aggregating or building "compilations or databases" of it (IV.3.a). — [Terms](#)
- Derived data: IV.2.d.5 covers "aggregate, anonymous or derivative data" and bars transferring it to third parties (ad networks, data brokers). This is about third-party transfer, not private local analysis. — [Terms](#)
- Data from users: only request data needed (V.3); "explicit consent" from the user who provided data (V.4); no selling Spotify Content or data obtained from Spotify (V.5). — [Terms](#)
- Personal data kept only "for as long as is necessary" and deleted when a user disconnects (Policy I.2). — [Policy](#)

- Personal/commercial: license is for "private personal use" (III.1.a). The word "non-commercial" is not used; Policy IV.2 says "commercial uses are not permitted for SDAs," with exceptions. — [Terms](#), [Policy](#)
- Spotify describes Development Mode as for learning, experimentation and personal non-commercial projects, not a business foundation. — [Feb 2026 blog](#)
- Attribution (Policy II.4): content must be attributed to Spotify; metadata, cover art and preview clips need a link back to Spotify. — [Policy](#)
- Playlist metadata may only be moved to another service at the user's direction (Policy III.9). — [Policy](#)
- Streaming of music limited to Premium users (Policy IV.1); no voice-enabled apps that control Spotify (III.3). Playback control and writing playlists are not otherwise addressed in the text retrieved. — [Policy](#)
- Extended Streaming History: downloadable from Account Privacy page; covers the lifetime of the account (vs one year for standard); can take up to 30 days. Third-party tools rely on users uploading it. — [Spotify GDPR Art. 15 page](#), [Spotify support](#), [example import guide](#)

Inferences

- The AI/ML clause targets Spotify Content obtained through the platform. The user's own privacy-export file was obtained under data-subject rights, not the developer platform, so it plausibly is outside the clause. Spotify's texts retrieved do not say so either way; ambiguous.
- Training a model on API-fetched metadata or audio features is the clearest prohibited design. Using the API only for live lookup, playback and playlist writing, with recommendations from open data, is the safest.
- Joining export history to API-fetched track metadata and persisting it is a gray zone under "no databases of Spotify Content" and the ML clause. Keep API metadata minimal and short-lived.

Gaps

- No Spotify text found that directly addresses the Extended Streaming History export in a developer context.
- Spotify Terms did not mention "Development Mode"; a separate quota document governs it. Not read directly.
- "Private personal use" vs hobby app interplay is not defined; no official guidance found.

Development Mode rules (Premium, 5 users, 2026 changes)

Takeaway

Since Feb/March 2026 Development Mode needs a Premium owner account, allows 5 authorized users, and removed many endpoints. A 23 July 2026 update raised Client IDs to 25 but with one shared per-account quota.

Cited Findings

- 6 Feb 2026 announcement: new Client IDs from 11 Feb require Premium, one Client ID per developer, max five authorized users, smaller endpoint set. From 9 March these apply to existing integrations except endpoint restrictions, which were postponed. — [Spotify blog](#)
- Previously up to 25 test users. — [TechCrunch via search result](#)
- Feb 2026 changelog removes (among others): Get Several Tracks/Albums/Artists, Artist's Top Tracks, New Releases, Get User's Profile, `POST /me/playlists`, library save/check, follow endpoints, and renames playlist `/tracks` to `/items`. Search `limit` max dropped from 50 to 10 (default 20 to 5). — [Feb 2026 changes](#)
- 23 July 2026: up to 25 Client IDs per developer account; quota counted per developer account (shared); 429 now includes `"reason": "QUOTA_EXCEEDED"`. — [Quota blog](#)
- Earlier, Nov 2024: Spotify cut access to recommendations, related artists, audio features/analysis and similar endpoints for new/dev-mode apps. — [TechCrunch](#)

- Extended quota reportedly needs ~250k MAU; this is user-reported only. — [Spotify community thread](#)

Inferences

- A single-user personal app fits the 5-user cap. Creating playlists via the removed `POST /me/playlists` may require its replacement endpoint; check the changelog.
- Cannot rely on Spotify audio-features or recommendations endpoints for a new app.

Gaps

- Numeric quota values are unpublished. Whether the postponed endpoint restrictions for existing apps have since taken effect was not found.

Open data sources

Takeaway

Last.fm, Deezer and MusicBrainz-supplementary data are explicitly non-commercial-friendly (a private project fits). MusicBrainz core data and ListenBrainz listens are CC0-style open. Several research datasets are restricted or withdrawn.

Cited Findings

- Last.fm API ToS: data "solely for non-commercial purposes" (cl. 3.1); 100 MB "Reasonable Usage Cap" on stored data (4.3.4); follow HTTP caching headers; attribution/link-back (2.7, 4.2.2); rate limits at Last.fm's discretion (4.4); delete data on termination (9.3); audio/images/artwork excluded (5.1.8). — [Last.fm API ToS](#)
- MusicBrainz data: core data CC0 (commercial OK); supplementary data, live feed and docs CC BY-NC-SA 3.0 (non-commercial, credit, share-alike). — [MusicBrainz data license](#)
- MetaBrainz: "Personal use of our datasets will always be free"; commercial users asked to become supporters. — [MetaBrainz datasets](#)
- MusicBrainz API rate limit: ~1 request/second per source IP; descriptive User-Agent with contact required, else throttled. — [Rate limiting](#)
- MetaBrainz API page says endpoints require an access token from the profile page (may apply to some APIs; not clearly MusicBrainz web service). — [MetaBrainz API](#)
- ListenBrainz: dumps (full twice monthly) of user listens; CC0 for listen data per a secondary source (AlternativeTo) only; verify officially. — [ListenBrainz dumps docs](#), [AlternativeTo](#)
- Discogs: API ToU splits data into CC0 Data and Restricted Data; monthly dumps are CC0. Official ToU page returned 403 so the details (images, rate limits) are unverified. — [Discogs ToU](#)
- LFM-2b: host page says "not available for download anymore due to license issues". — [JKU LFM-2b](#)
- Yambda: Apache 2.0 license, but "published exclusively for scientific and research purposes". — [Hugging Face](#)
- Spotify Million Playlist Dataset: no longer hosted on Alcrowd due to Spotify licensing restrictions; request access from Spotify Research. — [Alcrowd challenge](#), [Alcrowd forum](#)
- Music4All A+A: CC BY-NC-SA 4.0. Original Music4All / Onion license not confirmed. — [arXiv 2509.14891](#)

Inferences

- Yambda's "scientific and research" wording makes a personal hobby project ambiguous; it is not clearly commercial but not clearly research.
- Last.fm's 100 MB cap means no bulk mirroring; per-user scrobbles fit easily.

Gaps

- Official ListenBrainz data license text, Discogs rate limit and image terms, Music4All-Onion license were not verified. Spotify MPD license terms not seen. LFM-2b license text not seen.

Audio preview sources and ML feature extraction

Takeaway

No source found that permits ML feature extraction on previews; Apple's text restricts previews to promotion and Deezer's terms are silent on ML but non-commercial-only.

Cited Findings

- Deezer API terms: use "strictly limited for a non-commercial purpose and in a non-commercial environment"; non-Premium content only up to 30 seconds; no reverse engineering; no AI/ML/data-mining/caching mention. — [Deezer terms](#)
- Apple Search API docs: promotional content incl. song previews only "to promote store content and not for entertainment purposes"; sound samples must be near a store badge. — [Apple Search API docs](#)
- Spotify previews: Policy requires link back with preview clips. Note Spotify removed `preview_url` for new apps in Nov 2024 (not re-verified here). — [Policy](#)

Inferences

- Deezer's silence on ML is ambiguous, not permission. Apple's "promotion only" wording makes analysis a poor fit.

Gaps

- Apple's full media terms and Deezer's main ToU (not just developer terms) not read.

Enforcement cases

Takeaway

Documented enforcement is mostly API access revocation, often with unclear reasons; I found no cases of legal action against hobby projects.

Cited Findings

- 2020: Spotify reportedly threatened developers (SongShift) over playlist-transfer to competing services, with API loss. — [AppleInsider](#)
- Community reports of apps disabled/revoked with no appeal path, one citing third-party service integration. — [Spotify community](#), [another](#)
- Nov 2024 endpoint cuts, which some developers believed aimed at AI scraping. — [TechCrunch](#)

Inferences

- Practical risk is key revocation, not litigation; combining Spotify data with other services is a reported trigger.

Gaps

- No systematic list of enforcement; anecdotal forum evidence only.
-

PART E — Research prompt

Source: `RESEARCH_PROMPT.md`.

Research prompt: elevating "Personal Radio"

Paste everything below the line into a deep-research tool or a fresh Claude conversation. Attach `HANDOFF.md` for full project context.

Role and objective

You are a research analyst with expertise in recommender systems, music information retrieval, and applied ML engineering. Produce a **decision-oriented research report** that tells me what open-source projects, datasets, models, and methods I should adopt, adapt, or avoid to build a **single-user personal music recommender** that beats Spotify's own recommendations for me. Today is October 2026. Prioritize sources from 2023–2026, but include older foundational work where it still holds.

Project context (condensed)

- **Single user, personal use.** Not a product, not multi-tenant. Small personal data (thousands to low hundreds of thousands of plays), but I can borrow public datasets for pretraining.
- **Capture:** server-side polling of Spotify playback (Windows desktop + iPhone), plus Extended Streaming History import. Events: play, skip, complete, repeat, seek, save, follow, top items, and a `start_kind` tag (user-started vs autoplay vs skip-navigation) with context type (artist/album/playlist/lone track). Spotify exposes **no search history**, so hand-started plays are my proxy for intent.
- **Labels:** graded implicit feedback (skip <10 s strongly negative, completion weakly positive, completion + save/repeat strongly positive).
- **Hard constraints:**
 - Spotify removed `/recommendations`, `/audio-features`, `/audio-analysis` for new apps; `preview_url` is null; dev mode is limited (Premium owner, 5 users, small search limits).
 - Spotify's Developer Policy reportedly forbids training ML models on Spotify content. Assume Spotify is only the **player and catalog**; candidates and features must come from **open data** (ListenBrainz, Last.fm, MusicBrainz, Discogs, public datasets) and my own behavior logs.
 - No scraping or ripping audio. Any audio-based method must name a legitimate audio source (e.g., my own purchased files, licensed previews from another service whose terms allow it).
- **Planned architecture:** candidate generation (open data) → ranker (LightGBM first) → exploration (contextual bandit) → nightly playlist writer, plus an optional session-aware layer and one-track-lookahead queue control. Python, DuckDB, FastAPI dashboard.
- **Success definition:** beat Discover Weekly on hit rate (saved or completed) in a blind A/B, recover from a bad mood match within ~3 tracks, support natural-language steering, explain every pick, and not narrow my taste over time.

Research questions

Answer each with concrete recommendations, not a survey. For every item name the repo/paper, its license, maintenance status (last commit, stars, activity), and what specifically I would reuse.

A. Prior art: has this been done?

1. Open-source personal or self-hosted music recommenders, playlist generators, and "Spotify replacement" projects (including agentic/LLM playlist builders and those built after the 2024 API removals). What worked, what broke, how did they survive API changes?
2. Scrobble-driven recommenders (ListenBrainz's own recommendation stack, Last.fm-based tools, Maloja/Koito/Multi-scrobber-style ecosystems, Navidrome/Plex/Jellyfin recommendation plugins).
3. Academic or industry write-ups of **single-user / small-data** personalization (not just web-scale CF): what techniques are known to work with only one person's history?

B. Candidate generation from open data

4. Best available open sources for "similar tracks/artists" and their coverage, freshness, rate limits, and terms: ListenBrainz (recommendations, similar-users, LB Radio), Last.fm, MusicBrainz, Discogs, AcousticBrainz (archived; what replaced it?), Wikidata, Bandcamp/other tag sources.
5. Public listening datasets usable for **pretraining** collaborative or sequential models: LFM-1b/LFM-2b, Yambda, Spotify Million Playlist Dataset, Spotify Sequential Skip Prediction, Music4All(-Onion), MSD/Taste Profile, ListenBrainz data dumps. Compare size, license, track-ID mappability to MusicBrainz/ISRC, and whether pretraining then fine-tuning on one user is demonstrated to help.
6. Track identity resolution: robust methods and tools for mapping Spotify URI $\leftrightarrow$ ISRC $\leftrightarrow$ MusicBrainz recording MBID (MusicBrainz ISRC lookups, ListenBrainz MBID mapper, fuzzy matching). Failure rates and how to handle remixes/live/regional variants.

C. Modeling methods

7. **Sequence-aware / session-based recommendation:** SASRec, BERT4Rec, GRU4Rec, and current successors (e.g., HSTU/generative recommenders, Mamba-style, LLM-based sequential recommenders). Which are practical on one user's data with transfer from a public dataset? Reproducible open implementations (RecBole, Recommenders/Microsoft, LensKit, Merlin, implicit, TorchRec, RecPack).
8. **Skip prediction and implicit-feedback modeling:** lessons from the Spotify Sequential Skip Prediction challenge winners; how to treat skips as noisy negatives; position/exposure bias correction (inverse propensity weighting, counterfactual learning to rank, dueling/doubly-robust estimators).
9. **Exploration:** contextual bandits (LinUCB, Thompson sampling, neural bandits), slate/playlist bandits, Spotify's published bandit and "explore vs. exploit" work, and practical exploration budgets for a single user. Libraries (Vowpal Wabbit, MABWiser, Open Bandit Pipeline, RecoGym).
10. **Imitation learning / inverse RL / behavior cloning for recommendation:** any credible evidence that these beat supervised ranking on implicit feedback; offline RL for playlists/sessions; what "imitation" should actually mean here given exposure bias.
11. **Representation learning for music:** audio embeddings (MERT, CLAP, MuQ, MusicFM, MusiCNN/Essentia models, OpenL3, LAION-CLAP), contrastive audio-text models, and metadata/tag/graph embeddings (node2vec, LightGCN on listening graphs). Which work **without** audio (tags, lyrics themes, artist graphs), and which need audio I may not legally have? Offer a no-audio fallback path.
12. **Multi-interest / taste-cluster modeling:** MIND/ComiRec-style multi-interest extraction, user clustering, long-term vs short-term preference fusion; how to weight recent behavior without forgetting long-term taste.
13. **Mood/context modeling:** time-of-day, day-of-week, device, activity inference; context-aware recommenders (e.g., factorization machines, DeepFM, contextual ranking); mood/emotion tags in MIR (valence/arousal, AudioSet-style tags).
14. **LLM-assisted recommendation:** natural-language steering ("darker this week"), LLMs as rerankers or explainers, text-to-music-attribute mapping, and cost/latency/privacy trade-offs. Which designs keep the

LLM out of the hot path? Evidence on LLM recommenders vs classical baselines.

15. **Diversity, novelty, serendipity, and anti-narrowing:** metrics (intra-list diversity, coverage, novelty, serendipity, long-term filter-bubble measures), re-ranking methods (MMR, DPP), and feedback-loop mitigation.

D. Systems, evaluation, and engineering

16. **Offline evaluation for a single user:** time-split protocols, leave-last-out pitfalls, off-policy evaluation with logged bandit feedback, minimum data requirements, and how to get statistically meaningful signal from one person (N is small). Practical design of the **blind A/B vs Discover Weekly**.
17. **Feature store / pipeline tooling** appropriate for a single-machine setup: DuckDB-centric patterns, lightweight experiment tracking (MLflow, DVC, Weights & Biases alternatives), scheduling (cron, Prefect, Dagster), and what to skip as overkill.
18. **Real-time/session layer:** designs for lightweight online adaptation (exponentially weighted state, online logistic regression, River library) vs. nightly retraining; evidence that online learning of the big model is not worth it.
19. **Capture robustness:** how other projects reliably track playback (polling vs. MPRIS/SMTC/MediaSession vs. scrobbles like Pano Scrobbler), handling Private Session, offline, and multi-device handoff; ListenBrainz/Last.fm as redundant record.
20. **Spotify control surface:** current state of queue/playlist write APIs, rate limits, and workarounds; patterns for "just-in-time queue" without being able to edit queued items.

E. Legal, policy, and risk

21. Summarize the **actual current text** of Spotify's Developer Terms/Policy on ML training, data storage, and caching, and the terms of each open-data source I'd use (Last.fm, ListenBrainz, MusicBrainz, Discogs, any dataset). Flag what is permitted for a **private, non-commercial** tool, quoting the clause (≤ 15 words) and linking it. Where ambiguous, say so rather than guessing.
22. Lessons from projects that were shut down or broken by API/policy changes; design patterns for staying resilient.

What I want back

1. **Executive summary (≤ 400 words):** the 5–8 highest-leverage changes to my plan, ranked by (expected quality gain) $\div$ (effort), each with a one-line justification.
2. **Comparison tables** for: open-data sources; public datasets; sequence/ranking libraries; bandit libraries; audio/metadata embedding options. Columns: license, maintenance, data requirement, fits single-user?, effort, key risk.
3. **Recommended stack and build order** revising my architecture: what to add, replace, or drop; a minimal viable version I can ship in two weekends and a stretch version.
4. **Evidence quality grading:** for each major claim mark **replicated / single paper / blog or anecdote / my inference**. Call out hype (e.g., claims that LLM or imitation-learning recommenders beat tuned classical baselines) and say when evidence is weak.
5. **Contradictions and open questions:** where sources disagree, and what experiment on my own data would settle it.
6. **A 10-item reading list** (papers, repos, talks) in the order I should read them.
7. **Risks I haven't considered.**

Rules

- Prefer primary sources (papers, repos, official docs) over blog summaries; cite every non-obvious claim with a link. Do not invent repositories, papers, or statistics; if you cannot verify something, say so.
 - Verify that each recommended repo is **still maintained** (recent commits) and its **license permits personal use**.
 - Do not recommend scraping Spotify, ripping audio, or anything that violates a service's terms; propose a compliant alternative instead.
 - Be skeptical: single-user data is tiny, so penalize methods that need large per-user histories unless transfer from a public dataset is demonstrated.
 - Keep recommendations concrete and implementable in Python on one machine.
-

PART F — Package README

Source: `personal_radio/README.md`. Note: setup now also needs `export SPOTIPY_*` or the interactive prompt, and tracker entry point is `python -m radio.run`.

Personal Radio (Phase 1)

Capture your Spotify listening (Windows PC + iPhone both land in your account state, so one server-side poller sees both), turn it into plays, and grade them into labels.

Setup

1. Spotify dev app (owner needs Premium). Redirect URI `http://127.0.0.1:8888/callback`.
2. `python3 -m venv .venv && .venv/bin/pip install -r requirements.txt`
3. `export SPOTIPY_CLIENT_ID=... SPOTIPY_CLIENT_SECRET=... SPOTIPY_REDIRECT_URI=http://127.0.0.1:8888/callback`
4. `.venv/bin/python -m radio.poller` (first run prints an auth URL; keep it running on an always-on host)
5. Optional: request Extended Streaming History on Spotify's privacy page, then `.venv/bin/python -m radio.history_import path/to/dir_or_zip`

Layout

- `radio/tracker.py` – pure state machine: playback snapshots -> finished plays (skip/complete/repeat). Tested.
- `radio/poller.py` – Spotify polling loop, adaptive delay, 429 backoff, writes `raw_events` + `plays`.
- `radio/history_import.py` – Extended Streaming History -> `plays`.
- `radio/labels.py` – graded labels (strong/weak pos/neg), exclusions, skip streaks.
- `radio/db.py` – DuckDB schema.

Tests (stdlib only): `python3 -m unittest discover -s tests -t .`

Notes

- Private Session plays are invisible to the poller; incognito rows in history are excluded from labels.
- Store only minimal Spotify metadata; Spotify policy reportedly forbids ML training on Spotify content (see handoff).

Always-on tracking on your Windows desktop

1. `python -m venv .venv && .venv\Scripts\pip install -r requirements.txt`
2. Set `SPOTIPY_CLIENT_ID`, `SPOTIPY_CLIENT_SECRET`, `SPOTIPY_REDIRECT_URI` (user environment variables).
3. Run `.venv\Scripts\python -m radio.run` once; a browser opens for Spotify login (token cached in `.spotify_cache`).
4. `.\install_windows_task.ps1` registers a hidden logon task (30s delay, auto-restart). Logs: `radio.log`.

On every start it backfills missed plays from recently-played, then syncs saves, follows and top items (and again every 6h). Each sync step fails independently, so a removed endpoint won't stop tracking.

Search / intent signals

Spotify exposes no search history. `plays.start_kind` (user/auto/nav) and `context_type` tag how each play began; the `intent_plays` view = hand-started plays from artist/album pages or as lone tracks. An optional Windows UI-Automation watcher for the search box is possible but fragile; not built.

PART G — Research prompt 2 (architecture improvement + queue/genre probing)

Source: `RESEARCH_PROMPT_2.md`, written 2026-10-08. Paste into a deep-research tool with this file attached.

Research prompt 2: improving the built "Personal Radio" architecture, and the value of queue injection + genre probing

Paste everything below the line into a deep-research tool or a fresh Claude conversation. Attach `CONSOLIDATED_HANDOFF.md` for full context. Written 2026-10-08 after the first working end-to-end version.

Role and objective

You are a research analyst with expertise in recommender systems, preference elicitation, bandits, music information retrieval, and applied ML engineering for single-user systems. I have **built and run** a first version of a personal music recommender (details below). Produce a **decision-oriented report** that (1) critiques this specific architecture and tells me which components to improve, replace, add or delete, ranked by expected gain per unit effort, and (2) answers a specific question: **how much would actively injecting songs into my Spotify queue, and measuring my reaction to particular genres, improve the quality of the playlist?** Today is October 2026. Prioritize 2023–2026 sources, include older foundational work where it still holds. Be skeptical: one user's data is tiny.

What exists today (implemented and tested; be specific in your critique)

Capture. A Python poller reads Spotify `current_playback` every 4 s (8 s paused, 10 s idle, 1.5 s near track end) from my account, so Windows desktop and iPhone listening are both seen. A pure state machine converts snapshots into plays with `end_reason` (completed/skipped/other), repeat detection, seek counts, and `start_kind` (user-started / auto-continued / navigation-after-skip) plus `context_type` (artist/album/playlist/collection/lone track). A track within 10 s of its end counts as completed (the last poll before a change lands 4–6 s short). Heartbeat file, 429 handling, `QUOTA_EXCEEDED` logging. Runs on a Mac laptop that sleeps (planned: always-on Windows PC). Extended Streaming History importer exists but I have not received my export yet.

Data volume today. Roughly 20 tracked plays, 50 recently-played backfill rows, 102 liked songs (real `added_at` dates), 4 follows, 278 top-item rows, a 67-track "Chill" playlist. **16 labelled plays from one day.** This is the binding constraint.

Labels. Graded implicit feedback: skip <10 s = -1.0, 10–30 s = -0.6, ≥30 s skip = -0.3, completed = +0.4, completed + (saved or replayed within 24 h) = +1.0. Sample weights: user-started 1.0, nav 0.8, auto 0.6, ×1.5 for "intent" plays (hand-started from an artist/album page or as a lone track; Spotify exposes no search history). Private Session plays are invisible.

Identity. Cache mapping Spotify URI ↔ ISRC ↔ MusicBrainz recording/artist MBID (own DuckDB file). ISRCs come from Spotify single-track lookups (batch `GET /tracks` returns 403 in Development Mode); MBIDs from MusicBrainz ISRC search at ≤1 req/s. Measured match rate so far ~81% (123/152 checked).

Taste basis. Liked songs + selected playlists (Chill), deduplicated, kept local (nothing is written to my Spotify library). Per-artist weight = $\sqrt{\text{sum of track weights decayed with a 365-day half-life}}$; top tracks $\times 0.25$ and recent labelled plays $\times 0.5$ as minor boosts. ~40 seed artists.

Candidate generation. For each seed artist, ListenBrainz Labs `similar-artists` (algorithm `session_based_days_9000_session_300_contribution_5_threshold_15_limit_50_skip_30`, no token). Per candidate artist, score = $\sum$ over seeds of `seed_weight` $\times$ (similarity / that seed's max similarity). Artists I already know (played/saved/top/in basis) are removed. Then up to 3 tracks per candidate artist from a live **Spotify artist search** (limit 10, cached 1 day), with track score = artist score $\times$ [1.0, 0.7, 0.5]. Never repeats a past recommendation.

Selection / exploration. 30 tracks: 70% "confident" (top score, propensity 1), 20% "explore" (softmax-sampled from the next band, logged inclusion probability), 10% "wildcard" (uniform from the tail, logged probability); max 2 per artist; final order shuffled so position is independent of score. Every pick is logged with slot, score, propensity, reason, position, playlist id.

Delivery. Private Spotify playlist "Personal Radio - Weekly Auto", rewritten with `PUT /playlists/{id}/items`, then read back (a prior write was accepted but the playlist later vanished).

Evaluation kit (built, no data to run on). Baselines: most-popular, recency-decayed replay, item-kNN on session co-occurrence. Rolling-origin splits with a gap, NDCG@10 / Recall@10, paired block bootstrap over days, `--novel` mode (score only never-trained-on tracks).

Not built yet. A learned ranker (LightGBM planned), Thompson sampling over clusters, DPP/MMR + calibration, session layer / queue control, LLM steering/explanations, dashboard, ListenBrainz play mirroring, popularity features, tag/genre features, blind A/B vs Discover Weekly.

Verified constraints (Oct 2026; re-verify anything that matters)

- Spotify removed `/recommendations`, `/audio-features`, `/audio-analysis`, related artists, artist top-tracks, batch `GET /tracks`, and `popularity/followers` fields for my Development Mode app. `external_ids.isrc` remains. Search limit max 10. `isrc`: search works (17/20 exact hits). Playlist writes work via `POST /me/playlists` and `/playlists/{id}/items`; responses carry the track under `item`.
- Development Mode: Premium owner, 5 users, shared per-developer quota, no published rate numbers; I saw one 429 in about 10 minutes of 4-second polling.
- Spotify Developer Terms (v10) reportedly bar using Spotify Content to train/ingest into ML models and limit caching to what is "strictly necessary"; no "compilations or databases". I store minimal Spotify data and treat my own behavior labels as the training signal (gray area).
- ListenBrainz `popularity/*` endpoints now return **401 and need a token** (I have none yet); `similar-artists` works without one. Last.fm needs a key (I have none). MusicBrainz ≤ 1 req/s; frequent 503s.
- Queue: `POST /v1/me/player/queue` is Premium-only, append-only as documented, needs an active device, and "order of execution is not guaranteed" with other Player calls. Queued items cannot be removed or reordered via the API.

Known weaknesses I already see (confirm, rank, and fix)

1. Recommendations are **mainstream hub artists** (Ariana Grande, Bieber, Post Malone, The Weeknd) because there is no popularity/hubness correction.
2. Track choice per artist is "whatever Spotify search returns first": popular songs, remixes, features, no quality or fit signal.
3. Candidate generation is **artist-level only**; no recording-level similarity, tags, genres, moods, or lyrics themes.

4. Everything excluded if I know the artist, so the playlist can never contain "more from artists I like".
5. Feedback only arrives if I choose to play the playlist; one list per run yields a handful of labelled plays.
6. Labels are noisy (skip $\neq$ dislike; "saved" can inflate positives for tracks in my library; label uses current save status).
7. No learned model; score is similarity to seeds. No evidence yet that this beats "replay what I recently played" or Discover Weekly.
8. Laptop sleeps, so capture has gaps. Timestamps stored as UTC (local time via `RADIO_TZ` at analysis time).

Part A: Holistic architecture review

Answer each with concrete recommendations; for every library, dataset or paper name its license, maintenance status (last commit, stars/activity) and what exactly I would reuse.

1. **Candidate generation.** Given only open data and a Spotify catalog, what is the best way to move from artist-level to **recording-level** candidates (ListenBrainz CF and recording similarity, Last.fm `track.getSimilar`, MusicBrainz relationships, tag/genre graphs, co-listening embeddings from the ListenBrainz dumps)? What is the evidence on **hubness / popularity bias** in neighbor-based music recommendation and the cheapest reliable corrections (inverse-popularity weighting, CSLS-style rescaling, rank-based normalization, mutual proximity)? What popularity proxies are available now without Spotify (ListenBrainz with a token, Last.fm listeners, MusicBrainz rating counts, Wikipedia pageviews)?
2. **Track selection within an artist.** Principled alternatives to "first Spotify search hit": use of ListenBrainz/Last.fm top recordings, release-type filtering (original vs remix/live/feature), fit to my taste cluster (tags, era, tempo/energy where legitimately obtainable), and diversity across an artist's catalog.
3. **Taste model.** Is " $\sqrt{\text{sum of decayed track weights}}$ per artist" a sensible seed weighting? Compare multi-interest / cluster-based user models for a user with ~ 170 tracks (taste clusters from tag or co-listening embeddings, max-similarity to centroids weighted by recency), long- vs short-term fusion, and how to weight liked songs vs a curated mood playlist (Chill) vs actual play behavior. Evidence for whether such models help at $N=1$.
4. **Ranking.** At what data size does a learned ranker (LightGBM on cluster similarity, familiarity, tag overlap, popularity, context, `start_kind`) plausibly beat decayed replay + item-kNN + the current heuristic? What features transfer across items for a single user? How to avoid leakage (a save happening after the play) and exposure bias in training labels?
5. **Exploration policy.** Evaluate my fixed 70/20/10 split with logged propensities against Thompson sampling over genres/clusters/artists, UCB variants, and "best arm" identification for tiny budgets. Give recommended budgets, how to size them for $\sim 20\text{--}30$ plays/day, and estimators (SNIPS, clipped IPS, doubly robust) that are usable with a few hundred exploratory plays.
6. **Diversity and anti-narrowing.** DPP vs MMR vs calibration for a 30-track list, metrics to monitor, and evidence on feedback loops when a system supplies a large share of one user's listening.
7. **Labels and signals.** Better use of the signals available: completion fraction, replays, saves, playlist adds, seeks, time-of-day/device, `start_kind`, intent plays. Ordinal vs binary targets, handling of "skipped because I wasn't in the mood", session-position and previous-skip effects ("skip begets skip").
8. **Evaluation at $N=1$.** Concrete protocol: rolling-origin offline checks, off-policy estimation from my logged picks, and the blind A/B against Discover Weekly (arm size, novelty adjustment, test choice, pre-registration). Minimum data needed before any offline conclusion; how to use the Extended Streaming History as a pseudo-experiment.
9. **System design and reliability.** What to simplify or cut. Capture redundancy (ListenBrainz mirror via direct `submit-listens`, OS-level sources), always-on host, quota management when ad-hoc scripts share the tracker's quota, token expiry, dev-mode fragility, and what happens if Spotify revokes the key (graceful degradation).

10. **LLM role.** Where an LLM adds value without being on the hot path (natural-language steering parsed into a validated schema, grounded explanations, tag/genre normalization), with privacy trade-offs.
11. **Legal/policy re-check.** Re-read the actual current Spotify Developer Terms/Policy text on ML, caching and databases, and tell me which of my stored tables (plays with track names/URLs, taste basis, identity cache, picks log) are most at risk and how to reduce exposure. Quote clauses (≤ 15 words) with links; say where ambiguous.

Part B: Does queue injection plus genre-reaction probing improve the playlist? (main question)

Treat this as a quantitative question, not a survey. My hypothesis is that actively adding probe tracks to my Spotify queue during normal listening, and learning my reaction by genre, would speed up learning and improve playlist hit rate; I want evidence for or against, with numbers.

The mechanism I am considering. While I listen, a controller adds one track at a time (just-in-time: 1–2 tracks ahead, near track end) from chosen genres/clusters/artists, observes completion/skip/save, updates a per-genre (or per-cluster) preference estimate (e.g., Beta-Bernoulli with Thompson sampling), and feeds those estimates into the next playlist build. Alternatives to compare: (a) only the playlist (current), (b) queue-injected probes, (c) a short explicit "taste quiz"/pairwise preference elicitation, (d) passive inference from existing history only.

Answer, with sources and flagged inferences:

1. **Value of information.** What does the literature on active learning, preference elicitation, bandits for recommendation, and cold-start interviews say about how much *actively chosen* probes improve over *passive* exploration for a user who already has a rich liked-songs history? Is genre the right granularity for probes, or are artist/cluster/tag arms better? How many probes per arm are needed to estimate a skip/complete rate with useful precision (give a worked example: base rate ~ 0.3 , ± 10 points)? How many effective independent samples can one user generate per day or week, and how long until the estimates beat simply trusting my liked-songs genre mix?
2. **Expected effect size on playlist quality.** Any evidence (A/B tests, user studies, simulations) of lift from interactive/active elicitation or in-session adaptation in music or comparable domains; how big, how durable, and for which users? Where do gains vanish (taste already well known, noisy skips, mood-driven variance)?
3. **Confounds and biases created by queue injection.** Injected tracks arrive in my listening flow: mood and context effects, order/position effects, the "interruption" cost, queue items I cannot remove, how a queued track's `context_uri` and `start_kind` will look to my tracker, and whether injected-track feedback is comparable to feedback from tracks in the playlist. How should I log propensities and randomize so these plays are usable for off-policy estimates? What is the risk of training on my own system's reactions (feedback loops, narrowing)?
4. **Genre reaction specifically.** How reliable are genre labels (MusicBrainz genres/tags, Last.fm tags, Discogs styles, Spotify artist genres if still available) as arms? Hierarchical/shrinkage models that pool related genres, handling multi-genre artists, and time/mood context ("same genre, different mood"). Should reaction be modeled per genre, per artist, or in a learned embedding space?
5. **Spotify control surface reality check.** Given the queue API limits (append-only, Premium, active device, unordered with other Player calls, rate/quota), what controller design is robust? How to avoid fighting my own manual queueing, handle multi-device handoff, Private Session and offline gaps, and stay within quota alongside the 4-second poller? Is a "next-up suggestion" notification or a second small playlist a better channel than direct queue writes?
6. **Cost-benefit ranking.** Compare the four options in (a)–(d) on: expected lift in blind hit rate on unfamiliar tracks, time to detect it, engineering effort, user burden/annoyance, policy risk, and failure modes. Give a clear recommendation and a staged plan, including the smallest experiment on my own data that would

show whether queue probing helps before building the full controller (design, sample size, success criterion).

7. **Alternatives that may beat queue injection:** e.g., richer playlist exploration with better arm design, reading existing reactions from the Extended Streaming History to pre-fit genre estimates, LLM-assisted genre/mood priors, or using ListenBrainz/Last.fm population priors to shrink estimates. Say which of these likely dominates and why.

What I want back

1. **Executive summary (≤ 400 words):** 5–8 highest-leverage changes ranked by (expected quality gain) $\div$ (effort), each with a one-line justification, and a direct yes/no/maybe on queue injection + genre probing with the key numbers behind it.
2. **Component-by-component verdict table** for the architecture above: keep / improve / replace / add / delete, with reason and effort.
3. **Comparison tables:** candidate-generation and popularity-correction options; exploration policies; elicitation strategies (playlist-only vs queue probes vs quiz vs passive); libraries (license, maintenance, data requirement, fits single user?, key risk).
4. **A staged roadmap** from today's state: what to do with 2 weeks of data, what with the Extended History, what only after a blind A/B shows a signal. Include the minimum viable experiment for the queue/genre question.
5. **Evidence grading** for each major claim: replicated / single paper / blog or anecdote / my inference. Call out hype.
6. **Contradictions and open questions**, each with an experiment on my own data that would settle it.
7. **A 10-item reading list** in the order to read it.
8. **Risks I have not considered** (policy, quota, feedback loops, over-engineering for one user).

Rules

- Prefer primary sources (papers, repos, official docs); cite every non-obvious claim with a link. Do not invent papers, repos or statistics; say when you cannot verify something.
 - Verify each recommended repo is still maintained and its license permits personal use.
 - Do not recommend scraping Spotify, ripping audio, or violating any service's terms; propose a compliant alternative instead.
 - Penalize methods that need large per-user histories unless transfer from public data is demonstrated.
 - Keep recommendations concrete and implementable in Python on one machine, on top of DuckDB and the existing modules.
 - Where you give numbers (sample sizes, lift), show the assumptions so I can check them.
-

PART H — Research round 2 report (answer to prompt 2)

Source: `reports/Fix labels and candidates before probing anything.md`. Verification notes are in section 0.1 above.

Fix labels and candidates before probing anything

Decision report, 8 October 2026. Single-user Spotify recommender, version 1. Output of `RESEARCH_PROMPT_2.md`. Saved verbatim as received; verification notes are in `CONSOLIDATED_HANDOFF.md` section 0.2.

The biggest gains for this system come from cheap fixes upstream of any model. The labels leak and are weighted the wrong way. The candidates are artist-level and biased toward hub artists, even though a tokenless recording-level co-listening endpoint already exists. Track choice within an artist takes Spotify's first search hit. Capture spends quota on 4-second polling and loses plays whenever the Mac sleeps. A learned ranker, Thompson sampling and DPP diversity are all premature at 16 labelled plays. They become worth doing only after the Extended Streaming History (ESH) import, which is the system's one real source of thousands of labels. On the quantitative question, **injecting queue probes and measuring reactions per genre is a "no" for now and a "maybe" later as a gated experiment**. Estimating one arm to ± 10 points at a 0.3 base rate takes about 81 independent probes, or 97–146 once within-session correlation is counted. Probing can affect at most 9 of the 30 weekly slots. Even perfect exploration would lift whole-playlist completion by only about 2 percentage points, which would take years of weekly playlists to detect. Two other risks deserve more attention than the user has given them. Spotify's Developer Policy also restricts *analysis* of Spotify Content, not just storage and ML training. And since July 2026, every Dev Mode app on one developer account draws from the same quota.

Throughout, my own arithmetic and reasoning are labelled "inference". Effort is graded **S** (about a day or less), **M** (2–5 days) or **L** (1–2 weeks or more). These grades are my estimates. Policy wording is mostly paraphrased with clause numbers and links, and quoted only where the exact words matter.

1. Executive summary: eight changes ranked, and a "not yet" on probing

Highest-leverage changes, ranked by expected gain per unit of effort:

1. **Fix the labels (S)**. Mark a save as positive only if it happened after the play. Make `start_kind` a feature, and stop down-weighting auto plays. Auto plays are the closest thing to an unchosen, recommended track, so the current 1.0/0.8/0.6 weights point the wrong way.
2. **Switch candidates to ListenBrainz Labs `similar-recordings` (S–M)**. It returns recording-level co-listening neighbours and needs no token. It replaces "similar artist, then first Spotify search hit."
3. **Correct for hubness and popularity (S)**. Use mutual proximity or a CSLS-style penalty, damp breadth, then multiply by $\text{pop}^{-\alpha}$ using LB popularity (the path form works tokenless). Finish with a calibrated re-rank.
4. **Choose tracks within each artist properly, and add a deep-cuts slot (S–M)**. Use LB top recordings, filter by MusicBrainz release-group type, apply MMR across albums, and cap known-artist deep cuts at 20–30% of the playlist.
5. **Fix capture (S)**. Poll adaptively and reconcile with recently-played hourly on the Windows PC. This cuts calls 5–10× and closes sleep gaps.

6. **Harden compliance (S).** Purge API-derived data on a TTL. Build features from non-Spotify sources. Train only on ESH labels. Never recreate the Client ID.
7. **Build the ESH importer and pre-fit shrunken rates (M).** This is the only route to enough labels for any learned ranker.
8. **Run a blind, team-draft-interleaved test against Discover Weekly (M, 12–20+ weeks).** It is the only decisive evidence, and DW must be copied by hand.

Defer LightGBM until there are about 2,000–5,000 labelled plays. Prefer calibration plus MMR over DPP. Use an LLM only to parse steering requests.

Queue injection plus genre probing: no for now, maybe later.

- Holding ± 10 points at $p = 0.3$ needs **81 independent probes per arm** (Wilson half-width 0.098). Within-session correlation raises that to **97–146**.
- At 5 probes/day, which is **17–25% of all listening**, five arms take about **12 weeks** before the correlation penalty.
- If ESH gives a prior worth 40 pseudo-counts, 30 probes shrink the interval only from **± 0.14 to ± 0.11** .
- Probing can influence only the **9 exploratory slots out of 30**. Even perfect allocation adds about **+2 pp** to whole-playlist completion. Detecting that would take **about 9,700 tracks per arm, or more than 6 years** (inference).

If you probe at all, use an "Up Next" playlist, not the queue, and only once ESH shows at least 5 uncertain clusters.

2. Component-by-component verdicts

Component	Verdict	Reason	Effort
Poller (4 s <code>current_playback</code> , Mac)	Replace	21,600 calls/day against an account-wide Dev Mode quota. It misses plays while the Mac sleeps. Switch to adaptive <code>currently_playing</code> on the Windows PC (Spotify migration guide ; July 2026 changelog)	S
Play state machine (end_reason, repeats, seeks, start_kind, context_type)	Keep	Matches the export's <code>reason_start</code> / <code>reason_end</code> vocabulary. Add session-state features (skips in the last 3 tracks)	S
Recently-played reconciler	Add	Hourly, 24 calls/day. Recovers about the last 50 plays from any device (recently-played reference)	S
macOS MediaRemote fallback	Delete / don't build	Blocked for non-Apple processes since macOS 15.4. The workaround uses a private framework (Keyboard Maestro forum)	—
Windows SMTC fallback	Add (optional)	Key-independent capture of title, artist and position (Microsoft Learn)	M
ESH importer	Add (top priority when it lands)	Lifetime per-play <code>ms_played</code> , reasons, <code>skipped</code> , <code>shuffle</code> (Spotify support)	M
Label scheme (−1.0 ... +1.0 graded)	Improve	The shape is fine. Fix save leakage and make labels provisional until the save window closes	S
Sample weights (user 1.0 / nav 0.8 / auto 0.6, intent $\times 1.5$)	Replace	Wrong direction for learning what to recommend. Turn them into features, with auto $\geq$ user	S

Component	Verdict	Reason	Effort
Identity cache URI↔ISRC↔MBID (81%)	Keep, improve	Needed for the LB/MB route. Store keys only and resolve names from MusicBrainz	S
Taste model (sqrt decayed per-artist, 365-day half-life)	Improve	Add a 30–90-day short-term profile. Move to 4–8 track clusters with medoids, scored by max-similarity (PinnerSage)	M
Candidates (LB similar-artists, then 3 first search hits)	Replace	Recording-level LB Labs <code>similar-recordings</code> , with similar-artists kept as a secondary route (LB Labs)	S–M
Hub/popularity correction	Add	Hub dominance is a known weakness. Mutual proximity reduces hubs in music similarity (Schnitzer et al.)	S
Within-artist track choice	Replace	LB top-recordings, filters, fit score and MMR across release groups	S–M
"Known artist" deep-cuts slot	Add	Low-risk novelty, capped at 20–30%	S
70/20/10 split with logged propensities	Keep the budget, fix the logging	Logging k-softmax is wrong. Estimate inclusion probabilities by Monte Carlo replay of the whole generator	S
Explore band (softmax)	Improve later	Hierarchical Thompson sampling over 5–8 clusters once ESH priors exist	M
Wildcard (uniform)	Keep permanently	Floor against narrowing and degeneracy (Jiang et al.). Refresh the tail weekly	—
Max 2 per artist, shuffled	Keep, extend	Add a per-track 4-week cooldown and a per-artist rolling cap	S
Delivery (private playlist PUT plus read-back)	Keep	Low policy risk. Add a <code>snapshot_id</code> check and recreate the playlist via <code>POST /me/playlists</code>	S
Eval kit (baselines, rolling origin, NDCG/Recall, block bootstrap, <code>--novel</code>)	Keep, extend	Add a 1–7-day gap, Recall@50/100 for novel items, and a paired bootstrap over weeks	S
LightGBM ranker	Defer	16 labelled plays is roughly 5–8 minority-class events. Revisit at 2,000–5,000 plays	—
Thompson sampling	Defer to stage 2	Allocation only, never a "winner" claim	M
DPP / MMR / calibration	Add calibration plus MMR. Skip DPP	Calibration targets genre-mix narrowing directly. DPP adds little at N = 30	S–M
Queue control	Do not build now	See Part B	—
LLM steering	Add later, as a parser only	Off the hot path, with schema validation	M
Blind A/B vs DW	Add (stage 2)	Team-draft interleaving, pre-registered	M
obp dependency	Avoid	Unmaintained since 2023. Vendor the roughly 20-line estimators	—

Where the notes overturn current beliefs

Belief	What the notes show
"LB popularity endpoints need a token (I got 401)"	The probes on 2026-10-08 returned 200 without a token using the path form <code>/1/popularity/top-recordings-for-artist/{artist_mbid}</code> . The query-string form <code>?artist_mbid=</code> returned 308 and then 404. <code>POST /1/popularity/recording</code> also worked tokenless (LB popularity docs). The 401 most likely came from the wrong path or a temporary gate. Get a free token anyway, because MetaBrainz has been gating endpoints against AI scrapers (MetaBrainz blog).
"LB only offers artist similarity"	LB Labs lists <code>/similar-recordings</code> and <code>/mlhd-similar-recordings</code> . A tokenless POST returned scored recording neighbours (LB Labs).
"Down-weighting auto plays reduces noise"	For learning what to <i>recommend</i> , user-started plays are self-selected positives. Auto-continued plays are closest to random exposure, so they deserve weight $\geq$ user-started, or should enter as a feature (inference grounded in Schnabel et al. and Saito et al.).
"The policy risk is storage and ML training"	Policy III.13 also forbids <i>analysis</i> : "Do not analyze the Spotify Content or the Spotify Service for any purpose". Its listed examples include derived listenership metrics and user profiles (Developer Policy). Skip/complete labels fit those examples closely.
"Dev Mode limits are per app"	Since July 2026 Dev Mode quotas are counted per developer account , so all Client IDs share buckets. A 429 now carries <code>"reason": "QUOTA_EXCEEDED"</code> (July 2026 changelog). The docs describe the Dev Mode quota as a separate mechanism from the rolling 30-second rate limit (rate limits). My inference: treat <code>QUOTA_EXCEEDED</code> as "account budget exhausted", which calls for long backoff across <i>every</i> app, not a 30-second retry.
"I can read Discover Weekly via the API"	After February 2026, <code>GET /playlists/{id}/items</code> is limited to playlists the user owns or collaborates on (migration guide). Spotify-owned DW is therefore probably unreadable, and algorithmic/editorial playlist access was already cut in November 2024 (Spotify blog). Copy DW into an owned playlist by hand.
"macOS now-playing is a good fallback"	It is blocked since macOS 15.4 for non- <code>com.apple</code> processes. The <code>mediaremote-adapter</code> workaround depends on a private framework and could break (mediaremote-adapter).
"I can just recreate the app if something breaks"	Don't. Endpoint restrictions were postponed for existing integrations (Feb 2026 blog ; community thread). A new Client ID gets the reduced set immediately and shares the same quota anyway.

3. Comparison tables

Candidate generation and popularity correction

Option	What it fixes	Data/cost	Key risk	Verdict
Current: LB similar-artists summed over ~40 seeds, then 3 first Spotify search hits	—	Cheap	Hubs win on breadth, not fit. Search hits favour hits and remasters	Replace

Option	What it fixes	Data/cost	Key risk	Verdict
LB Labs similar-recordings over ~137 MBID-matched seed tracks	Recording-level fit and deep cuts. Removes the search-hit problem	~137 calls at 1 rps ≈ 2.5 min, cached (LB API rate guidance)	Labs endpoints are unversioned. MetaBrainz may gate them	Primary
mlhd-similar-recordings	A second, older Last.fm-derived population (likely less Top-40 dominated; inference)	Same cost. Payload untested (LB Labs)	Dated population	Secondary blend
Last.fm <code>track.getSimilar / tags</code>	A third graph, and tags	Free key, non-commercial, 100 MB storage cap (Last.fm ToS)	Sparse in the long tail; noisy tags (inference)	Optional
LB CF recommendations	Personal CF	Needs the user's listens on LB plus a model run. Labelled "experimental" (LB rec docs)	Slow to warm up	Later, via LB mirroring
Local embeddings from MLHD+ (~240 GB)	Independence from APIs	Heavy (MLHD+ index)	License unstated	Only if the APIs vanish
Mutual proximity (empirical rank)	Structural hubs	One extra neighbour call per candidate	Zeroes pairs that aren't mutual (by design)	Use (Schnitzer et al.)
CSLS-style penalty <code>sim - ½(r(c)+r(s))</code>	Hubs	Same as MP	Formula adapted from word translation (Conneau et al.)	Alternative to MP
Breadth damping Σ/n_c^{β} , $\beta \approx 0.5$	Many-weak-seeds effect	Free	Heuristic (inference)	Use
<code>pop^(-α)</code> with LB <code>total_user_count</code>	Residual mainstream pull	Batched POST, tokenless today	LB's user base is small and skewed. Use percentiles	Use, α tuned so median ≈ the user's own median
Calibrated popularity re-rank	Over- or under-shooting obscurity	Bucket by LB percentiles	—	Use (Abdollahpouri et al.)
xQuAD popularity variant	Same	—	Formula not re-verified	Optional (arXiv 1901.07555)

Exploration policies

Policy	Strength at ~15 exploratory plays/week	Propensities loggable?	Degeneracy	Verdict
Current 70/20/10 with softmax explore and uniform wildcard	Same pattern as Spotify's BaRT (epsilon-greedy with logged propensities) (BaRT)	Yes, but logged wrongly (k-softmax can exceed 1)	Floor present	Keep the budget, fix the logging
Thompson sampling, hierarchical, 5–8 cluster arms	Simulation: 0.75 chance of being within 5 points of the best arm at K=5, n=100 (vs 0.68 uniform)	Yes, by Monte Carlo over posterior draws	Degenerates faster than UCB or random (Jiang et al.), so keep the wildcard	Stage 2
Top-two TS	Within noise of plain TS at this budget; guarantees are asymptotic (Russo)	Yes	Same	Skip

Policy	Strength at ~15 exploratory plays/week	Propensities loggable?	Degeneracy	Verdict
UCB	Deterministic, so propensities are 0/1	No , which breaks IPS	Slower	Skip
Explore-then-commit	About 2× regret in the 2-arm case; needs the gap known in advance (Lattimore & Szepesvári)	Yes	Commits permanently	Skip
Linear/logistic TS on tag embeddings (d≈8–16)	Sample cost scales with d, not K	Yes	—	Stage 3 option
VW — cb_explore_adf	Full contextual bandit, actively maintained	Yes	—	Overkill for one user

Simulation numbers come from the exploration notes' own run: K arms with true rates uniform on [0.35, 0.65], 600 replications, Monte Carlo SE about ± 0.02 .

Elicitation strategies

Strategy	Data rate	Bias / confounds	User cost	Info value for this warm user	Verdict
(a) Playlist only, better arms	~9 exploratory tracks/week	User chose to play the playlist, so context is cleaner	None	Moderate, concentrated in explore and wildcard	Keep
(b) Queue probes	35–70/week at 5–10/day	Session state, skip-begets-skip, contrast, novelty; context attribution unreliable	17–50% of listening	Low for core genres, some for unseen ones	No, for now
(b') "Up Next" probe playlist	User-paced	Clean <code>context.uri</code> attribution	Opt-in	Same as (b), less confounded	Preferred if probing
(b'') Notify "queue these 2?"	Low	Acceptance becomes an explicit label	Small	Explicit preference	Acceptable variant
(c) One-off taste quiz / pairwise	High per minute	Gap between stated and revealed preference	~10 min once	Good for unseen regions only; cold-start literature (Rashid et al. ; Christakopoulou et al.)	Optional
(d) Passive ESH	Thousands of plays	Self-selected familiar music; drift	Zero	Dominant for priors	Do first

Libraries and datasets

Item	License	Maintenance (as checked)	Data requirement	Fits a single user?	Key risk	Reuse
LB Labs API (<code>similar-recordings</code> , <code>tag-similarity</code>)	Open MetaBrainz service; license not stated on index	Live 2026-10-08 (LB Labs)	MBIDs	Yes	Anti-scraping	Candidate generator

Item	License	Maintenance (as checked)	Data requirement	Fits a single user?	Key risk	Reuse
LB core API (popularity, metadata <code>inc=tag</code>)	—	Active; server pushed 2026-10-08 (listenbrainz-server , GPL-2.0)	MBIDs; 1 rps	Yes	<code>/metadata/lookup</code> already token-gated	Popularity, tags, ISRCs
troi-recommendation-playground	GPL-2.0	Active, pushed 2026-09-28; PyPI 2026.9.1.0 (GitHub)	LB	Yes	GPL only matters if distributed	Copy the pipeline ideas
liblistenbrainz	GPL-3.0	Pushed 2026-09-24 (GitHub)	LB token	Yes	—	Listen submission
MLHD+ dataset	Not stated	Snapshot dated 2023 (index)	~240 GB	Barely	License; disk	Skip
MusicBrainz core data	CC0 (MB wiki)	Server very active	MBIDs	Yes	Supplementary data is CC BY-NC-SA	Genres, release-group types
python-musicbrainzngs	NOASSERTION (BSD-2 in practice; unverified)	Stale, last push 2024-06 (GitHub)	—	—	Bitrot	Use raw <code>requests</code> instead
pylast	Apache-2.0	Active, 7.2.0, pushed 2026-10-06 (GitHub)	Free key	Yes	100 MB cap, non-commercial	Tags, similar tracks
AcousticBrainz dump	CC0	Shut down; download timing out	—	—	MetaBrainz itself called the data not good enough (archived post)	Avoid
Deezer public API (<code>bpm</code> , <code>rank</code> by ISRC)	Terms unverified; guidelines cover audio only (Deezer)	Live in probes	ISRC	Yes	Unknown metadata terms	Optional tempo proxy
scikit-hubness	BSD-3	Effectively unmaintained (2024-05) (GitHub)	—	—	Bitrot	Copy formulas
LightGBM	MIT	Active, v4.x (docs)	≥2,000–5,000 labelled plays (inference)	Only after ESH	Overfitting	Stage 3
scikit-learn	BSD-3	1.9.1, 2026-09-11	Small	Yes	—	Logistic calibrator, $\hat{q}$ for DR
Open Bandit Pipeline (obp)	Apache-2.0	Last PyPI release 0.5.7 (2023), push 2024-06 (GitHub)	Logged propensitys	Yes	Dependency friction with numpy 2	Vendor the estimators
Vowpal Wabbit	BSD-3 (PyPI) / NOASSERTION (GitHub)	Active, 9.11.9, 2026-09-27 (GitHub)	—	Overkill	Complexity	Skip

Item	License	Maintenance (as checked)	Data requirement	Fits a single user?	Key risk	Reuse
fast-map-dpp	Apache-2.0	Stale (2020) (GitHub)	Similarity kernel	Yes	—	Reference only
DPPy	MIT	0.3.3, 2024-08 (GitHub)	—	—	Built for sampling, not MAP	Skip
spotipy	MIT	2.26.0 (FreshPorts)	—	Yes	Support for Feb 2026 <code>/items</code> and <code>/me/library</code> unconfirmed	Check or call raw HTTP
pywinrt (<code>winrt*</code>)	MIT (unverified)	Active; <code>winsdk</code> archived Oct 2024 (pywinrt)	Windows	Yes	Package names in flux	SMTTC fallback
mediaremote-adapter	BSD-3 (unverified)	Community workaround (GitHub)	macOS	—	Private framework	Avoid
<code>arch</code> (block bootstrap), PyMC/bambi, Ollama/llama.cpp, pydantic	NCSA / MIT / Apache / MIT per the notes' prior knowledge, not verified this session	Believed active	—	Yes	Verify on PyPI	Bootstrap, Bayesian A/B, local LLM parsing

4. Staged roadmap

Stage 0: the next two weeks, on the data you have now (all S). Port the poller to the Windows PC as an adaptive `currently-playing` loop. Poll at $\min(\text{remaining_ms} + 1.5 \text{ s}, 15 \text{ s})$ while playing, 30–60 s when idle, and 120 s after 10 idle minutes. On a 429, honour Retry-After, with exponential backoff capped at 15 minutes ([evaluation notes](#), [rate-limit guide](#)). Add the hourly recently-played reconciler. Rewrite labels as of play time and replace sample weights with features. Swap candidate generation to `similar-recordings` with mutual proximity, breadth damping, pop^{α} and a calibrated re-rank. Rebuild within-artist selection on LB top-recordings and add the deep-cuts slot. Replace k-softmax propensity logging with a Monte Carlo replay of the whole generator ($R = 10,000\text{--}50,000$). Add the TTL purge and the disconnect-and-delete path. Get a free LB token. Two weeks of this produces roughly 280–420 captured plays (inference, at 20–30/day). That is enough to estimate the share of auto plays and per-session skip autocorrelation, and too few to train anything.

Stage 1: once the Extended History arrives (M). Import it, mapping `reason_start/reason_end` onto `start_kind` and validating the mapping against overlapping poller days. Use only the export for model training, which is the lowest-risk input under the Developer Terms (inference; see Part A §11). Fit empirical-Bayes shrunk skip and completion rates per artist, tag cluster and contextxshuffle, with $m \approx 10\text{--}20$ pseudo-plays. Build 4–8 taste clusters with long-term and short-term half-lives. Measure the within-day ICC of completion, which the A/B power calculation needs. Run the offline `--novel` backtest over years of rolling origins against decayed replay and item-kNN. Start the **pre-registered blind interleaved test against DW** (copied by hand). Turn on hierarchical Thompson sampling for the explore band only, keeping the wildcard. Fit a small regularised logistic calibrator if there are $\geq \sim 300$ plays (about 100 skip events).

Minimum viable experiment for the queue/genre question (run in stage 1, only if gated). The gate: ESH leaves at least 5 clusters with posterior SD > 0.10 or fewer than 20 historical plays. Take 8–12 such uncertain arms and randomly assign half to PROBE and half to CONTROL, stratified by prior mean. PROBE arms get at most one probe per session, about 5 per day in total, with a logged coin $q \approx 0.3$ at each eligible opportunity.

Preferably deliver them through a dedicated "Up Next" playlist rather than the queue. Both conditions keep identical playlist slots. Run 4–6 weeks, aiming for 30–40 probes per probe arm and ≥ 30 scored playlist tracks per condition. The primary metric is each arm's predictive log-loss on next-week playlist completion, relative to the ESH-only prior. Success means all of the following: PROBE's improvement minus CONTROL's has a bootstrap 90% CI (over weeks) excluding 0; overall playlist completion falls by no more than 5 pp; and self-rated annoyance is acceptable. Otherwise retire probing. Power is honest but limited: only an improvement of about ≥ 0.05 nats/track is detectable, assuming per-track log-loss SD ≈ 0.3 (an assumption).

Stage 2: only after the blind A/B shows a signal (L). Promote the logistic model to a LightGBM binary or ordinal model once there are 2,000–5,000 labelled plays, using a time-based split. Keep it only if it beats decayed replay and item-kNN by more than the bootstrap CI. Use LambdaMART only with genuine candidate slates. Add SNIPS and DR off-policy evaluation, reporting ESS. Add the LLM steering parser. Re-test with the ranker as a new arm in the same interleaved design.

Part A. Architecture critique in detail

A1. Recording-level co-listening beats summing artist neighbours

Today's pipeline sums normalised artist similarities over about 40 seeds. A hub artist that appears in many seeds' neighbour lists therefore wins on breadth rather than fit. The fix is to change the unit of candidacy. **LB Labs similar-recordings returned HTTP 200 without a token.** A Portishead seed returned Björk's "Venus as a Boy" with score 76 (LB Labs). With about 137 of the user's ~ 170 taste tracks matched to MBIDs (the 81% rate), one pass costs about 2.5 minutes at LB's one-request-per-second guidance (LB API docs). Score as $\text{cand}(r) = \sum_s w_s \cdot \text{sim}(s, r) / \max \text{sim}(s, \cdot)$. Then apply the hub correction in a fixed order:

1. **Mutual proximity.** Use the empirical-rank form $(1 - \text{rank}_s(c)/\text{IN}(s)) \cdot (1 - \text{rank}_c(s)/\text{IN}(c))$. It drops to 0 when the seed is not in the candidate's own top-50, so it suppresses hubs automatically. MP "significantly increased retrieval quality, reducing hubs" in music similarity (Schnitzer et al., JMLR 2012).
2. **Breadth damping.** Divide by $n_c^{0.5}$.
3. **Popularity penalty.** Multiply by $\text{pop}^{-\alpha}$ with $\alpha \in [0.1, 0.5]$, where pop is LB `total_user_count` from `POST /1/popularity/recording` (LB docs), log-scaled and expressed as a percentile.
4. **Calibrated popularity re-rank.** Make the list's Head/Mid/Tail mix match the user's own (Abdollahpouri et al.).

The motivation for steps 3 and 4 is well established. On Last.fm data, algorithms favour popular items and serve users with low mainstream inclination worse (Kowald et al.). The weights (α , β , the calibration λ) are tuning choices, not sourced values. Tune α so the median candidate popularity is about the median of the user's own tracks. Keep similar-artists, followed by `top-recordings-for-artist`, as a secondary route. Popularity proxies outside Spotify matter because Spotify **removed popularity from track, artist and album objects** in February 2026 (Feb 2026 changelog).

A2. Within-artist choice should prefer deep cuts that fit

Taking Spotify's first three search hits picks the hit single, remaster or live version. Search `limit` is now capped at 10 (Feb 2026 changelog) and artist top-tracks is gone in Dev Mode, so the replacement has to be non-Spotify.

1. Pull 50–100 recordings from LB `top-recordings-for-artist` (path form). It returns `release_mbids`, `length` and listener counts (LB docs).
2. Keep only Album, EP and Single release groups. Drop the secondary types Live, Remix, Compilation, DJ-mix and Demo, and apply a title regex for remix, live, karaoke, "sped up" and similar.
3. Score each remaining track as $t_{\text{fit}} = a \cdot \max_s \text{sim_rec}(s, t) + b \cdot \cos(\text{tags}_t, \text{user_tags}) - \alpha \cdot \log \text{pop}(t)$.

4. Pick with MMR, penalising same-release-group pairs, which naturally limits picks to about one per album.

Then resolve to a Spotify URI with an `isrc:` search. ISRCs survived because `external_ids` was reinstated in March 2026 ([March 2026 changelog](#)). Add a separate, capped "deep cuts from known artists" slot of 20–30%. The supporting evidence is suggestive rather than strong. Spotify's exploration study shows listening alternates between exploration phases and revisiting ([ICWSM 2022](#)). A small study ($n = 19$) found that repeated exposure raised adoption of new songs ([ISMIR 2020](#)). Re-surfacing promising tracks two or three times is therefore reasonable but weakly supported.

A3. The taste model needs several interests and two timescales

A square-rooted, decayed per-artist sum is a sensible concave damper: $9\times$ the weight gives $3\times$ the seed weight. But it averages taste into a single profile. PinnerSage keeps several medoids per user, weighted by decayed importance. Offline it improved relevance by **+110% versus +28%** for a decayed-average embedding, and light users get only 3–5 clusters ([PinnerSage](#)). Spotify's CoSeRNN found recent and contextual consumption "much more" predictive than static averages ([Hansen thesis](#)).

For about 170 tracks, use 4–8 Ward clusters on tag or co-listening vectors and score candidates by max-similarity to the top-3 medoids. Keep a 365-day "who I am" profile alongside a 30–90-day "what I'm into now" profile, combined as $\max(\text{sim_long}, \alpha \cdot \text{sim_short})$ with $\alpha \approx 0.5\text{--}0.7$ to tune. Route the 67-track Chill playlist into its own context cluster rather than pooling it. Otherwise it is about 40% of the profile (inference). No source tests any of this at $N \approx 170$, so the direction is reliable and the magnitude is unknown.

A4. A learned ranker wins only with thousands of labels

Reproducibility studies keep finding that tuned simple baselines match or beat complex models. Only 7 of 18 neural recommenders were reproducible, and 6 of those 7 were often beaten by kNN or graph heuristics ([Ferrari Dacrema et al.](#)). kNN usually won in session-based recommendation ([Ludewig et al.](#)). Sample-size criteria for prediction models call for roughly 4.8–23 events per parameter ([Riley criteria](#)).

With 16 labelled plays, about 5–8 of which are minority-class events, nothing learned is supportable. The inferred thresholds are:

Labelled plays	What is supportable
~300 (~100 skip events)	Logistic regression with 5–10 features
2,000–5,000	GBDT with <code>num_leaves</code> 7–15 and <code>min_data_in_leaf</code> 30–50

Start with a binary or ordinal objective. LambdaRank needs integer labels and query groups ([LightGBM docs](#)), and a single playback stream doesn't produce real groups.

Fix leakage now:

- `saved_asof_play` (`added_at` < play time) is a familiarity *feature*.
- `saved_within_W` ($W = 24\text{ h or }7\text{ days}$) is the only thing that upgrades a label.
- Labels stay provisional until W closes.

Fix exposure bias with features (`start_kind`, session skip state) and by training on plays the recommender itself served, with logged propensities ([Schnabel et al.](#); [Saito et al.](#)). Global-timeline splits avoid the leakage that random splits introduce ([Ji et al.](#)).

A5. Keep the 70/20/10 budget and fix how propensities are computed

At about 15 labelled exploratory plays per week, no algorithm can identify the best of 10–20 arms within months. In the notes' simulation with $K = 15$ and $n = 300$ (about 20 weeks), Thompson sampling landed within 5 points of the best arm only 77% of the time. The right moves are:

- Collapse to 5–8 arms, or use a hierarchical Beta prior $\theta_c \sim \text{Beta}(m\mu, m(1-\mu))$ with $m \approx 5\text{--}15$.
- Use TS only to *allocate* explore slots.
- Discount posteriors weekly ($\gamma \approx 0.97$).
- Keep the uniform wildcard, because TS degenerates faster than UCB or random selection ([Jiang et al.](#)).

The current logging has a concrete bug. The explore band samples without replacement (Plackett–Luce), so the inclusion probability is **not k-softmax**. In the notes' worked example (20 items, $k = 6$, $\tau = 0.02$), the naive value for the top item is **1.40, which is impossible**, against an exact 0.90. The artist cap shifts probabilities further. Replay the entire weekly generator 10,000–50,000 times and $\log \hat{\pi}(i) = \text{count}/R$. Keep τ high enough that the max/min π ratio within the band stays $\leq 5\text{--}10$. There is also now an exact, sample-free method ([ECIR 2026](#)).

For OPE at about 300 exploratory plays:

- Primary estimator: SNIPS with weights clipped at 10–20.
- Check: Switch-DR with a cross-fitted logistic $\hat{q}$ ([obp estimator definitions](#)).
- Always report $\text{ESS} = (\sum w)^2 / \sum w^2$.
- Bootstrap over weeks.

The uniform wildcard over a tail of 500 produces IPS weights of about 167. It is good for exploration but nearly useless for evaluating a concentrated target policy. OPE at this scale catches only large differences ($\geq 15\text{--}20$ points).

A6. Calibration beats DPP at 30 tracks, and narrowing has to be monitored

Use Steck-style calibration as the main re-ranker, so the list's genre mix matches a 6–12-month history that includes organic listening ([Steck 2018](#)). Add an MMR penalty on artist or tag embeddings ([Carbonell & Goldstein](#)) and the artist cap. Fast greedy DPP takes about 30 lines ([Chen et al.](#)), but no head-to-head study shows it beating MMR at $N = 30$ on tag-only features. It is optional, not a priority.

On feedback loops:

- At Spotify, algorithmically driven listening is associated with lower diversity ([Anderson et al. 2020](#)).
- Training on recommender-shaped data homogenises behaviour in simulation ([Chaney et al.](#)) and amplifies popularity bias ([Mansoury et al.](#)).
- Newer simulations disagree on whether *individual* diversity falls ([Anwar et al.](#); [arXiv 2510.14857](#)).

So monitor rather than assume. Track the generalist–specialist score, $\exp(\text{genre entropy})$, the top-5-artist share (alert above about 40%), and the playlist's share of total listening (raise exploration above about 50%). These thresholds are inferences. Never train on confident-slot outcomes without IPS weighting.

A7. Labels: skips mean "not now" more often than "never"

Population skip rates are high. Reported Spotify figures from 2014 are 24% within 5 s, 35% within 30 s and 48.6% before the end ([Hypebot on Lamere](#)). Skips are autocorrelated within sessions ([MSSD](#)). The user's own behaviour features dominate skip prediction over content ([Meggetto 2023](#)). Skip times cluster at section boundaries specific to each song ([Montecchio et al.](#)).

The practical consequences:

- Use completion fraction rather than fixed second cut-offs.
- Make session skip state a feature.
- Treat skips as negative for track $\times$ context rather than for the track globally.
- Decay negatives faster (30–60-day half-life).
- Require ≥ 2 skips in different sessions before a track-level penalty outweighs one completion.
- Drop "skipper" sessions (session skip rate above about 70%, an assumption) from track-level aggregation.

The ordinal grades for LightGBM, 0–4 from a <10 s skip up to completion plus a save or replay within W, keep the current shape of the scale. Replays count only when user-started. Seeks stay features until there is calibration data. The $\times 1.5$ intent multiplier and the start_kind weights should become features (see the corrections table).

A8. Evaluating one listener: offline screens, the blind test decides

The kit already splits on a global timeline, which is the literature's main requirement (Ji et al.; Hidasi & Czapp). The split strategy alone can reorder model rankings (Meng et al.). Improvements:

- Weekly expanding origins.
- A 1–7-day gap between train and test.
- In `---novel` mode, report Recall@50/100 as well as @10. Only about 15–60 novel positives fall in each test week (inference).
- Always include recency-decayed replay as a baseline.

Blind test against DW. Use team-draft interleaving of DW and model picks into one neutrally named private playlist. Interleaving was validated in 38 online experiments and is far more sensitive than A/B splits (Schuth et al.; Chapelle et al.). Apply a familiarity filter before drafting. Record the arm only in DuckDB. Pre-register following CENT (BMJ). If you want to peek, use always-valid inference (Johari et al.).

Sample size uses a two-proportion test ($\alpha = 0.05$, power 0.8) with **assumed** ICCs:

Effect	n per arm	Weeks at 15 tracks/arm/week
0.35 $\rightarrow$ 0.50	170	≈ 12 independent, ≈ 20 at design effect 1.7
0.45 $\rightarrow$ 0.55	392	≈ 26 –45

Multiply by about 1.4 if the familiarity filter removes 30% of tracks. A sign-flip permutation test across weeks needs at least 6 weeks for $p < 0.05$ to be possible at all.

ESH as a pseudo-experiment. Shuffle is user-chosen, so the history supports calibrating outcome definitions, ICCs and position effects within shuffle-on sessions, but not causal claims. `skipped` sometimes disagrees with the reason codes, and some rows lack metadata (Spotify Community).

A9. Reliability: fewer moving parts, more reconciliation

Cut:

- 4-second polling, replaced by adaptive polling (~ 5 – $10\times$ fewer calls).
- The Mac from the capture path.
- Last.fm scrobbling, which duplicates ListenBrainz.
- LB `playing_now`, which is stored only temporarily (LB JSON docs).

Keep: one poller on the always-on PC, the hourly recently-played reconciler, the ESH import deduplicated on (uri, played_at $\pm$ 30 s), and a heartbeat that alerts on 24 h with no plays, token-refresh failure, or a PUT/GET mismatch.

Recently-played gives `played_at` but not `ms_played`. Backfilled rows get a null completion, and gaps longer than about 1.5–2 days are lost until the next export (inference). Always persist the newest refresh token, because PKCE refresh tokens rotate ([Spotify](#)). Keep the canonical weekly list in DuckDB and treat the playlist as a rendering of it. If it vanishes, recreate it with `POST /me/playlists`. If Premium lapses the app stops working, so the fallback is SMTC capture plus a URI list to paste.

A10. LLMs belong around the recommender, not inside it

Zero-shot LLMs can be competitive conversational recommenders but over-represent popular items ([He et al.](#)). An Amazon study found the opposite bias direction in movies, at lower accuracy than CF ([Lichtenberg et al.](#)). Spotify's Text2Tracks shows that generating titles is inefficient and needs entity resolution, and that emitting IDs works better ([Text2Tracks](#)).

Use LLMs for three things:

- **Steering.** Parse requests into a pydantic schema, resolve artist names against the local table, and reject output that fails validation.
- **Explanations.** Generate them only from logged features, with an extractive check.
- **Tag normalisation.** Map raw tags onto a fixed taxonomy.

A local 7–8B model is enough for all three. Send only aggregates to cloud APIs. If an LLM is ever tried as a ranker, it must pick from the candidate set and enter as one more arm in the `--novel` backtest. A widely repeated "+127% Hits@10" figure for Text2Tracks is unverified.

A11. Spotify terms re-check: analysis, not storage, is the sleeper risk

The current texts are **Developer Terms v10 and the Developer Policy, both effective 15 May 2025**, with nothing newer through October 2026 ([Terms](#); [Policy](#)). In summary (paraphrased; check the clause numbers at the links):

- **Terms §II** defines Spotify Content as essentially any data made available through the platform. That covers names, URIs, ISRCs fetched from Spotify, and playback state.
- **Terms §III.1** limits the licence to private personal use on approved devices.
- **Terms §IV.2.a.1 and Policy III.14** prohibit using Spotify Content to train, or otherwise ingest into, an ML or AI model.
- **Policy III.13** prohibits analysing Spotify Content for any purpose. Its examples include new or derived listenership metrics and building user profiles. This is the clause most likely to cover skip/complete labels and per-genre completion rates.
- **Terms §IV.3** bars storing, aggregating, or compiling databases of Spotify Content except as strictly necessary to operate the app, and requires reasonable efforts to delete older data. Content may not be stored indefinitely. Local caching is limited to temporary metadata and cover art.
- **Terms §V.7–8 and the Data Protection Appendix** allow processing personal data only as long as necessary, and require deletion within five days of disconnection.
- **Policy III.11** bars mimicking or replicating a core Spotify experience. This is ambiguous for an automated recommend-and-queue loop that resembles autoplay.
- **Terms §II** includes controlling a background Spotify app within "Streaming", and the Policy restricts streaming to Premium users. Driving your own queue is therefore within scope.

No clause exempts single-user apps, and no official interpretation says whether a bandit or heuristic counts as an "ML model". Note too that February 2026 framing says Dev Mode should not be relied on as a foundation for a business ([Feb 2026 blog](#)).

The user's tables ranked by risk, highest first (inference from the clauses above):

1. **Picks log with scores and propensities**, once any model is fit on API-derived features. This hits training (IV.2.a.1, III.14) and analysis (III.13).
2. **Plays table** with Spotify URIs, names, timestamps and polling-derived skip labels. It is a growing database of personal data and plausibly a derived listenership metric.
3. **Liked-songs and Chill mirror**. A compilation, defensible only as a short-lived cache refreshed each run.
4. **Identity cache**. URIs and Spotify-sourced ISRCs are Spotify Content. MBIDs are not.
5. **Labels derived from the ESH export or manual ratings**. Arguably outside the Developer Terms because they were not obtained through the platform. This is an inference: the privacy-download page was not checked.
6. **The private playlist written via PUT**. It lives inside Spotify, so it is low risk.

Compliant mitigations:

- Store keys only and resolve display text from MusicBrainz.
- Derive every model feature from MusicBrainz, ListenBrainz or Last.fm.
- Train only on ESH or self-logged labels, and use the API for actuation and transient state.
- TTL-purge raw API plays after days to weeks.
- Keep the app non-commercial and unshared, with a working disconnect-and-delete path.
- If you want certainty, ask Spotify directly.

None of the recommendations in this report involve scraping Spotify.

Part B. Queue injection plus genre probing

B1. Value of information is front-loaded and mostly gone for a warm user

Every quantified gain from active elicitation in the notes is measured against a *cold* baseline. Christakopoulou et al. report a 25% improvement after just two questions ([MSR](#)). Rashid et al. and Golbandi et al. study new-user interviews ([Rashid](#); [Golbandi](#)). Elahi and Rubens report that active-learning strategies behave differently once natural, passive data also flows in ([Elahi et al.](#); secondhand). Deezer found that registration-day streams alone were already a strong baseline in one setting, with NDCG@50 of 17.72 vs 20.38 ([Briand et al.](#)). Value of information is front-loaded. For core genres this user is long past it. What remains sits in clusters absent from history, in drift, and in context-conditional preferences.

Worked sample size (assumptions: $p = 0.3$, 95%, independent probes):

- $n = 1.96^2 \cdot 0.3 \cdot 0.7 / 0.10^2 = 80.7 \rightarrow 81$. The Wilson half-width at $n = 81$ is 0.098. For ± 0.05 , $n = 323$.
- With 3 probes per session at $p = 0.1$ – 0.2 , or AR(1) $r = 0.1$ – 0.3 , the requirement rises to **97–150 raw probes**. These p values are assumptions, because MSSD confirms the correlation but gives no coefficient ([MSSD](#)). One probe per session on separate days brings the design effect back to about 1.
- Weeks to reach 81 per arm:

Arms (K)	At 5 probes/day	At 10 probes/day
3	6.9	3.5

Arms (K)	At 5 probes/day	At 10 probes/day
5	11.6	5.8
10	23.1	11.6
20	46.3	23.1

Multiply by the design effect (1.2–1.8).

- Detecting a 0.30 vs 0.45 difference between arms takes about 162 probes per arm. 0.30 vs 0.40 takes about 356.

When do probe estimates beat the prior? A Beta prior worth k pseudo-counts saves about k probes (81 $\rightarrow$ 71 at $k = 10$, $\rightarrow$ 41 at $k = 40$).

- **Liked-songs prior only (pre-ESH).** This prior contains only positives, so it is weak about completion. My inference is that it is worth only a few pseudo-counts per genre, and about 20–40 probes per unseen genre (1–3 weeks per arm with 2–3 arms rotating) is where probe data starts to dominate.
- **ESH prior.** For major genres with hundreds of historical plays, probe data **essentially never** beats it. With $k = 40$, 30 probes move the half-width only from 0.14 to 0.11.

The James–Stein check gives the general rule. At $n = 30$ the sampling SD is about 0.084. If this user's true per-genre completion rates spread with $SD \leq 0.08$ around the liked-songs mix, 30 probes per genre will not beat the pooled prior. Fit the Beta-binomial and read the shrinkage factor directly.

B2. Expected effect on playlist quality is about +2 points at best

This bound is inference, with every assumption shown. Probing can only improve the allocation of exploratory slots: 6 explore slots, plus 3 wildcard slots that should stay uniform. Suppose the arms' true completion rates are spread uniformly over $[0.35, 0.65]$, as in the exploration notes' simulation. A random arm then averages 0.50 and the best of five averages 0.60. A **perfect oracle** therefore adds 10 points on 6 of 30 tracks, which is **+2 pp whole-playlist completion**. Probing only speeds up learning that the playlist's own exploratory plays would produce anyway, so its incremental contribution is a fraction of that ceiling. In the simulation, Thompson sampling beat uniform round-robin by only 0.07 in probability of finding a near-best arm at $K = 5$, $n = 100$.

Detecting $0.45 \rightarrow 0.47$ needs about **9,700 tracks per arm**, about 325 weeks at 30 tracks per week (computed with the same two-proportion formula). A single week's 30-track playlist already has a hit-rate SE of about ± 0.09 . The user-study evidence on steering shows perceived benefits such as diversity and control, at a cost in cognitive load, and no quantified accuracy lift ([MusicBot](#)). Expect probing to sharpen estimates for unseen genres. Do not expect it to show up in measured playlist quality.

B3. Injection confounds and how to log and randomise against them

A queued probe is a different treatment from a playlist track:

- **Session state.** Skips are autocorrelated ([MSSD](#)), behaviour features dominate prediction ([Meggetto 2023](#)), and session types vary by time and playlist ([Meggetto 2021](#)).
- **Contrast.** An unfamiliar-genre probe gets judged against the familiar tracks around it.
- **Novelty and annoyance drift** over the course of the experiment.
- **Start-reason differences.** One user's history shows skip rates near 100% for `popup` starts versus about 2% for `trackdone` starts ([community dashboard](#)).

Randomise: at each eligible opportunity, flip a coin with logged probability q , choose the arm by TS with its Monte Carlo selection probability logged, and allow at most one probe per session. Inject either at a random

point or only after ≥ 2 consecutive completions.

Log: probe_id, the previous 1–3 outcomes, the session skip rate, neighbour genres and embedding distance, time since the experiment started, and the delivery source as a covariate.

Outcome: use `ms_played` thresholds (>30 s, $\geq 80\%$) and exclude logout, unexpected-exit and trackerror endings. Unexposed probes count as missing, not as skips.

B4. Genre labels are a poor arm unit

Even after mapping to coarse top-level genres, crowd sources agree only **75.8–84.1%** pairwise, and only 67.4% of Last.fm-tagged tracks could be mapped at all ([Schreiber, ISMIR 2015](#)). About 1 in 5 probes would therefore be credited to the wrong arm. That is a noise floor probing cannot remove (inference). Spotify artist `genres` were not listed as removed, but a forum report describes a user's distinct-genre count falling from 1,138 to 373 ([Spotify Community](#)). The batch artist GET is also gone.

Arm options, ranked by expected value:

1. Clusters over the user's own ESH co-listening graph or embeddings.
2. A hierarchical Beta-binomial, or logistic regression on multi-hot MusicBrainz/Discogs tag vectors with partial pooling.
3. Artist arms nested under clusters.
4. Flat genre arms, which are the weakest.

Genre $\times$ context interactions cannot be learned from 35–70 probes per week, only from ESH.

B5. A controller that survives the queue API, and why a playlist is better

The API facts set the constraints:

- `POST /me/player/queue` is Premium-only and append-only. It returns 204 "Command received", which does not confirm placement, and its execution order is not guaranteed relative to other player calls ([Add to queue](#)).
- No remove or reorder endpoint is documented.
- `GET /me/player/queue` can't tell user-queued items from context items ([Get queue](#)). Community reports say it is capped at about 20 items and padded with repeats ([thread](#)).
- `context` can be null ([playback state](#)), and adding tracks via the API can drop the playlist context ([thread](#)).

A robust controller works within those limits:

1. Inject only when the track is playing, the device is active and unrestricted, the session is not private, and the item type is `track`.
2. Queue once, 1–2 tracks ahead.
3. Detect exposure by matching `item.uri` or ISRC in later polls, never by `context`.
4. Mark a probe unplayed if it doesn't appear within N minutes.
5. Back off fully on `QUOTA_EXCEEDED`.

A second "Up Next" probe playlist is better. It gives clean `context.uri` attribution, the user chooses when to play it, it adds no injection-time confounds, and it uses the same playlist PUT path that already works. A **notification** ("queue these 2?") is the next-best option. Acceptances become explicit labels and the user stays in control. Both avoid unwanted probes that can't be removed, and both sidestep the ambiguity of Policy III.11 around an automatic queue loop that resembles autoplay.

B6. Cost-benefit: passive history first, playlist second, probing last

Option	Labels per month	Confounding	Effort	Policy exposure	Gain on playlist quality	Rank
(d) Passive ESH	Thousands at once (inference: 7,000–11,000 plays/year at 20–30/day)	Self-selected familiar music, drift	Monocentric	Lowest (user-owned export)	Largest: priors for every heard cluster	1
(a) Playlist only, better arms	~40–60 playlist plays, ~12–18 exploratory	Low	S	Low	Moderate, in explore slots	2
(c) Taste quiz / pairwise	20–50 explicit answers once	Stated vs revealed	S	None (no Spotify data)	Small, for unseen regions only	3
(b) Queue probes	150–300 at 5–10/day	High	M–L	Medium (III.11, III.13)	≤ +2 pp ceiling	4

Recommendation: do (d) and (a) now. Optionally do a 10-minute (c) over unseen clusters, as weak priors ($k \approx 2-5$). Run (b) only as the gated minimum experiment in §4, preferably delivered through the probe playlist.

Staged plan:

1. Stage 0: build the ESH importer and probe-ready logging.
2. Stage 1: fit ESH priors and identify uncertain clusters. Run the minimum experiment only if at least 5 clusters qualify.
3. Stage 2: fold successful arms into the TS explore band, and retire probing if the success criterion fails.

B7. Alternatives that may dominate probing

ESH pre-fitting dominates for everything the user has heard. **Population priors** from LB/Last.fm co-listening and tag similarity (e.g. `tag-similarity`) give cheap shrinkage targets for unseen genres ([LB Labs](#)). **Offline embeddings before any online questioning** "helps a great deal" ([Christakopoulou et al.](#)). **LLM-assisted Bayesian elicitation** improves question efficiency, but only in cold-start studies ([OPEN](#); [PEBOL](#)). **Repeated exposure of promising tracks** in the playlist may matter more than learning which genres to probe (weak evidence, $n = 19$).

5. Evidence grading

Claim	Grade	Hype check
Simple tuned baselines often match or beat complex recommenders	Replicated (Ferrari Dacrema ; Ludewig ; Rendle)	Counters "add LightGBM now"
Hubness reduction improves music similarity	Replicated (several Schnitzer/Flexer papers)	—
Popularity bias hurts niche-taste users	Replicated (Kowald ; Abdollahpouri)	—
Ignoring the global timeline causes leakage	Replicated (Ji ; Sun ; Hidasi)	—
Interleaving is more sensitive than A/B splits	Replicated at web scale (Chapelle ; Schuth)	Single-user transfer is inference
Skips autocorrelate within sessions; behaviour features dominate	Replicated qualitatively (MSSD, Meggetto $\times 2$)	No coefficient published

Claim	Grade	Hype check
Multi-interest medoids beat averaged profiles (+110% vs +28%)	Single paper (PinnerSage)	Pinterest scale; N=1 size unknown
Algorithmic listening associates with lower diversity	Single paper , observational plus RCT (Anderson)	Secondary write-ups overstate it as "filter bubbles"
Individual diversity falls under feedback loops	Contested (simulations disagree)	Monitor, don't assume
TS degenerates faster than UCB or random	Single paper , simulation (Jiang)	—
Active elicitation yields large gains	Replicated, cold-start only	Hype for warm users : no warm-user study found
Genre labels agree 76–84% at top level	Single paper (Schreiber)	—
Population skip rates (24% / 35% / 48.6%)	Blog , 2014 data (Hypebot)	Old; per-play vs per-listener unclear
<code>trackdone</code> starts skip ~2%, <code>popup</code> ~100%	Anecdote , one user	Check in your own ESH
LB popularity and similar-recordings are tokenless	Direct probe 2026-10-08	Could be gated at any time
Dev Mode quota shared per account; <code>QUOTA_EXCEEDED</code>	Primary doc (July 2026)	Meaning for backoff is my inference
DW unreadable via API	My inference from the owns/collaborates rule	Test it
LLMs as rankers	Conflicting single papers	Hype : use as parser only. "+127% Hits@10" for Text2Tracks is unverified
DPP beats MMR for 30-track lists	No evidence at this scale	Hype for this use
AcousticBrainz features as taste signals	Maintainers themselves called them unreliable	Avoid
Probe gain $\leq +2$ pp; 81 probes per arm	My inference / arithmetic	Assumptions shown in Part B
Sample-size thresholds for GBDT (2,000–5,000 plays)	My inference from Riley criteria	—

6. Contradictions and open questions, each with a settling experiment

Contradiction / open question	Experiment on your own data
Were endpoint restrictions postponed for existing Dev Mode apps (Feb blog), or did existing apps migrate on 9 March (migration guide)?	With the <i>existing</i> Client ID, call one removed endpoint (e.g. artist top-tracks) once and log the status. Don't create a new app to compare.
User saw a 401 on LB popularity; probes returned 200	Call the path form and the query form once each, with and without a token, and log the codes.
Can DW be read via API?	One <code>GET /playlists/{DW_id}/items</code> . A 403/404 confirms the manual-copy plan.
Does Spotify <code>genres</code> still return values in Dev Mode?	Single-artist GETs for the 40 seeds; count the non-empty <code>genres</code> lists.
MSSD's "34–35%" figure is worded as a non-skip rate and conflicts with other figures	Compute your own non-skip rate on unfamiliar tracks from ESH, and use that as the base rate instead of 0.3.
Do <code>skipped</code> and <code>reason_end</code> agree?	Cross-tabulate them in ESH; define skip from the subset where they agree.

Contradiction / open question	Experiment on your own data
Within-session skip correlation (ρ) is unknown, so the design effect is assumed	Lag-1 autocorrelation of skips within ESH sessions (gap >30–60 s); plug it into the B1 and A8 formulas.
Does recently-played include plays under 30 s?	Play and skip a few tracks at under 30 s, then compare recently-played with poller logs.
LLM popularity bias: more (He) or less (Lichtenberg)?	Add an LLM "pick from these 200" arm to the <code>--novel</code> backtest; compare median LB user counts.
Does individual diversity fall under your own loop?	Track the GS score and exp(genre entropy) weekly; compare 8 weeks before and after deployment.
Is between-genre variance large enough for probing to beat priors?	Fit a Beta-binomial on ESH per cluster. If $\tau < \sim 0.08$, probing can't beat the prior at $n \approx 30$.
Do queued probes complete at different rates than playlist tracks?	In the minimum experiment, put one shared arm in both channels and estimate the source offset.
Does a bandit or heuristic count as an "ML model" under §IV.2.a.1?	Not settleable by experiment; ask Spotify. Meanwhile train only on ESH labels.

7. Reading list, in order

1. **Spotify Developer Policy and Developer Terms v10** — the constraints that apply to everything else ([Policy](#); [Terms](#)).
2. **Feb 2026 migration guide plus the July 2026 changelog** — what still works and how quota is now counted ([guide](#); [July](#)).
3. **ListenBrainz Labs index and Popularity API docs** — your new candidate and popularity sources ([Labs](#); [Popularity](#)).
4. **Schnitzer et al., mutual proximity (JMLR 2012)** — the hub fix ([PDF](#)).
5. **Abdollahpouri et al., calibrated popularity** — keeping your own level of mainstreamness ([arXiv 2007.12230](#)).
6. **Brost et al., Music Streaming Sessions Dataset** — what skips mean ([ar5iv](#)).
7. **PinnerSage** — multi-interest taste with medoids ([ar5iv](#)).
8. **Ferrari Dacrema et al., "Are we really making much progress?"** — why baselines first ([ar5iv](#)).
9. **Russo et al., A Tutorial on Thompson Sampling** — hierarchical priors for the explore band ([arXiv 1707.02038](#)).
10. **Chapelle et al., interleaved evaluation (TOIS 2012)** — the design of the blind DW test ([DOI](#)).

8. Risks you haven't considered

Policy.

- III.13's ban on analysis covers exactly what the system computes. ML training on API data breaches IV.2.a.1 regardless of scale.
- Keeping play history indefinitely conflicts with the storage clause.
- The five-day deletion duty requires a real disconnect path.
- An auto-queue loop may fall under the mimic-or-replicate clause (III.11).
- Spotify can revoke keys or require deletion with or without notice ([Terms](#)).
- If Premium lapses, the Dev Mode app stops working ([migration guide](#)).

Quota.

- Every Dev Mode app on the account shares one budget, so a second "probe controller" app competes with the poller ([July 2026](#)).
- Limits are unpublished and "subject to change" ([quota modes](#)).
- 4-second polling already produced a 429.
- Single-item GETs replace batch calls, so per-candidate Spotify lookups multiply.
- LB expects about 1 rps and is tightening against scrapers, so cache everything in DuckDB.

Feedback loops.

- Retraining weekly on confident-slot outcomes is the algorithmic confounding Chaney et al. describe.
- Thompson sampling on a fixed pool degenerates.
- A calibration target computed only from playlist plays chases its own output.
- If the playlist exceeds half your listening, organic exploration falls with it (Anderson's association).
- Opening DW outside the trial contaminates the familiarity filter.

Over-engineering.

- Building LightGBM, DPP, VW or LLM ranking before ESH exists.
- Depending on unmaintained obp or scikit-hubness instead of vendoring formulas.
- Keeping Mac capture alive through a private-framework workaround.
- Running Last.fm and ListenBrainz scrobbling in parallel.
- Building a queue controller whose best-case measurable gain is about 2 points.

The cheapest guard is a one-page pre-registration and a monthly "what can I delete?" review.

Conclusion

The central question was how to get more information about this user's taste. The notes answer that the user already has far more information than the system can use: years of export data, a tokenless recording-level co-listening graph, and per-play reason codes. Version 1's problems are about routing that information correctly, not acquiring more. Labels need to respect time. Weights need to respect exposure. Candidates need to respect hubness. Features need to come from sources the Spotify terms don't govern. Active probing is the most visible way to "learn faster", and it is also the one where the arithmetic is least forgiving at $N = 1$. That holds unless ESH reveals a handful of truly unknown clusters, and even then a playlist does the job more cleanly than the queue.

The second takeaway is that compliance and engineering point the same way. Moving model training onto the user-owned export, deriving features from MetaBrainz data, purging API-derived tables on a TTL, and polling adaptively each reduce policy exposure. Each also makes the system more robust to the next round of Dev Mode changes, which have arrived roughly every few months since November 2024.

PART I — Build log (7–8 October 2026)

Earlier status snapshots and the research-integration matrix (formerly “Part 0”), followed by the progress notes written during the 8 October work. Newer entries supersede older ones where they disagree.

0.1 Current state (end of session 2026-10-07)

- **Tracker** (`python -m radio.run`) is running on the Mac with the fixed completion logic; Windows PC not set up yet. Spotify app `personalized-spotify-rec`, Development Mode, token cached.
- **22 unit tests pass.** Real-API verified: recently-played, saves, follows, top items, single `GET /tracks/{id}`, `isrc:` search (17/20 exact), playlist create/add/replace/read/unfollow. Batch `GET /tracks` is 403.
- **Identity cache** now lives in its own file `radio_ids.duckdb` (moved out of the events DB so it never contends with the tracker's write lock, and to keep Spotify-derived events separate from open-data IDs). Last run: 162 tracks (plays + saves + top), 162 ISRCs, 71 MBIDs, 88 checked. MusicBrainz returned many transient 503s; the other ~74 unchecked rows are retries, not misses. Rerun `python -m radio.ids` to continue. Match rate so far (~65%) is the only measured Spotify→MBID rate (no published one exists).
- **Candidates:** ListenBrainz Labs similar-artists works with no token; first artist list produced but popularity-biased.
- **Not yet done:** restart tracker is not needed (already restarted post-fix); 4 hyphenated ISRCs in the old `track_ids` table of `radio.duckdb` are obsolete (the table moved); Extended Streaming History not yet requested; no Last.fm key / ListenBrainz token; Windows setup; Spotify Developer Terms primary text unread.
- **Secrets:** the Spotify Client Secret was pasted into chat. The user said they do not mind; rotating it is still advisable. It is deliberately not written in this file.

Progress since this section was written (2026-10-08)

- [done] [open]1 heartbeat: `radio/health.py` (poller rewrites `heartbeat.json` each loop; `python -m radio.health [--max-age 180] [--notify]` exits 1 when stale). **Takes effect after the tracker is restarted.** Supervisor config and Premium-lapse check still open.
- [done] [open]3 timezone: `radio/timeutil.py` (`RADIO_TZ` or system zone; stored data stays naive UTC; DST-safe).
- [done] [open]6 `QUOTA_EXCEEDED` reason is logged and written to the heartbeat. (Observed overnight: no 429s, only read timeouts.)
- [done] MusicBrainz resolver backs off exponentially and stops after 5 consecutive failures.
- [done] Bug fixed: OAuth token refresh had no timeout (`read timeout=None` seen in `radio.log`); `SpotifyOAuth` now gets `requests_timeout`.
- Identity cache: 164 tracks, 164 ISRCs, 123 MBIDs of 152 checked (**~81% match**, up from the partial 65%); 12 unchecked after MusicBrainz 503s.
- Tests: 25 passing. Overnight the Mac tracker kept running but logged only ~20 plays and several read timeouts (laptop sleep/wake gaps) — evidence for the always-on-host requirement.

- [done] Baselines + evaluation harness (`radio/evalkit/`): `most_pop`, `decayed_replay` (the bar to beat), `item_knn` (session co-occurrence); `rolling_origin` splits with a gap (no leave-one-out); `NDCG@10/Recall@10`; paired block bootstrap over days. Run: `python -m radio.evalkit.run [--novel]` (`--novel` scores only never-trained-on tracks, the discovery case). Tested on synthetic data (31 tests pass). **On real data it reports 0 evaluable days**: only 16 labelled plays from one day exist (13 `strong_pos`, mostly already-saved tracks), so no numbers are meaningful until the Extended Streaming History is imported or ~2-4 weeks of tracking accumulate. Heartbeat/OAuth-timeout fixes are live only after the tracker is restarted.
- [done] **First playlist written (2026-10-08)**: private "Personal Radio - Weekly Auto", id in `personal_radio/playlist_state.json` (`2rEcexSySLkdB6qIpbKh2U`), 30 picks = 21 confident ($p=1$) / 6 explore ($p\approx 0.44$) / 3 wildcard ($p=0.04$), order shuffled. Run: `python -m radio.playlist [--write] [--n 30] [--seed N]` (dry run by default; reruns replace items via `PUT /playlists/{id}/items`; never repeats a past pick). Every pick is logged with slot, propensity, reason and position in `radio_recs.duckdb` (picks table). Code: `radio/playlist.py`, `radio/candidates/picker.py`; 36 tests pass.
 - **Recommender-sourced tagging ([open]2) is now possible**: plays whose `context_uri` is `spotify:playlist:<playlist_id>` (from `picks`) came from the system. Not yet joined into labels/pipeline.
 - **Known weakness**: picks are mainstream hub artists (Ariana Grande, Bieber, Post Malone, The Weeknd, Lil Wayne). ListenBrainz `popularity/*` endpoints now return **401 and require a token**, so there is no popularity signal for the hubness penalty yet. Track choice per artist comes from a Spotify artist search (cached 1 day), so it favors popular songs and can include remixes. Needs the ListenBrainz token (also needed for `metadata/lookup`, `feedback`, `submit-listens`) and/or a Last.fm key.
 - Seeds were mostly saves/top tracks (25 seeds, 529 candidate artists, 105 candidate tracks).
- [done] **Seeds rebalanced toward liked songs (user request, 2026-10-08)**: `library_seeds` = per artist $\sqrt{\text{sum of saves decayed with a 365-day half-life}}$; top tracks $\times 0.25$ and recent plays $\times 0.5$ as minor boosts; `combine(top_n=40)` so all ~33 liked artists seed (82 of 102 saves have an artist MBID; library is exactly 102 tracks, no sync cap; `saves.ts` is Spotify's real `added_at`). Playlist re-written with 39 seeds (Steve Lacy, Clairo, Lana Del Rey, Mac Miller now appear). 37 tests pass. **Open question for the user**: all known artists are still excluded, so every pick is from an artist never played/saved; whether to add a "more from artists you like" slot (their unliked songs) is undecided.
- [done] **Chill playlist merged into the taste basis (user request, 2026-10-08)**: `taste_basis.json` = `{"playlists": {"Chill": "<playlist-id>"}}`; `radio/basis.py` rebuilds table `basis_tracks` (in `radio_ids.duckdb`) from liked songs (102) + configured playlists (Chill: 67), deduplicated; `library_seeds` reads it. **Local only: nothing was added to the user's Spotify Liked Songs**. Reading playlists needed new scopes `playlist-read-private` `playlist-read-collaborative` (re-consented). 40 seeds now (Frank Ocean joined). 39 tests pass.
- [warning] **Finding: the Weekly Auto playlist created earlier had vanished** (`GET /playlists/<id>` -> 404; `/me/playlists` listed only Chill), although the earlier `PUT .../items` had reported success. Cause unknown (deleted in the app, or the API accepted a write it did not keep) — unresolved, ask the user. `write_playlist` now verifies (exists, item count, listed in `/me/playlists`) and recreates on 404. New playlist id `51wLN6E7MlIJL9qjw7t7aU` (verified: exists, 30 items, in account). Candidate pool widened to 80 artists (145 candidate tracks). Lesson: never trust a 2xx from a playlist write without reading it back.

Research round 2 result received and verified (2026-10-08)

Report saved as `reports/Fix labels and candidates before probing anything.md` (full text in Part H).

Verdict on the user's question: **queue injection + genre probing = no for now, maybe later as a gated experiment** (81 independent probes per arm for ± 10 pts at $p=0.3$, 97–146 with within-session correlation; ceiling $\sim +2$ pp whole-playlist completion; ESH priors dominate for known genres). Prefer, in order: (d) passive Extended Streaming History, (a) better playlist arms, (c) optional taste quiz, (b) probes last, and via an "Up Next" playlist rather than the queue.

Claims I verified myself (same day):

- [done] POST `https://api.listenbrainz.org/1/popularity/recording` and GET `/1/popularity/top-recordings-for-artist/{artist_mbid}` return **200 with no token** now. Earlier the same day the same artist path returned **401** ("provide an Auth token") — so MetaBrainz gates these **intermittently**. Code must handle 401 gracefully and a free LB token is still worth getting. (Corrects my earlier statement that popularity "now requires a token".)
- [done] POST `https://labs.api.listenbrainz.org/similar-recordings/json` works tokenless; body must be a list: `[{"recording_mbids": ["<mbid>"], "algorithm": "session_based_days_9000_session_300_contribution_5_threshold_15_limit_50_skip_30"}]` (a bare string for `recording_mbids` gives HTTP 400). Returns `recording_mbid`, `recording_name`, `artist_credit_name`, `release_mbid`, `score`, `reference_mbid`.
- [done] Spotify Developer Policy text (fetched via a summarizing tool, so verify wording yourself): **III.11** "Do not build products or services that mimic, or replicate or attempt to replace a core user experience of Spotify"; **III.13** "Do not analyze the Spotify Content or the Spotify Service for any purpose,"; **III.14** "Do not use the Spotify Platform or any Spotify Content to train a machine learning or AI model". III.13's *analysis* ban is a bigger exposure for skip/complete labels than the ML clause; practical enforcement risk is key revocation. Mitigations in the report (§A11): store keys only, features from MusicBrainz/ListenBrainz/Last.fm, train only on the user-owned Extended Streaming History, TTL-purge API-derived tables, working disconnect-and-delete path.
- [unverified] Not yet verified: DW unreadable via API; Spotify artist `genres` in Dev Mode; whether existing app still has removed endpoints; skipped vs `reason_end` agreement (needs ESH).

Revised priority order (report's Stage 0, supersedes 0.3 where they conflict): (1) fix labels: a save only upgrades a label if it happens *after* the play (`saved_asof_play` is a feature, `saved_within_W` upgrades); replace sample weights with features and give auto plays weight $\geq$ user-started; (2) switch candidates to `similar-recordings` + mutual proximity + breadth damping + `pop-alpha` + calibrated re-rank; (3) within-artist track choice from LB top-recordings with release-group filtering and MMR, plus a capped (20–30%) deep-cuts slot from known artists; (4) adaptive polling on the Windows PC + hourly recently-played reconciler (cuts calls 5–10x, fixes sleep gaps); (5) compliance hardening; (6) ESH importer + empirical-Bayes shrunken rates; (7) blind team-draft-interleaved test vs Discover Weekly (copied by hand into an owned playlist). Defer LightGBM (needs $\sim 2,000$ – $5,000$ labelled plays), Thompson sampling (stage 2), DPP (skip; use calibration + MMR). **Bug to fix:** logged explore propensities `k*softmax` can exceed 1 (Plackett-Luce sampling without replacement); replace with Monte Carlo replay of the whole generator (10k–50k runs) and log counts/R.

0.2 Research integration matrix

Question asked: was everything in the research folder considered and integrated into the plan? Answer after reading all files: the report-level conclusions were integrated; several note-level details were not. Status below is honest as of this review.

Legend: [done] done/verified · [planned] in the plan, not built · [open] **was missing from the plan; added to the backlog in 0.3**

Report's eight changes

#	Recommendation	Status
1	Live API smoke test	[done] playlist writes, <code>isrc</code> : search, top/saved/follows verified (see Part B §B). Batch <code>GET /tracks</code> 403 discovered.
2	Decayed-replay and item-kNN baselines first	[planned] planned, not built
3	Log propensities; randomized exploration slice	[planned] planned, not built (needs playlist writer first)
4	Mirror plays to ListenBrainz; MBID/ISRC as primary key	[done] key layer built (<code>ids.py</code>); [planned] mirror not built (see [open] 4)
5	Rolling-origin time splits + blind novelty-adjusted A/B	[planned] planned, not built
6	Beta-Bernoulli Thompson sampling over clusters/artists	[planned] planned, not built
7	DPP/MMR re-rank + calibration floor + narrowing metrics	[planned] planned, not built
8	LLM off the hot path (schema steering, grounded explanations)	[planned] planned, not built

Details found in the notes that were NOT in the condensed report/plan

[open]	Gap	Source note	Why it matters
1	Heartbeat row on every poll + stale alert; Premium-lapse health check; supervisor (launchd/NSSM). A sleeping PC or lapsed Premium silently stops data/app.	llm_and_sys ems §3, §5; prior_art	Data gaps corrupt labels and evaluation
2	Tag each play recommender-sourced vs organic (Yambda <code>is_organic</code> analog): record the playlist ids the system writes (<code>recs</code> table) and mark plays whose <code>context_uri</code> matches	representatio ns_diversity Q4; modeling_me thods	Needed to correct exposure bias and to measure narrowing
3	Timestamps are naive UTC; time-of-day/day-of-week features need the user's local timezone (or inference from scrobbles)	representatio ns_diversity Q3	Silent bug source for context features
4	Direct POST /1/submit-listens from the poller (listen counts after min(half the track, 4 min)); multi-scrobber is a Node app, has no Windows SMTC source, and cannot see iPhone plays except via the Spotify API	llm_and_sys ems §3; prior_art	Redundant record + feeds ListenBrainz CF
5	Popularity must come from ListenBrainz/Last.fm — Spotify removed track/artist <code>popularity</code> and artist top-tracks	prior_art; legal_terms	The hubness penalty and per-artist recording picks need open sources, not Spotify
6	Handle 429 reason: QUOTA_EXCEEDED explicitly; quota is shared across all Client IDs on the developer account, so ad-hoc scripts can starve the live tracker (observed once)	legal_terms; llm_and_sys ems §4	Tracker uptime

[open]	Gap	Source note	Why it matters
7	Randomize playlist order and log positions so position bias can be estimated cheaply	modeling_methods (exposure)	Improves propensity/bias estimates
8	Evaluation specifics: global temporal split with a gap, paired bootstrap over days/sessions (not plays), permutation test or mixed-effects for the A/B, blind arm design (30 tracks/arm, exclude known tracks, shuffle, hide source), pre-registered metric	modeling_methods (eval)	The notes are more specific than the report
9	Spotify Policy details beyond the ML clause: attribution/link-back for metadata and covers (II.4), personal data deleted on disconnect (I.2), cache only what is "strictly necessary" (IV.3.b), no "compilations or databases" (IV.3.a), playlist metadata moves only at user direction (III.9)	legal_terms	Our long-lived <code>plays</code> table holds Spotify-derived names/URLs: gray zone; keep minimal and read the primary text
10	Experiment tracking = a <code>runs</code> table (config hash, metrics, date); skip MLflow/feature stores	llm_and_systems §5	Cheap reproducibility
11	Private Session / offline / multi-device handoff behavior is unknown — needs one empirical test each	llm_and_systems §3	Defines blind spots in the data
12	Source tags/tag richness: Last.fm tags are richer than MusicBrainz tags; needs a Last.fm API key; tag embeddings are the default no-audio representation	representations_diversity Q2	Taste clusters and tag-overlap features
13	Extended-history ISRC gap: history rows carry no ISRC; with batch endpoints gone, resolving a large import costs ~1 Spotify call/track (rate-limit and 429 risk)	open_data_datasets §3–4	Plan imports in throttled batches
14	Minimum-data guidance: a few thousand plays over 10+ weeks orders only large effects (kNN vs random); small differences need far more data	modeling_methods (eval)	Sets expectations for LightGBM-vs-baseline

Contradictions between sources, and their resolution

Question	Resolution
Was <code>POST /me/playlists</code> removed or the replacement?	Resolved by live test: it works; <code>legal_terms</code> note misread it as removed. New paths: <code>POST /me/playlists</code> , <code>/playlists/{id}/items</code> ; responses carry the track under <code>item</code> .
<code>isrc</code> : search after Feb 2026	Resolved: works, <code>limit=10</code> , 17/20 exact hits (2 empty, 1 rate-limited). Hyphenated ISRCs must be normalized.
Do saved/follows/top sync survive the Feb 2026 removals?	Resolved: yes, all three work.
Is batch <code>GET /tracks</code> available?	No (403) for this Development Mode app — consistent with the notes.
ListenBrainz Labs similarity endpoint	Notes could not verify paths. Verified: <code>labs.api.listenbrainz.org/similar-artists/json?artist_mbids=...&algorithm=session_based_days_9000_session_300_contribution_5_threshold_15_limit_50_skip_30</code> works with no token.
Refresh-token lifetime	Dashboard shows 180 days (earlier only third-party sources said 6 months). Re-login due ~early April 2027.
Everything else (Last.fm rate limit, Yambda license, Essentia license, LFM-2b/MPD availability, ListenBrainz dump license, private-session behavior, Spotify ML clause primary text)	Still open , see Part C "Contradictions" table and the notes' Gaps sections.

0.3 Prioritized backlog (supersedes the roadmap in Part A where they conflict)

1. **Identity**: finish MBID resolution (rerun `python -m radio.ids`; add backoff on MusicBrainz 503); resolve ISRC/MBID for saved and top tracks (done in the cache, retry pending); report match rate by category (remaster/live/regional) — [open]13.
 2. **Operational safety**: heartbeat + stale alert, `QUOTA_EXCEEDED` handling, supervisor config — [open]1, [open]6. Windows PC setup with `install_windows_task.ps1`.
 3. **Data correctness**: store local timezone for plays — [open]3; recommender-sourced tag via `recs` table — [open]2.
 4. **Candidates**: hubness/popularity penalty using ListenBrainz/Last.fm popularity — [open]5; per-artist recordings from ListenBrainz/Last.fm; Last.fm key + tags — [open]12; seeds from saves/top via the cached IDs.
 5. **Baselines + evaluation harness** (decayed replay, implicit item-kNN, rolling-origin split with gap, paired bootstrap over days) — [open]8, [open]14. Request the Extended Streaming History now (up to 30 days) so this has data.
 6. **First nightly playlist**: Spotify URI resolution via `isrc:` search → heuristic scorer → write "Weekly Auto" (private, replace items), randomized order with logged positions and sampling probability per pick — [open]7.
 7. **Thompson sampling over clusters/artists (~10%)**, DPP/MMR + calibration floor, narrowing metrics.
 8. **ListenBrainz mirror via direct `submit-listens`** — [open]4.
 9. **LightGBM ranker**, only if it beats baselines on the harness; blind A/B vs Discover Weekly scoring unfamiliar tracks only.
 10. **Stretch**: session layer + just-in-time queue, LLM steering/explanations, FastAPI dashboard, tag embeddings/Essentia, ListenBrainz-space pretraining. Defer sequence models, HSTU, Mamba, RL, audio foundation models.
 11. **Standing tasks**: read the Spotify Developer Terms/Policy primary text ([open]9); empirical tests of Private Session/offline/handoff ([open]11); rotate the Client Secret.
-

Progress 2026-10-08 (late): Stage 0 items 1-3 built (Claude Code)

- [done] **Labels** (`radio/pipeline.py`): a save upgrades a play's label only if it came *after* the play within `SAVE_WINDOW` (7 d); an earlier save is the feature `saved_asof_play`. `saves` is now read as `{uri: added_at}`. All sample weights are 1.0 (autoplay no longer down-weighted); `start_kind/intent` are features on each row.
- [done] **Propensities** (`radio/candidates/picker.py`): logged propensity = Monte Carlo inclusion probability from 10,000 replays of the whole generator (always in (0,1]); old `k*softmax` removed. Slots are now confident 50% / deepcut 20% (capped) / explore 20% / wildcard 10%; with no deep-cut pool the share reverts to confident.
- [done] **Candidates** (`radio/candidates/recordings.py`): LB `similar-recordings` per seed recording; score = sum of seed weight x normalised similarity x sqrt(mutual-proximity), / sqrt(#seeds) (breadth damping), x popularity^{-0.3} (LB `total_user_count`; skipped gracefully on 401). Then MusicBrainz lookup drops remix/live/extended/karaoke titles and Live/Remix/Compilation/DJ-mix/Demo release groups, keeps one track per release group, and resolves to Spotify by exact ISRC. Candidates by artists you already like become the deepcut slot. `python -m radio.playlist --artist-level` keeps the old path for comparison.
- Coverage caveat: only 39 of 90 seed recordings had neighbours; a candidate with no neighbour list of its own gets a neutral mutual-proximity factor (0.5), not zero. Calibrated Head/Mid/Tail re-rank (report step 4 of A1) and `top-recordings-for-artist` (401 intermittently) are **not** built.

- Offline run (stubbed Spotify) gave 87 candidates (69 new-artist, 18 deep cut), led by Saba, Oliver Tree, NIKI, keshi, Knucks rather than Ariana Grande/Bieber. The real Spotify resolve + `--write` has not been run yet.
- 49 tests pass. Not done: items 4 (poller on PC), 5 (compliance/TTL purge), 6 (ESH import).

Progress 2026-10-08 (night): playlist written; Stage 0 item 4 built (Claude Code)

- [done] **Playlist written** with the new pipeline: id `<playlist-id>` (30 picks: 15 confident / 6 deepcut / 6 explore / 3 wildcard, propensities all ≤ 1 , logged in `radio_recs.duckdb`). Deep cuts got an extra popularity penalty (`DEEP_EXTRA_ALPHA` 0.5). **The previous playlist 51wL... returned 404 only ~2.5 h after it was written and verified** (second time a Weekly Auto playlist vanished). Cause unknown; user asked whether they deleted it. If it recurs untouched, stop relying on a saved id.
- [done] **Adaptive polling** (`radio/poller.py`): playing mid-track 4 s, last 8 s of a track 1.5 s, idle/paused ladder 10, 10, 20, 30, 60 s, then 120 s after 60 idle polls (about an hour). Idle calls drop from about 8,600 a day to about 750 (simulated with the real code).
- [done] **Reconciler** (`radio/sync.py: backfill_recent/missing_recent`): hourly and immediately after any gap ($> \max(120 \text{ s}, 3 \times \text{planned delay})$), run *after* the poll's own plays are stored. Dedupes by time window per track (not "newer than last play"), so the poller and the reconciler never double-count. Gaps are logged as `raw_events` kind `gap`. Reconciled rows have `source='recent'`, `end_reason='other'` (exposure only, no label).
- Tests: 55 pass (`tests/test_polling.py` added); fake-clock loop smoke test confirmed the backoff, gap event and reconcile ordering. **Takes effect when the tracker is restarted** (`python -m radio.run`); the Windows PC is still not set up.
- Next: item 5 (compliance: TTL purge of Spotify-derived tables, disconnect-and-delete path), item 6 (ESH import when it arrives).

Progress 2026-10-08 (night): Stage 0 item 5, retention + disconnect (Claude Code)

- [done] `radio/retention.py: purge` deletes `raw_events` older than 30 days (`RAW_EVENTS_DAYS`) and Spotify-content cache files older than 24 h. Spotify cache keys (`sp-*`) now live in `.api_cache/spotify/`; legacy Spotify cache files in the cache root are recognised by content and removed. Runs at tracker startup and after every 6-hourly sync (`radio/run.py`).
- [done] `python -m radio.retention (report), --purge, --disconnect --yes` (deletes plays, saves, follows, top items, raw events, `track_ids`, `basis_tracks`, the `picks` log, all caches and the OAuth token; says to also revoke the app at `spotify.com/account/apps`). Needs the tracker stopped (DuckDB write lock).
- **Deliberate defaults, not legal advice:** plays/saves/follows/top items/labels are kept (they are the user's own listening record and what labels need); only raw payloads and Spotify content caches expire. The wider question (III.13 "do not analyze the Spotify Content") is unresolved; the primary text still needs to be read by the user. Whether to also expire `plays / saves` is open. DuckDB files do not shrink after DELETE; `--disconnect` runs CHECKPOINT only.
- 58 tests pass. Next: item 6 (Extended Streaming History import, once the export arrives; `radio/history_import.py` exists but is untested on real data).